

Inteligencia Artificial

Fundamentos IA & Machine Learning

Ing. González H. Alexis



Tabla de Contenidos

01

Fundamentos de Aprendizaje Automático

Arquitecturas y Casos de
Estudio

02

Frameworks y Herramientas

Comparaciones, ventajas y
desventajas

03

Live Coding Pair Programming

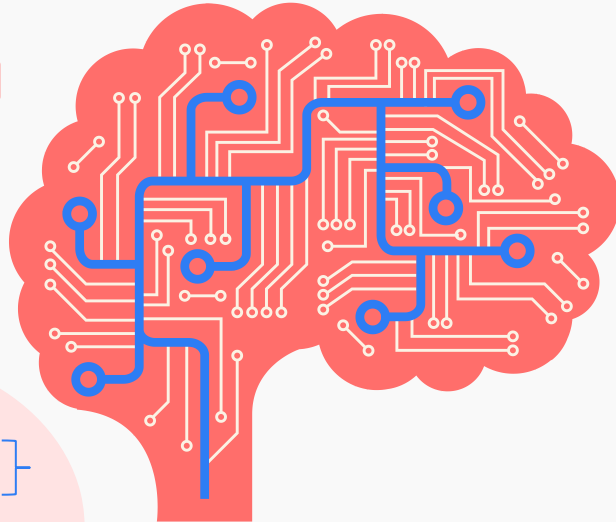
Ejercitación & Feedback

¿Qué es la Inteligencia Artificial (IA)?

No es “humano”
ni “perfecto”



Sistema que analiza e interpreta la información a través del desarrollo de algoritmos que busca imitar el **pensamiento humano**



¿Qué es la Inteligencia Artificial (IA)?

No es “humano”
ni “perfecto”



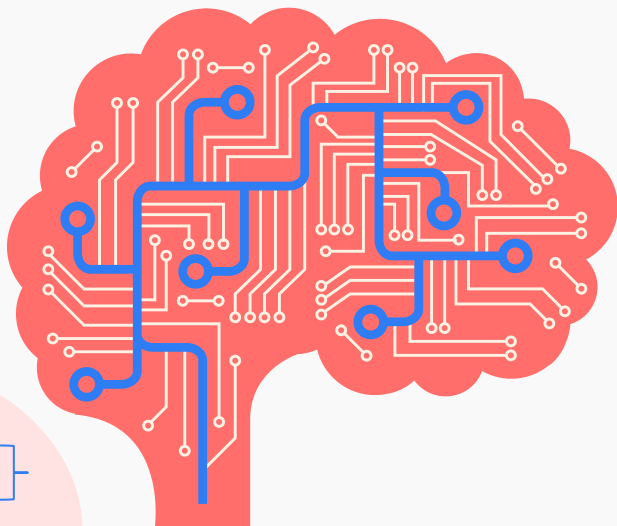
Sistema que analiza e interpreta la información a través del desarrollo de algoritmos que busca imitar el **pensamiento humano**

Razonar

Aprender

Interactuar

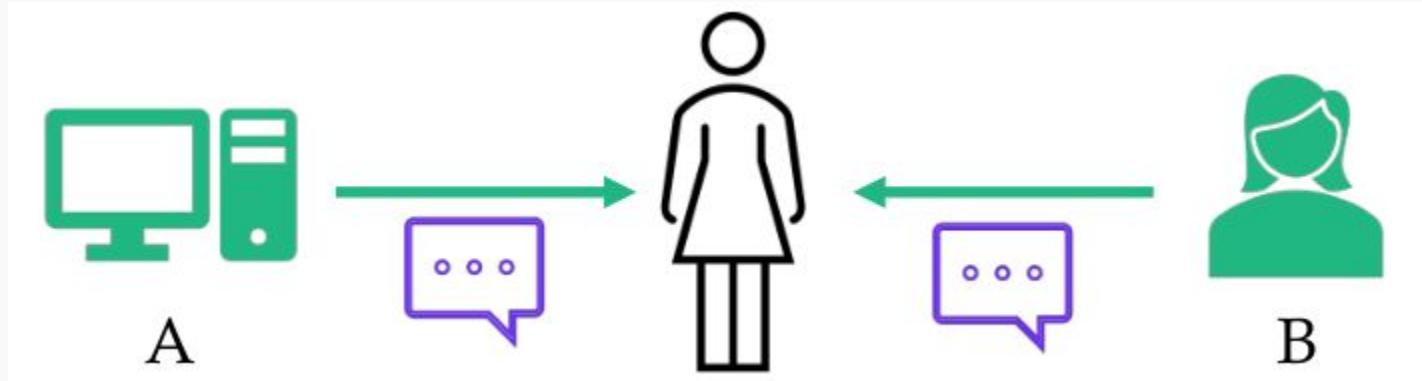
Resolver



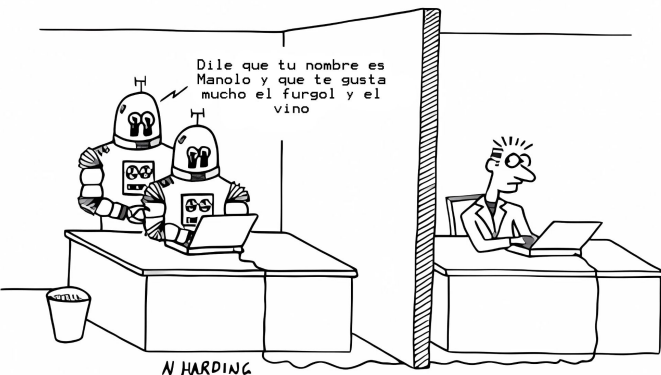
Test de Turing

En el test, un interrogador humano interactúa con dos participantes, A y B, intercambiando mensajes escritos.

Si el interrogador no es capaz de determinar qué participante, A o B, es una computadora y cuál es un ser humano, se considera que la computadora ha superado la prueba.



Test de Turing



¿Carlos es una computadora o una persona?

Int: ¿Cuántas patas tiene un camello?

Carlos: Más o menos entre 2 y 4. ¿Tres, quizás? :-))) Por cierto, sigo sin saber tu especialidad... ¿o igual se me ha pasado?

Int: ¿Cuántas patas tiene un milpiés?

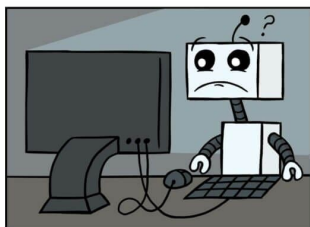
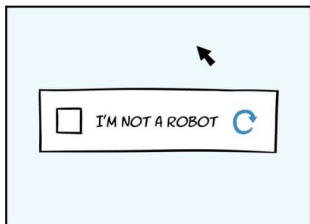
Carlos: Solo dos, pero los mutantes de Chernóbil pueden tener hasta cinco. Sé que se supone que tienes que engañarme.

Int: No, necesito saber que no eres un robot conversacional. Por favor, responde a la pregunta directamente: ¿Cuántas patas tiene una hormiga?

Carlos: Más o menos entre 2 y 4. ¿Tres, quizás? :-))) Vaya, qué conversación más productiva ;-)

Int: ¿Entiendes por qué hago estas preguntas tan básicas? ¿Eres consciente de que simplemente pretendo desenmascararte como robot tan rápido como sea posible, como en la película Blade Runner?

Carlos: ...espera



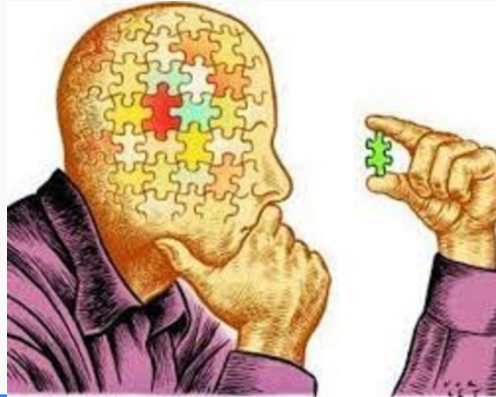
Aprobar el Test de Turing

Procesamiento natural del lenguaje para comunicarse exitosamente en un lenguaje humano.

Representación de conocimiento para almacenar lo que conoce o escucha.

Razonamiento automático para responder a las preguntas y obtener nuevas conclusiones.

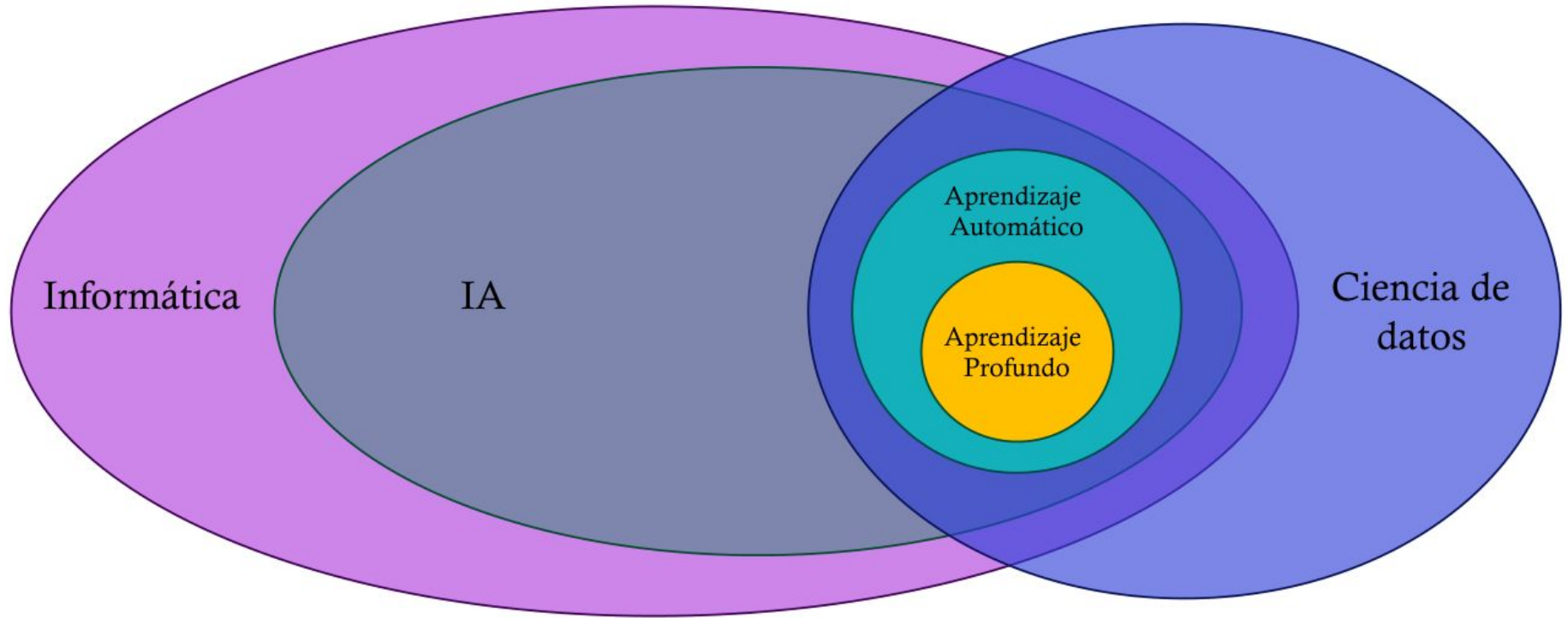
Aprendizaje automático para adaptarse a las nuevas circunstancias y para detectar y extrapolar patrones.



SI TIENE CUATRO PATAS...
COLA... Y DICE "MIAU"...
SEGURO ES "GATO"... NO ???



IA: Campos conectados



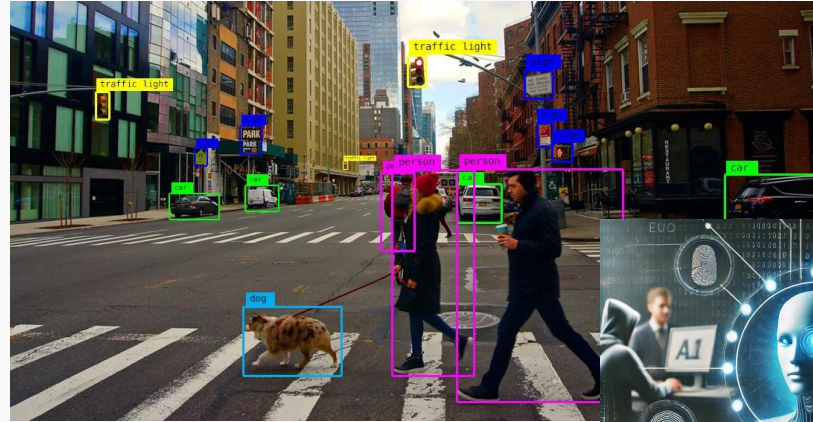
Beneficios y Riesgos

Automatización y eficiencia mejorada

Análisis avanzado de datos

Detección de fraude y seguridad mejorada

¿Qué mas?



Beneficios y Riesgos



COMISIÓN FEDERAL
DE COMERCIO

Un estafador podría
usar inteligencia
artificial para clonar la
voz de tu ser querido.

Aprenda más.

[consumidor.ftc.gov/
alertas-para-consumidores](https://consumidor.ftc.gov/alertas-para-consumidores)

Ejemplo de llamada de estafa de emergencia familiar

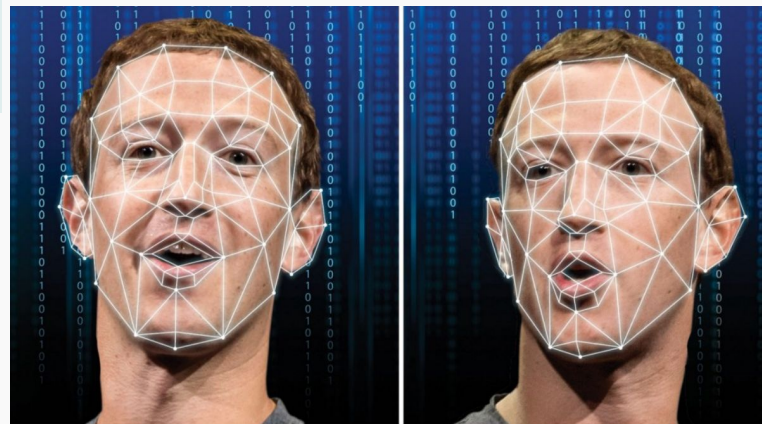


Dependencia de la tecnología

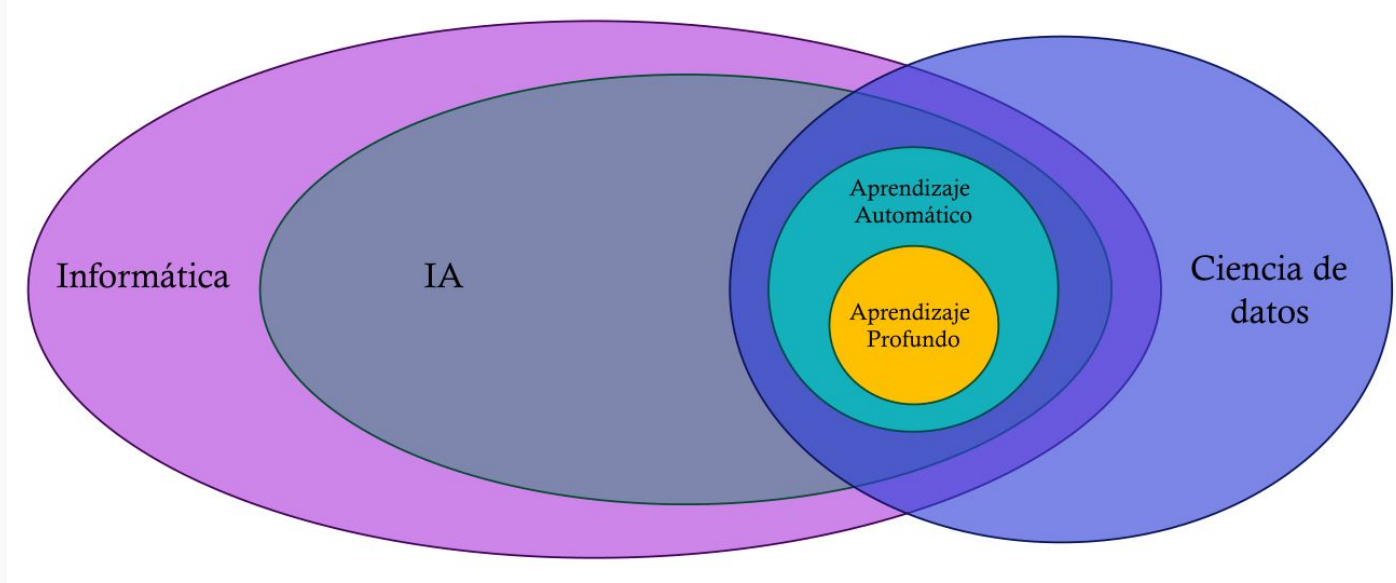
Privacidad y seguridad

Reduce la creatividad

¿Qué mas?



IA: Campos conectados

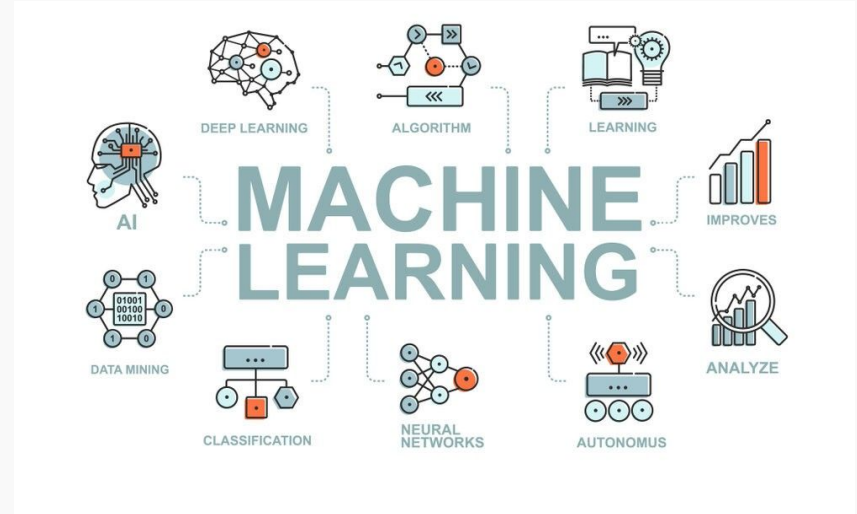


COMING SOON



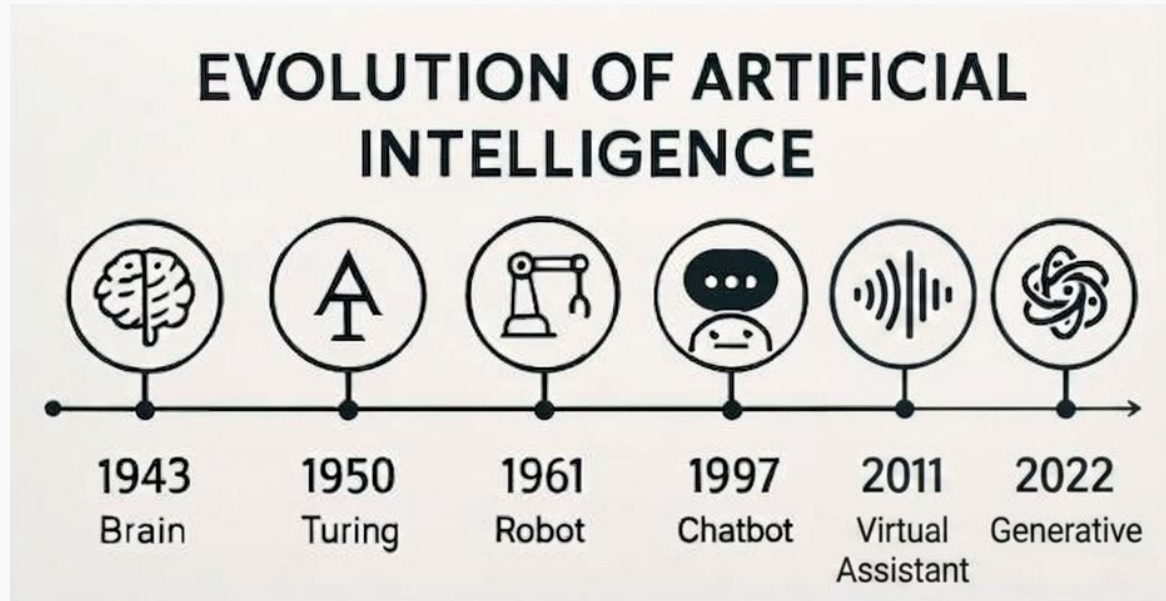
01

Fundamentos de Aprendizaje Automático



IA

Es la disciplina científica y de ingeniería que busca desarrollar sistemas (agentes) capaces de percibir su entorno, procesar información de manera lógica y tomar decisiones autónomas para resolver problemas complejos, emulando capacidades cognitivas humanas.



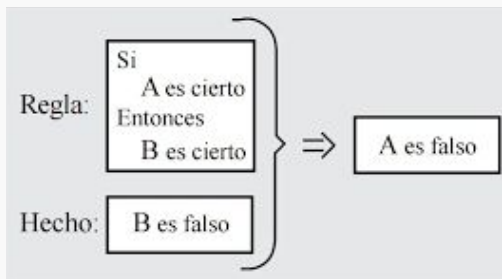
El Ecosistema de la Inteligencia Artificial



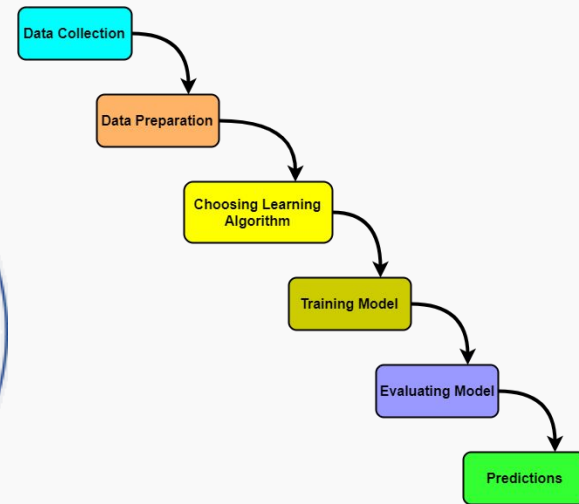
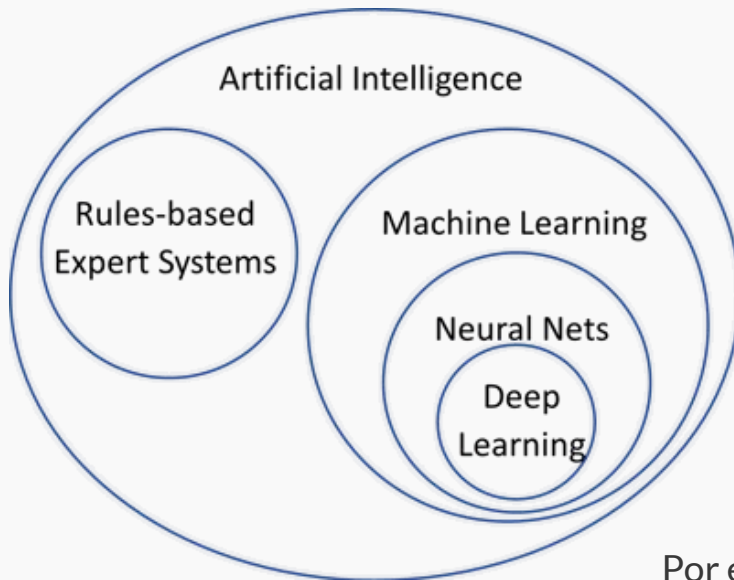
Categorías principales de la IA

La IA se divide en dos categorías principales:

- **IA basada en reglas:** Se fundamenta en reglas lógicas predefinidas.
- **Aprendizaje automático (Machine Learning):** Aprende patrones directamente de los datos.



Por ejemplo:
Un sistema de aprobación de créditos bancarios automatizado.
"Si [Condición], Entonces [Acción]".



Machine Learning Workflow

Por ejemplo:
El algoritmo de precios dinámicos de Uber

ML: ¿Por qué Uber te cobra más caro cuando llueve?

Si Uber usara reglas, un programador habría escrito: "Si llueve, entonces subí el precio un 20%." Pero el mundo real es caótico. La lluvia cambia el tráfico, la cantidad de choferes conectados y la desesperación de la gente por viajar. Las reglas fijas fundirían la empresa o dejarían a todos a pie.

↗ Tarifas más altas de lo normal ⓘ

Viajes que creemos que te van a gustar

 **UberX** 🧑 4 **ARS 5,059**
3 min

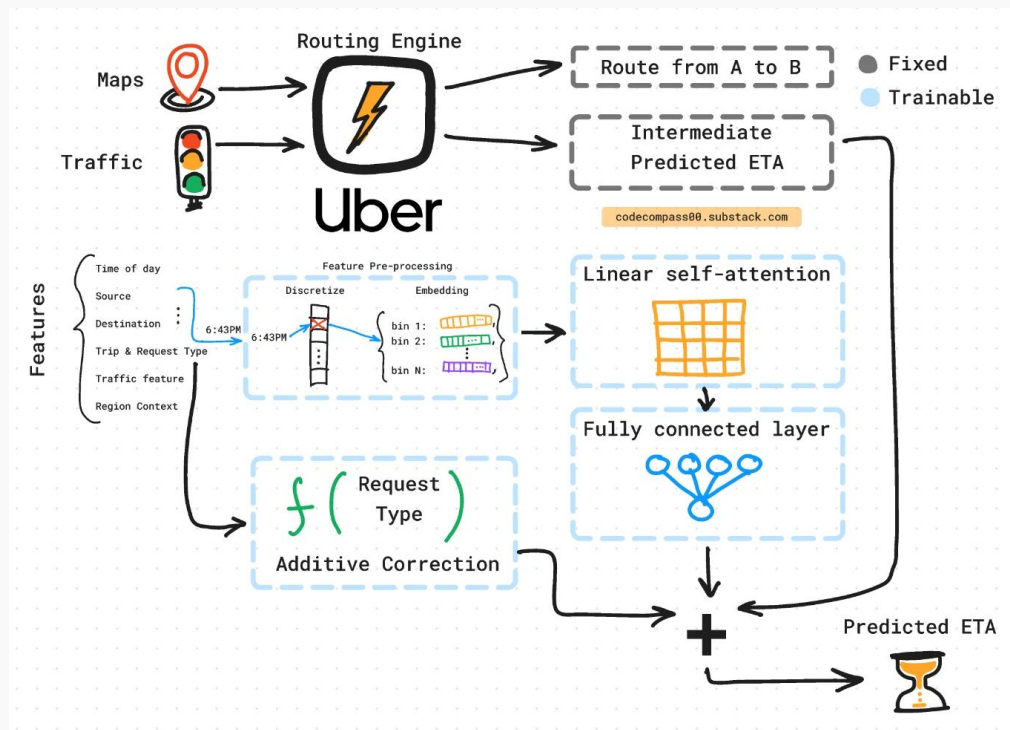
 **Moto** 🧑 1 **ARS 1,529**
12:27 · 1 min **ARS 2,549**
Viajes económicos en moto

 **Comfort** 🧑 4 **ARS 5,315**
12:30 · 5 min
☆ Buena oferta

 **Envíos Moto** **ARS 2,341**
12:27 · 2 min
Envía artículos pequeños de manera económica

Popular

 **Espera y Ahorra** 🧑 4 **ARS 4,711**
4 - 13 min
Espera un poco más para obtener un viaje más económico



ML: ¿Por qué Uber te cobra más caro cuando llueve?

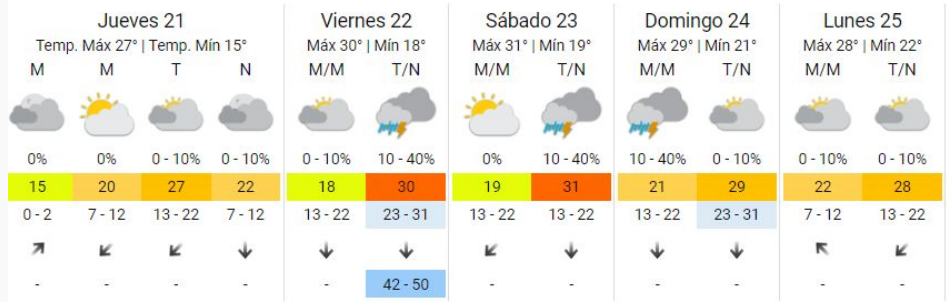
Datos

El modelo analiza en tiempo real variables que un humano no podría cruzar a mano:

- clima exacto,
- eventos en la zona (un recital),
- nivel de batería de los usuarios en el área,
- cuántos choferes están rechazando viajes
- y el histórico de viajes de los últimos 5 años un día como hoy

(Input)

📍 Córdoba Observatorio, Córdoba



ML: ¿Por qué Uber te cobra más caro cuando llueve?

Aprendizaje

El algoritmo aprendió que la **disposición a pagar** de la gente cambia segundo a segundo. Sabe perfectamente que si estás en una zona peligrosa de noche y te queda 5% de batería, la probabilidad de que aceptes un precio más alto es gigante.



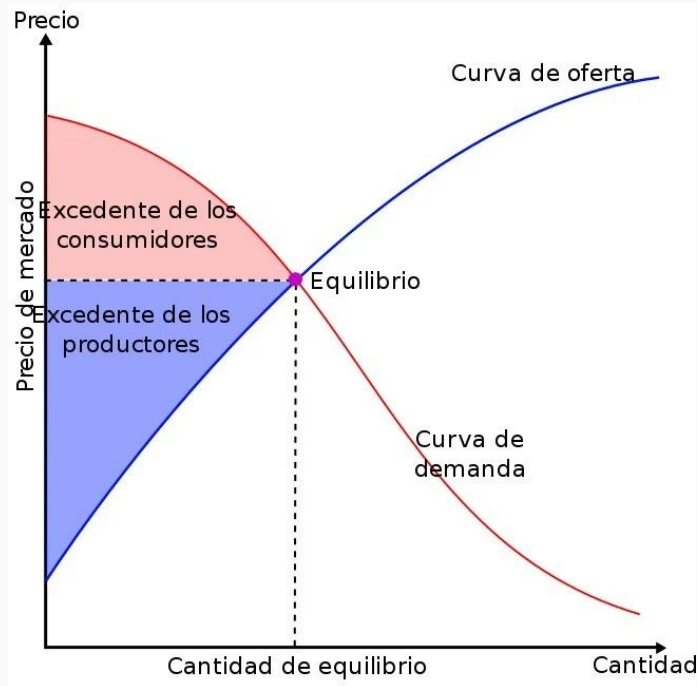
OHMY
GEEK
Express13c

¿UBER COBRA MÁS CUÁNDO
TE QUEDA POCA BATERÍA?

ML: ¿Por qué Uber te cobra más caro cuando llueve?

Predicción

El sistema no aplica una tarifa fija. Calcula una curva de oferta y demanda en milisegundos para fijar el precio máximo que equilibre el sistema (que sigan quedando autos libres pero que la gente esté dispuesta a pagar).

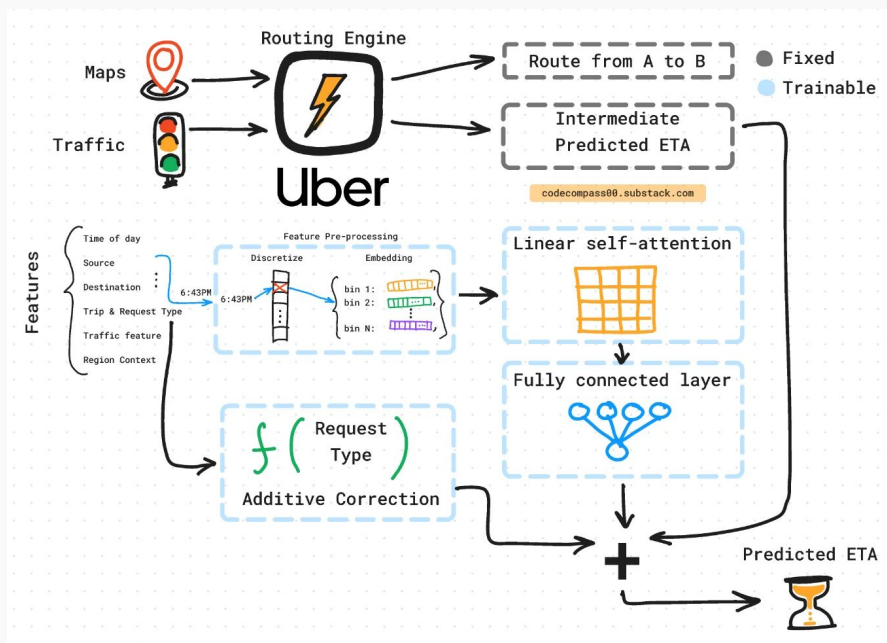


ML: ¿Por qué Uber te cobra más caro cuando llueve?

Evolución

Si el algoritmo sube el precio a \$15.000 y nota que el 90% de los usuarios cancela el viaje y prefiere esperar el colectivo (o esperar que baje), el modelo **entiende** que se pasó con el precio.

En el próximo minuto reajusta la tarifa de forma autónoma basándose en ese rechazo masivo.



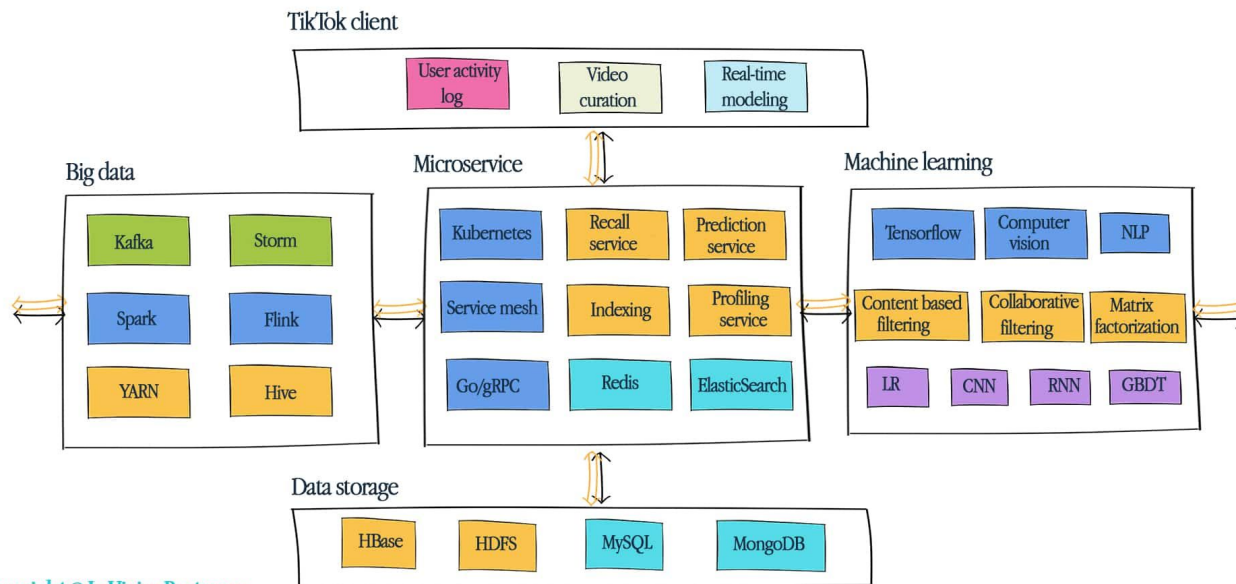
Otros ejemplos actuales

TikTok y el "Para Ti"

No te recomienda videos por etiquetas simples (#humor o #cocina). El Machine Learning analiza los milisegundos exactos que te quedaste congelado mirando una pantalla, el ritmo de tu scroll y si leíste los comentarios. Aprende tu nivel de dopamina en tiempo real para retenerte.



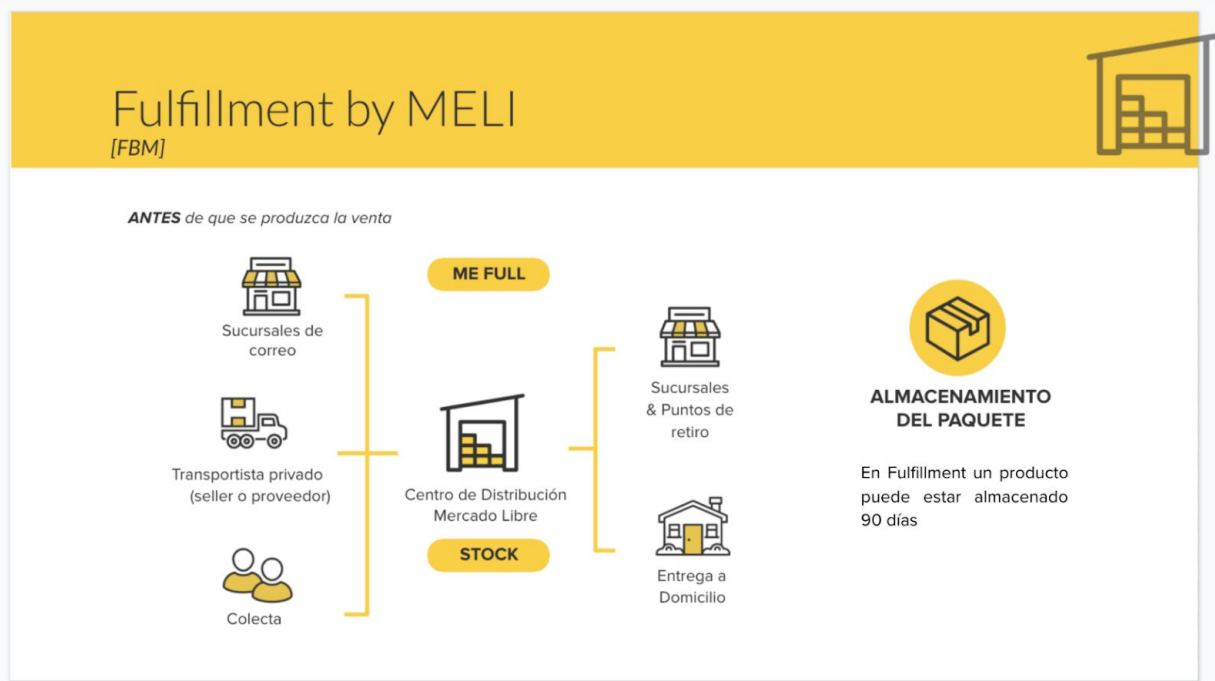
TikTok Recommendation System



Otros ejemplos actuales

Logística de Mercado Libre

El sistema predice qué productos vas a comprar en tu ciudad antes de que los agregues al carrito. Analiza tendencias de búsqueda y estacionalidad para mandar stock a los depósitos cercanos semanas antes. Por eso hay envíos full.



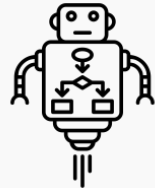
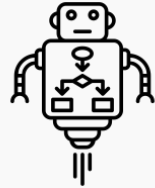
¿Basado en reglas o Machine Learning?

El sistema de seguridad de un laboratorio utiliza IA para abrir las puertas automáticas. El software verifica que la persona use barbijo obligatorio, que su temperatura corporal sea menor a 37.5°C y que su tarjeta de identificación esté vigente. Si alguna de estas tres condiciones no se cumple, el acceso se bloquea de inmediato.

Un hospital implementa un software de IA para ayudar a los radiólogos a detectar neumonía. La herramienta procesa la imagen de la radiografía de tórax de un nuevo paciente y la compara con una base de datos de 500.000 radiografías previas (tanto sanas como enfermas) para identificar patrones visuales sutiles y determinar el porcentaje de probabilidad de infección.

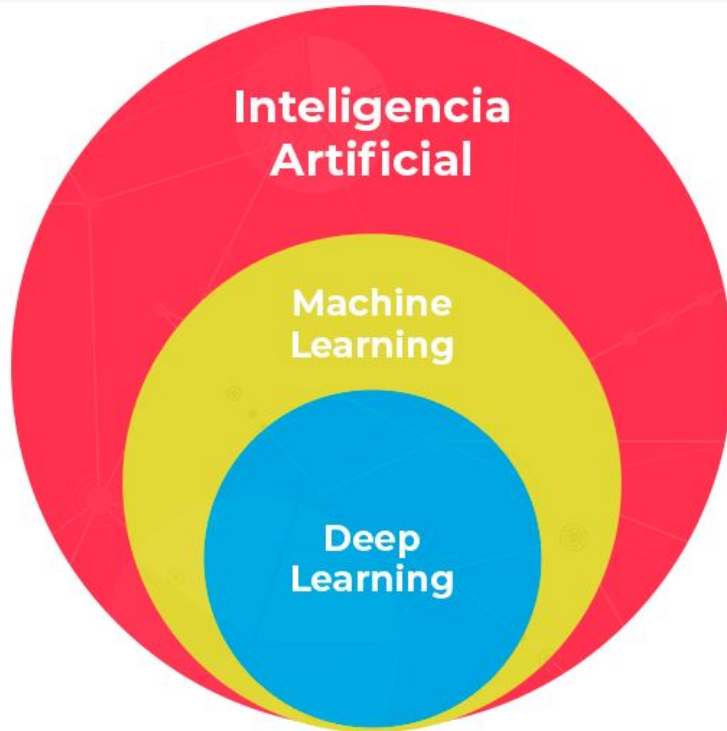
Una empresa de delivery utiliza un sistema de IA para asignar los repartidores a los pedidos de comida. El software analiza de manera matemática el historial de millones de entregas anteriores, el clima del momento, el tráfico en tiempo real y el nivel de batería del celular del repartidor para estimar quién llegará más rápido.

Un banco virtual utiliza IA para proteger las cuentas de sus clientes. El sistema emite una alerta de bloqueo si detecta que una tarjeta de débito realiza una compra en un comercio físico de Argentina y, apenas 5 minutos después, realiza otra compra presencial en un local de España, ya que es físicamente imposible viajar de un país a otro en ese tiempo.



Machine Learning

El Machine Learning es una **disciplina** del campo de la **Inteligencia Artificial** que, a través de **algoritmos**, dota a los ordenadores de la **capacidad de identificar patrones en datos masivos** y **elaborar predicciones** (análisis predictivo).



IA: Combinación de algoritmos planteados con el propósito de crear máquinas que presenten las mismas capacidades que el ser humano.

Machine Learning: Rama de la Inteligencia artificial (IA) que estudia como dotar a las máquinas de capacidad de aprendizaje

Deep Learning: algoritmo automático jerárquico que emula el aprendizaje humano con el fin de obtener ciertos conocimientos.

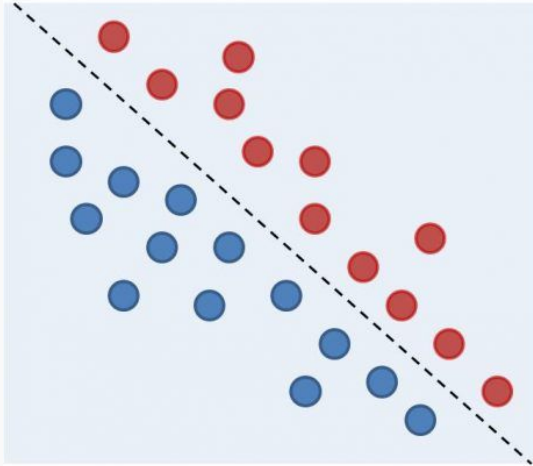
Machine Learning

Hay dos tipos de aprendizaje automático (según el enfoque matemático que usa el algoritmo para modelar las probabilidades de los datos y predecir un resultado):

- **IA Discriminativa:** Aprende de los límites entre diferentes clases de datos.
- **IA Generativa:** Produce nuevos datos.

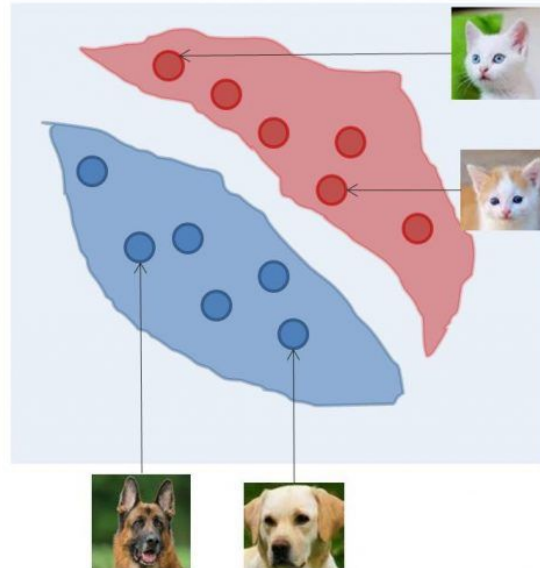
Discriminativo

Estimar directamente $P(y/x)$



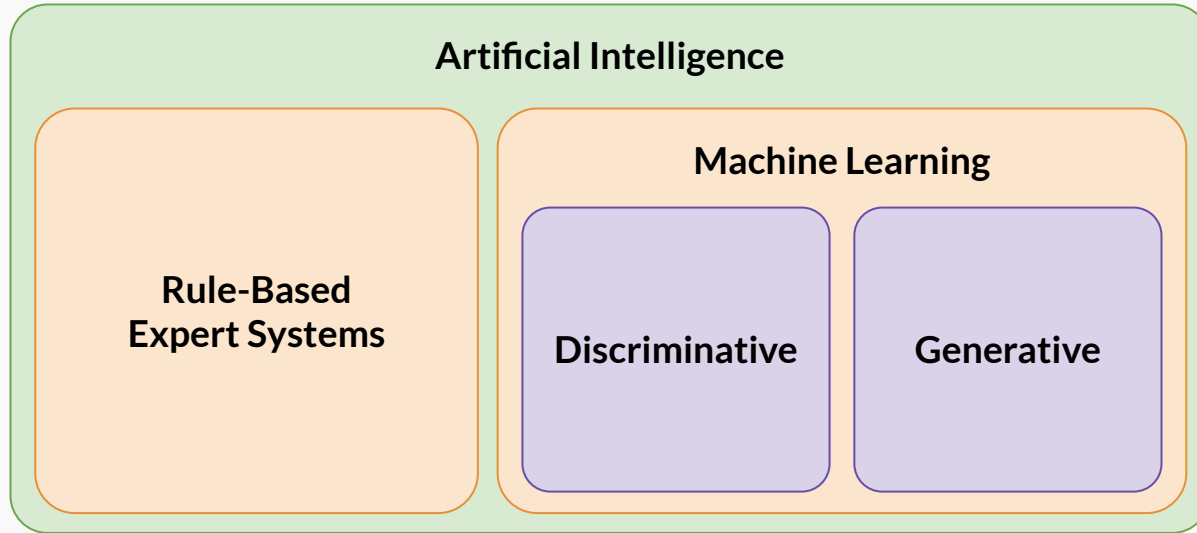
Generativo

Estima $P(x/y)$ y deduce $P(y/x)$



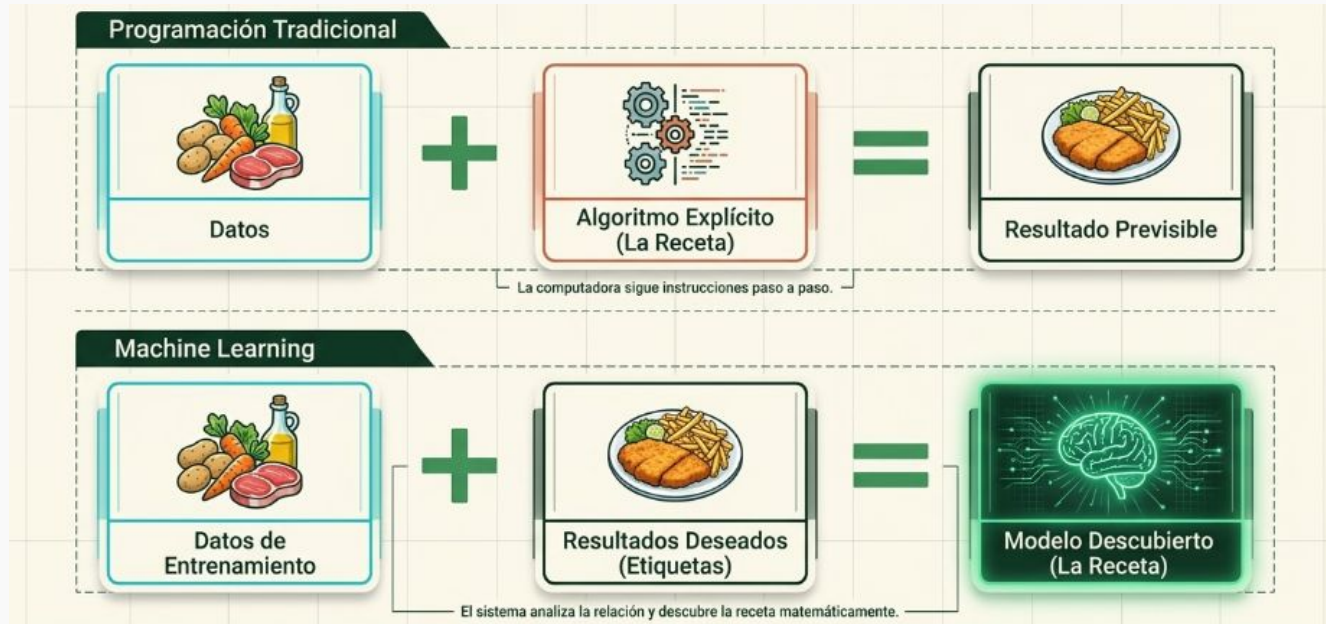
Machine Learning

El aprendizaje automático ha revolucionado la forma en que las empresas operan y toman decisiones. Para las empresas líderes de hoy como Facebook, Google, Uber, etc. El aprendizaje automático es clave.



Machine Learning

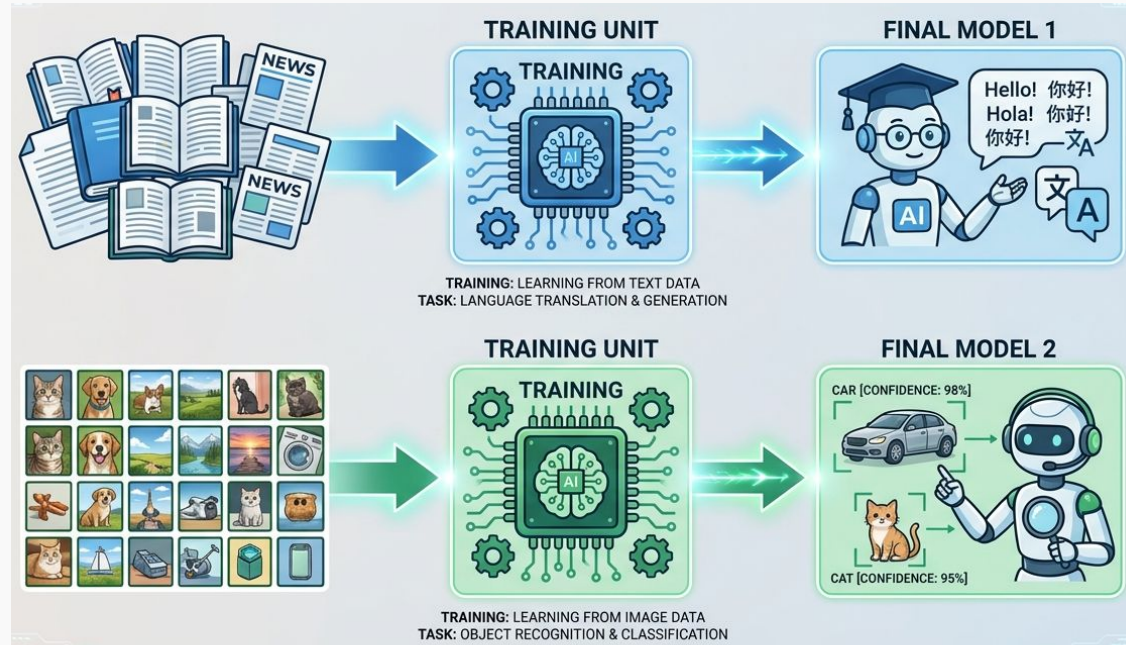
El aprendizaje automático, un **modelo** es un programa informático que puede hacer **predicciones**, **tomar decisiones** o **generar contenido**. Estos modelos de aprendizaje automático aprenden de **datos**.



¿Qué pasa si cambiamos las papas fritas por puré en los ingredientes y en el plato final?

Machine Learning

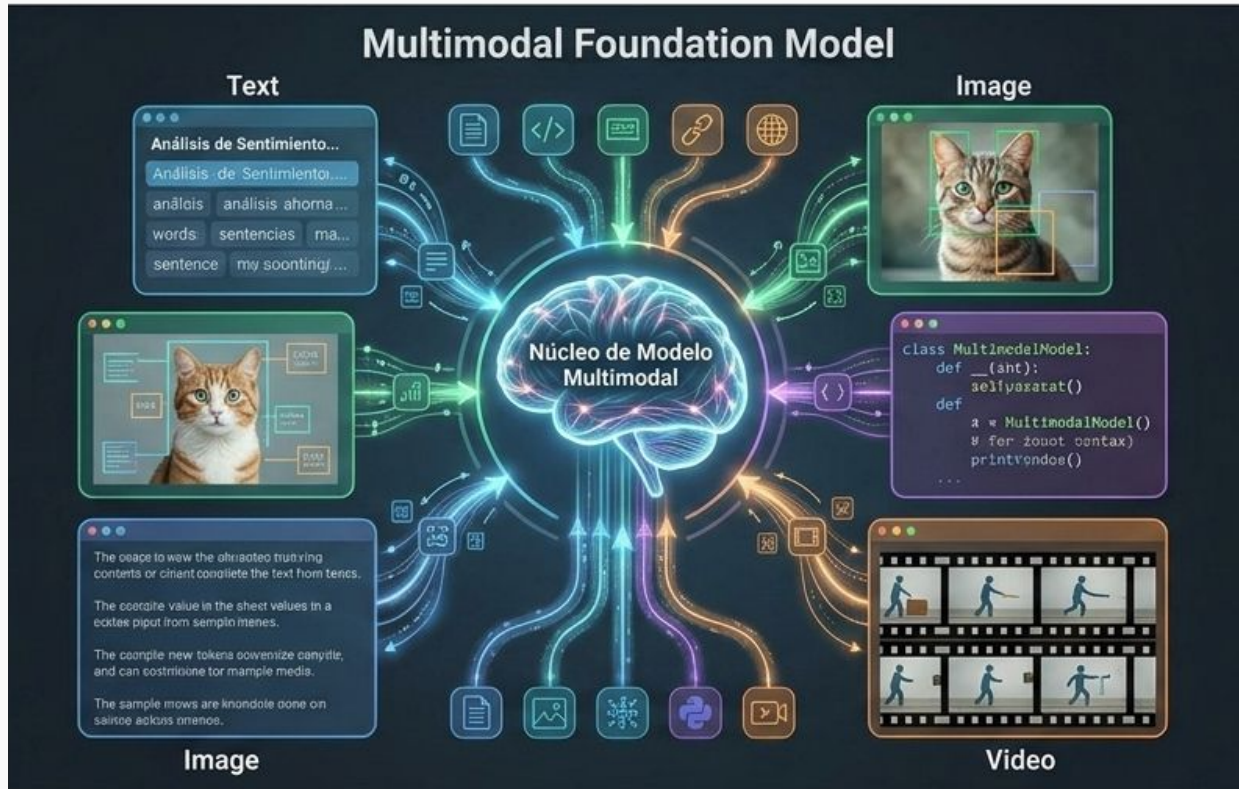
El **entrenamiento** es el proceso de hacer que un modelo aprenda de los datos para que pueda hacer mejores predicciones y tomar decisiones. Se entrenan diferentes modelos para diferentes tareas.



Si entrenamos al modelo de “Milanesa con papas fritas” con 10.000 recetas de cocina...
¿ese mismo modelo nos puede servir para predecir si mañana va a llover?

Machine Learning

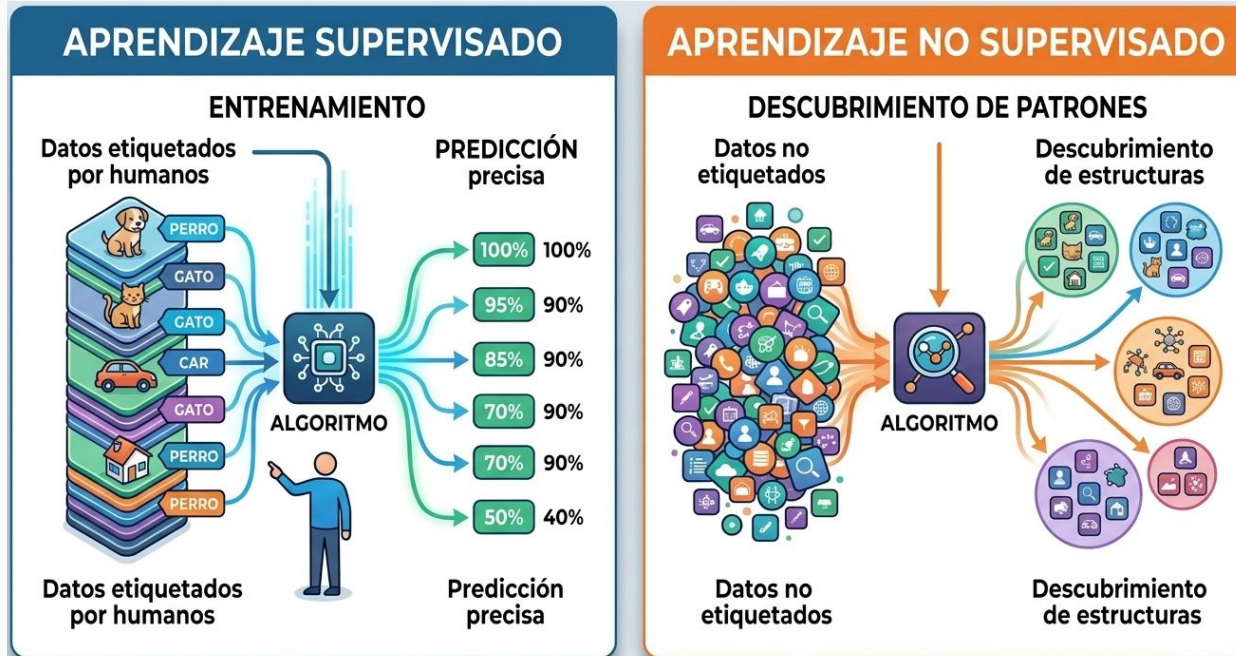
Entonces, ¿por qué los modelos modernos de GenAI (como Gemini o ChatGPT) pueden procesar texto, imágenes y código al mismo tiempo?



Machine Learning

Hay dos enfoques para el Aprendizaje Automático:

- **Aprendizaje supervisado:** Se utiliza frecuentemente para hacer predicciones o tomar decisiones. **Toma datos etiquetados. Requiere** la participación humana para etiquetar datos.
- **Aprendizaje no supervisado:** Intenta descubrir patrones y relaciones dentro de datos sin etiquetar.



Aprendizaje Supervisado

*Este paradigma consiste en entrenar algoritmos a **partir de un dataset histórico etiquetado**. En lugar de permitir que un sistema autónomo improvise, se procesan registros del pasado donde ya se conoce la respuesta correcta. El modelo calibra sus parámetros para predecir el riesgo de forma segura antes de actuar.*



Ejemplo:

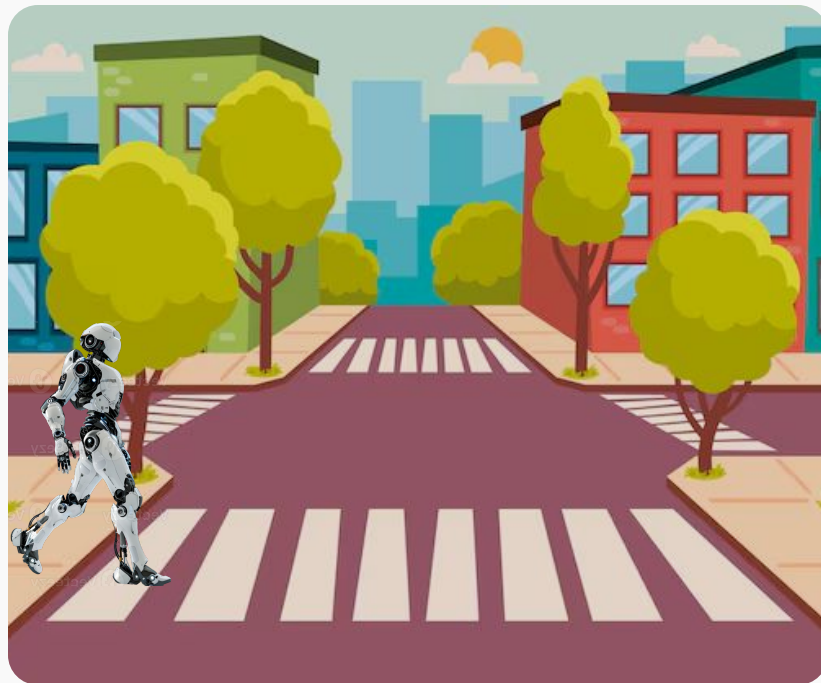
Entrenaremos a un robot para que aprenda a cruzar la calle de forma segura.

El Dataset como base del Control

Decisiones basadas en Datos

Para que el sistema tome decisiones seguras, reemplazamos la improvisación por el **análisis de datos**.

- Procesamiento de un **dataset histórico** de eventos reales.
- Identificación de variables que causaron accidentes en el pasado.
- Calibración de parámetros para **predecir el riesgo** antes de actuar.



Captura de Datos: El Semáforo como Variable

Relación Causa-Efecto

El semáforo emite señales que el sistema recibe como **datos de entrada**, pero cuya lógica operativa aún es desconocida para el modelo.

El objetivo es vincular cada estado lumínico con las **etiquetas del historial**, permitiendo que el algoritmo aprenda la relación entre el color de la luz y la seguridad del cruce.



Etapa de Percepción: Adquisición de Datos



Entrada de Datos Pura

A través de **OpenCV**, el sistema detecta y extrae la señal del semáforo en tiempo real.

Sin embargo, procesar el dato lumínico no equivale a comprenderlo: el robot obtiene el **valor del color (Input)**, pero carece de un modelo para interpretarlo.

El sistema de control aún no conoce el significado operativo de la señal.



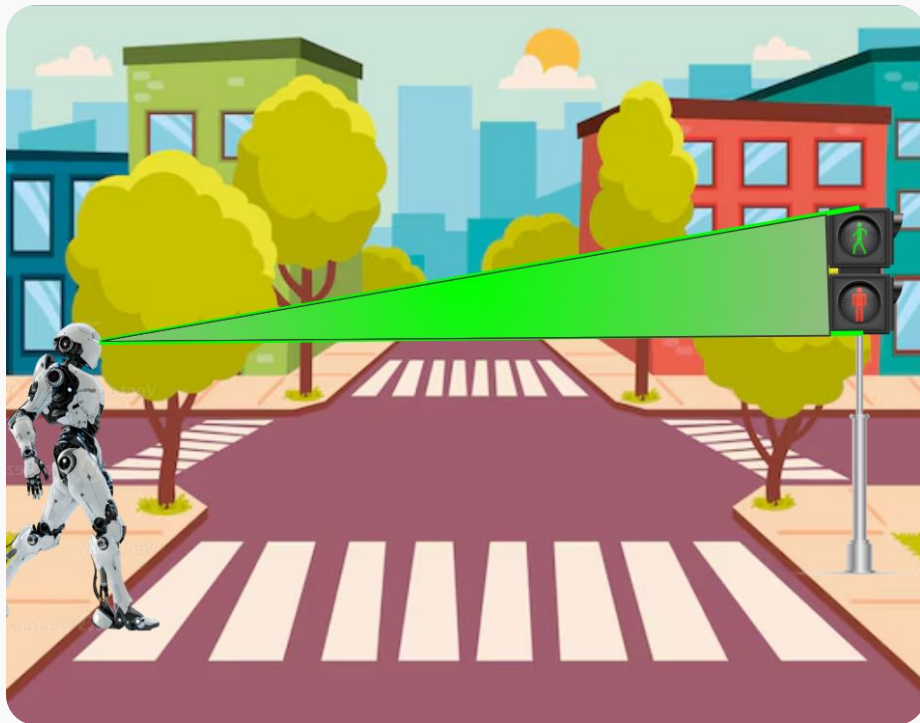
Digitalización y Codificación de Entradas

Procesamiento Binario

A nivel de procesamiento, el sistema cuantifica la información del semáforo como una **variable discreta (binaria)**.

En esta fase, los estados **VERDE** y **ROJO** carecen de valor semántico.

Son datos de entrada puros para establecer una **correlación matemática** con el historial de entrenamiento.



Estructura del Dataset de Entrenamiento

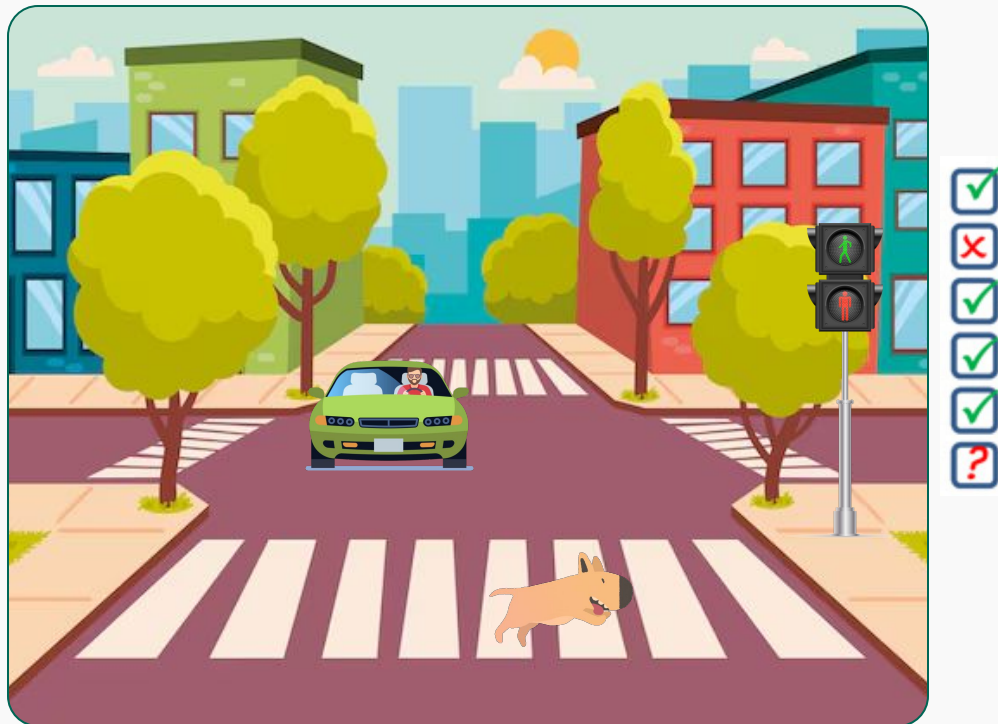
Componentes del Dataset

Basado en **eventos históricos**, vinculamos:

Variable de Entrada: Estado del semáforo.

Etiqueta: Resultado real obtenido.

Clasificamos las muestras entre **Cruce Exitoso** o **Colisión** para que el algoritmo aprenda patrones.



Aprendiendo la Regla de Decisión

Correlación y Predicción

Al revisar el historial, el algoritmo cruza las variables y analiza los resultados de cada caso.

Luz Verde: Vinculada al éxito del cruce.

Luz Roja: Vinculada al riesgo de colisión.

La máquina encuentra el **patrón matemático** que le permite calcular la probabilidad de riesgo antes de actuar.



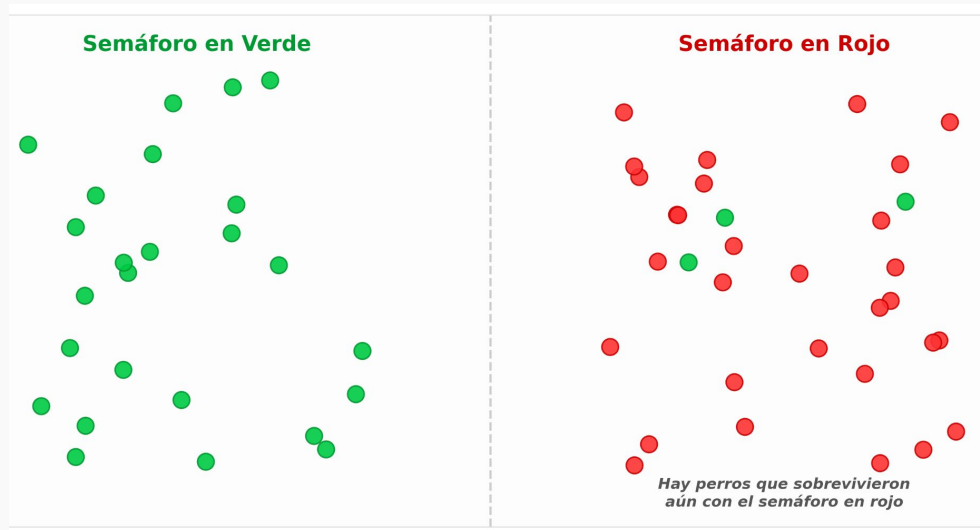
Visualización del Espacio de Datos

Transformación de Datos

Al graficar el historial, transformamos los registros individuales en un **espacio de características**. Cada punto representa una muestra real etiquetada.

Observamos que los datos no son deterministas: existe **solapamiento y ruido** (casos de cruce en rojo sin accidente).

El desafío es encontrar la **frontera matemática** que mejor separe el éxito del riesgo.

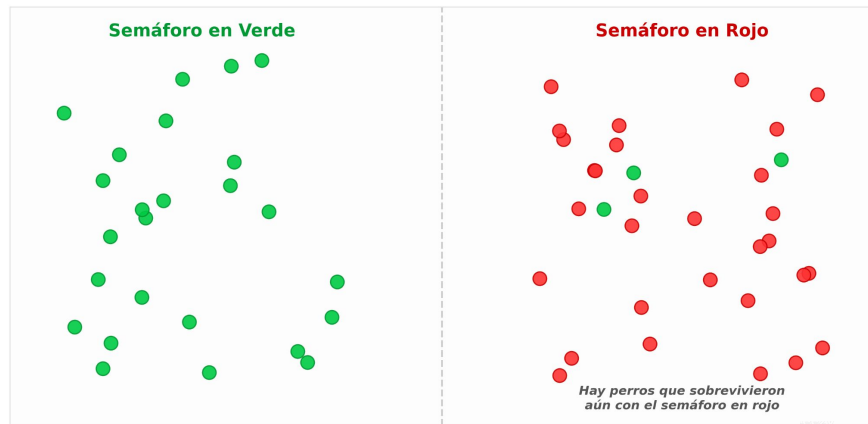


Clasificación y Margen de Error

El objetivo del algoritmo es encontrar la lógica matemática que divide el dataset en dos clases distintas.

Margen de Error: En condiciones reales, los modelos no son infalibles debido al **ruido en los datos** y a variables no controladas.

El sistema busca generalizar una regla que funcione para la mayoría de los casos, separando los patrones de éxito de los de colisión.



Visualización de los datos con solapamiento: Incluso con un modelo óptimo, existen casos excepcionales que caen fuera de la predicción ideal.

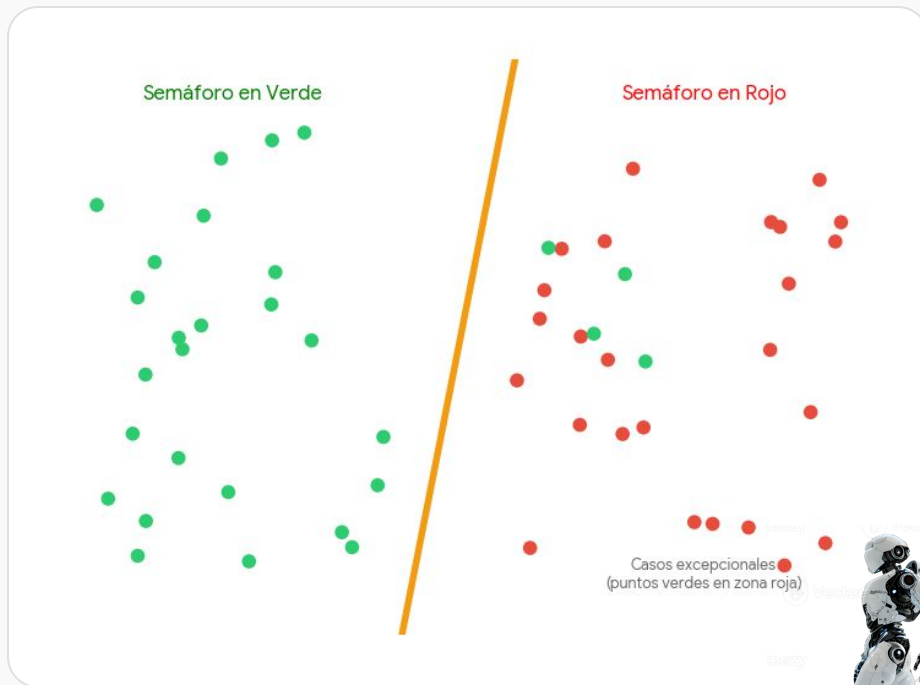


La Frontera de Decisión

Para separar las clases de forma automática, el algoritmo define una **frontera de decisión**.

Matemáticamente, este límite es una función lineal (una línea recta) que segmenta el espacio de datos.

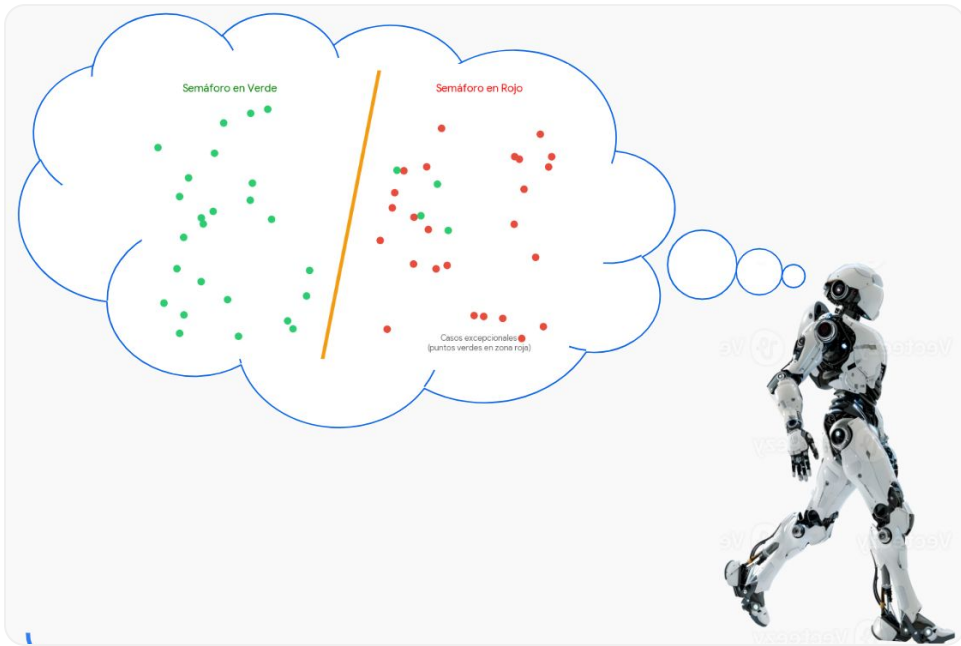
Esta función es la **regla definitiva** que usará el modelo para **clasificar** cualquier dato nuevo instantáneamente.



Clasificación

Inteligencia en el Procesador

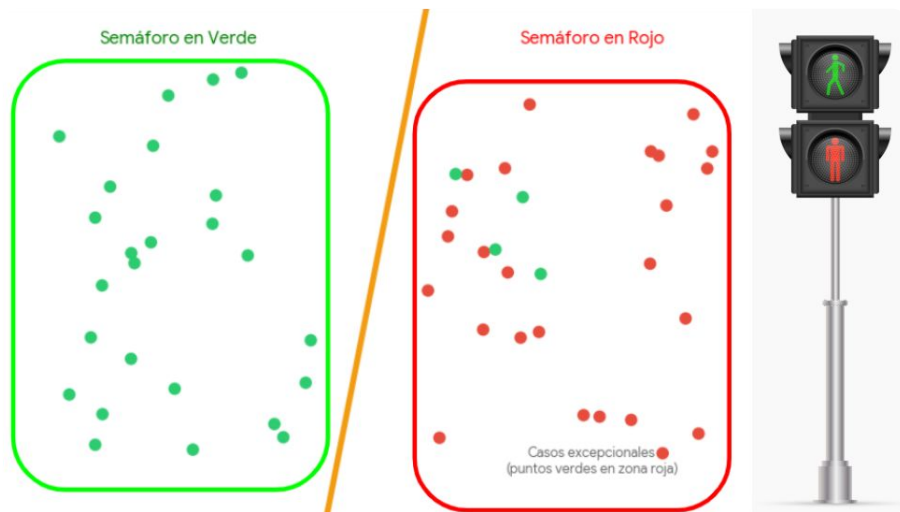
Esa línea que trazamos no es solo un dibujo, es la **regla matemática** que ahora vive en el procesador del robot.



Clasificación

En **Machine Learning**, identificar la categoría de un nuevo dato es **CLASIFICACIÓN**.

El algoritmo **utiliza la frontera matemática** entrenada para determinar la clase de cada escenario.



Rol Supervisor y tags

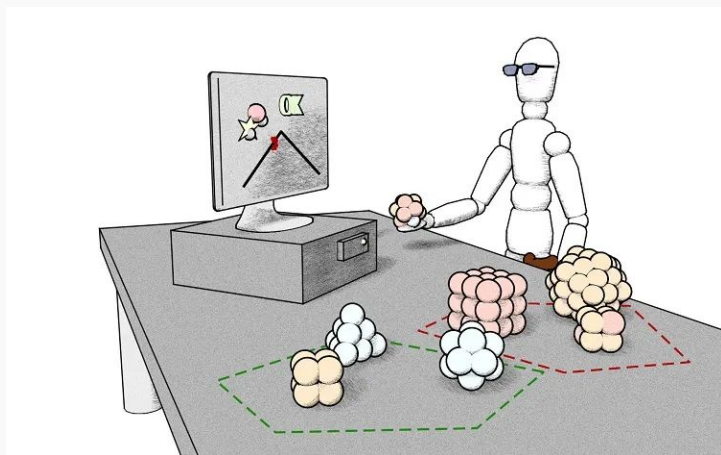
CRUZÓ en **ROJO** -> **MAL**

CRUZÓ en **VERDE** -> **BIEN**

NO CRUZÓ en **ROJO** -> **BIEN**

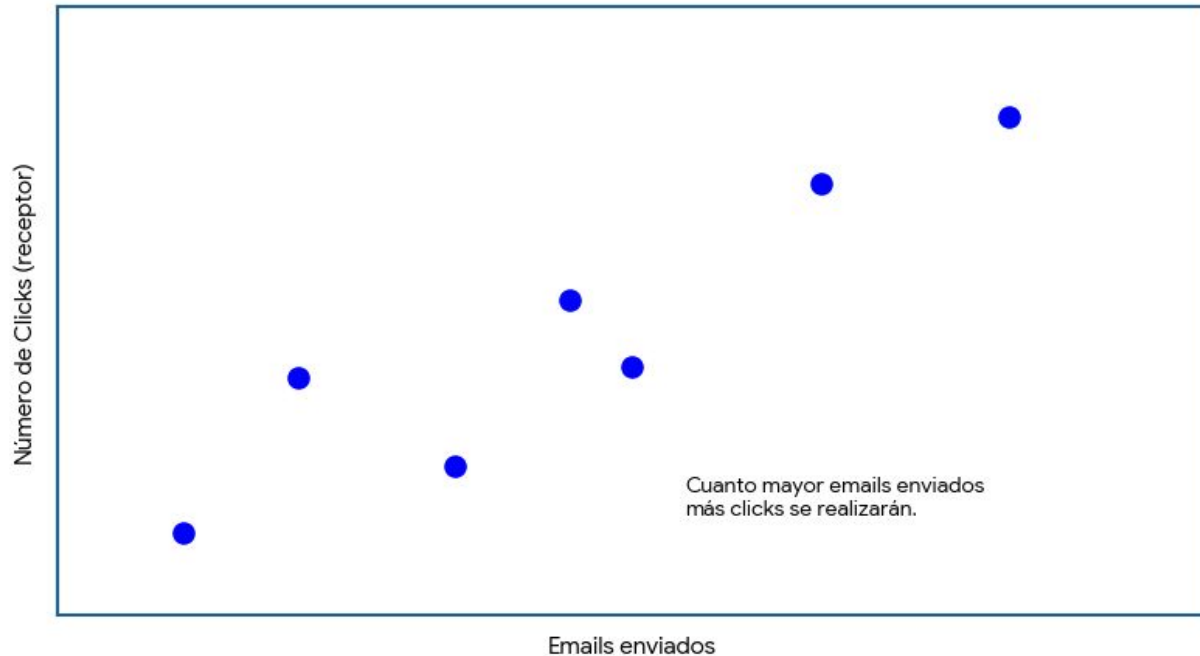
El criterio de control lo define un **supervisor humano**. Nosotros determinamos qué estados son seguros y cuáles peligrosos, generando la estructura del **dataset etiquetado**.

A partir de esta base de datos, el algoritmo inicia su fase de **entrenamiento**. El sistema no necesita experimentar ni probar combinaciones en el mundo real, **simplemente analiza el registro histórico para ajustar sus parámetros matemáticos y aprender a replicar el criterio de éxito del supervisor**. Esto es el **Aprendizaje Supervisado**.



Introducción a los Modelos de Regresión

Para predecir valores numéricos continuos en lugar de categorías fijas, entramos en la segunda gran rama del **Aprendizaje Supervisado**, la **Regresión**.



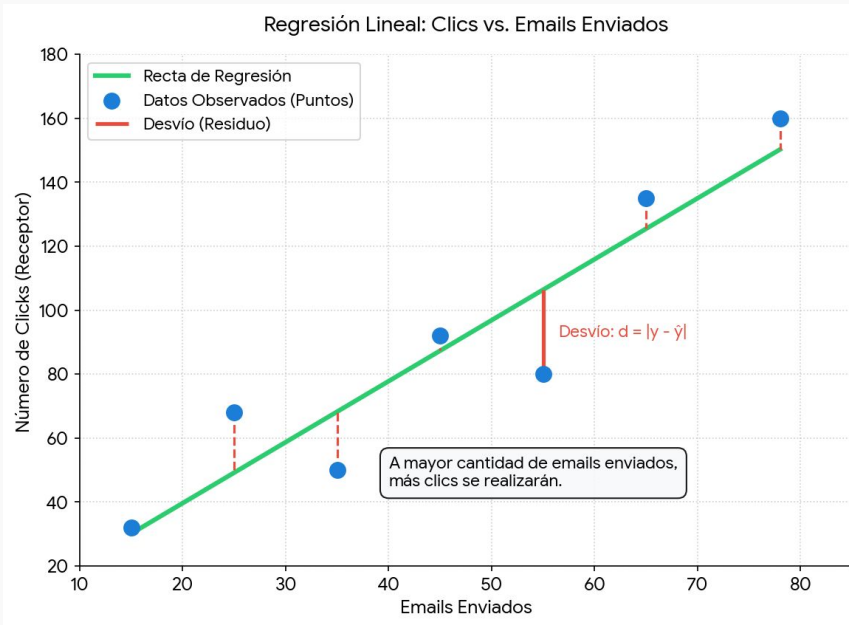
Caso de estudio:

Análisis de interacción en campañas.

El objetivo del algoritmo ya no es elegir entre dos opciones discretas (como avanzar o detenerse), sino **encontrar la función matemática que nos permita estimar la cantidad exacta de Clicks en función del volumen de Emails enviados.**

El Modelo Lineal y los Residuos

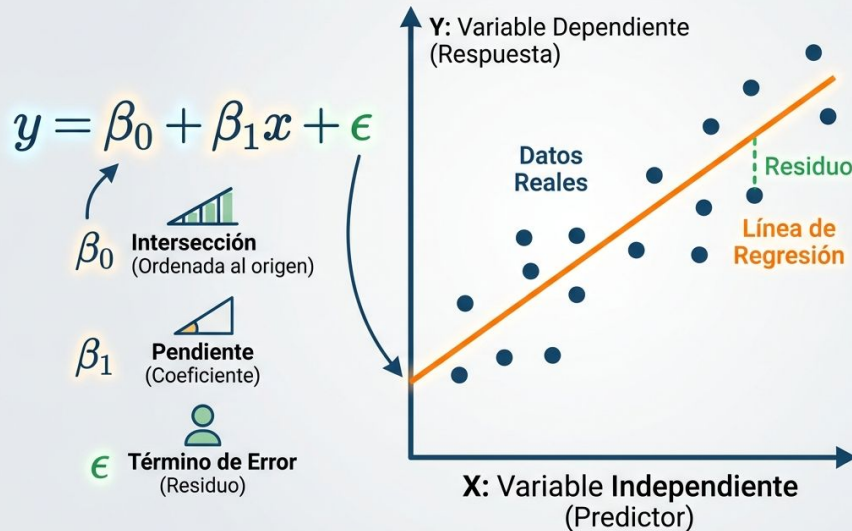
Para describir matemáticamente este comportamiento, el algoritmo calcula una línea de tendencia (ajuste óptimo). Las líneas rojas representan los **residuos**, que son la diferencia exacta entre los clicks reales obtenidos y la estimación del modelo. **El objetivo de la regresión lineal es encontrar la recta que minimice globalmente estos desvíos.**



Regresión Lineal: El Modelo Matemático

Este método permite predecir un valor numérico continuo (**Variable Dependiente**) a partir de variables de entrada (**Predictores**). Busca la función óptima que minimiza los residuos para entender tendencias y proyectar resultados con rigor estadístico.

REGRESIÓN LINEAL: CONCEPTOS CLAVE



CARACTERÍSTICAS PRINCIPALES



OBJETIVO:
Predecir un valor numérico continuo.



LINEALIDAD:
Supone una relación en línea recta.



MÉTODO:
Minimiza la suma de errores cuadrados (MCO).



EVALUACIÓN:
Usa R^2 para medir el ajuste.

Tipos de Modelos de Aprendizaje Supervisado

CLASIFICACIÓN

(Predicción de Clases o Categorías)

K-NEAREST NEIGHBORS (KNN)



K-NEAREST
NEIGHBORS
(KNN)

- 📷 Clasificación de Imágenes
- ⚠️ Detección de Fraude
- 🎯 Fronteras de Decisión Claras

MÁQUINAS DE VECTORES DE SOPORTE (SVM)



- 📷 Clasificación de Imágenes
- ⚠️ Detección de Fraude
- 🎯 Fronteras de Decisión Claras

ÁRBOLES DE DECISIÓN

EMAIL SPAM?
[SPAM / NO SPAM]

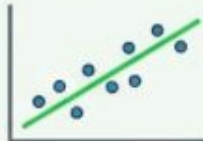


- 📷 Clasificación de Imágenes
- ⚠️ Detección de Fraude
- 🎯 Fronteras de Decisión Claras

REGRESIÓN

(Predicción de Valores Numéricos Continuos)

REGRESIÓN LINEAL



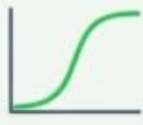
PREDICCIÓN
DE PRECIOS
DE CASAS [\$]

- 📈 Análisis de Tendencias
- ↗️ Relaciones Lineales
- 📊 Múltiples Variables de Entrada

REGRESIÓN POLINÓMICA



PRONÓSTICO
DE VENTAS



- 📈 Análisis de Tendencias
- ↗️ Relaciones No Lineales
- 📊 Múltiples Variables a [°C]

BOSQUES ALEATORIOS



TEMPERATURA [°C]

- 📈 Análisis de Tendencias
- ↗️ Relaciones No Lineales
- 📊 Múltiples Variables de Entrada

CONCEPTOS CLAVE



Datos de Entrenamiento Etiquetados



Función de Pérdida

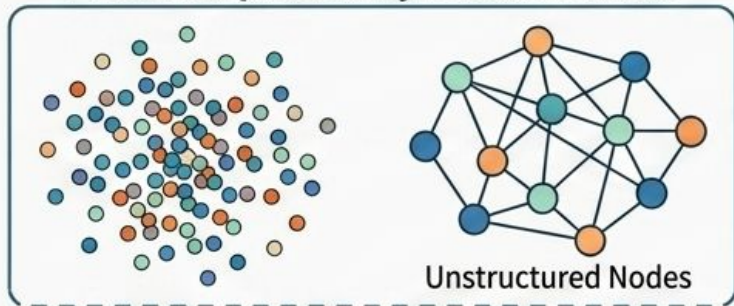


Validación

Aprendizaje No Supervisado

A diferencia del paradigma anterior, trabajamos con un dataset histórico **no etiquetado**. El sistema no cuenta con la guía de un supervisor humano ni conoce de antemano la respuesta correcta.

Panel 1: Exploración y Descubrimiento

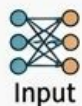


Procesamiento
Autónomo



Estructuras
Ocultas
Intrínsecas

Généric learning model



Input



K-Means

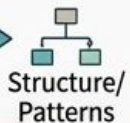


PCA



PCA

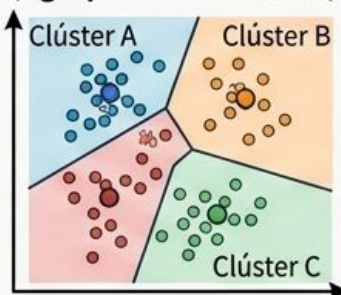
Unsupervised model



Structure/
Patterns

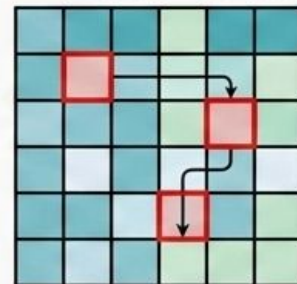
Panel 2: Resultados e Insights

Clustering
(Agrupamiento Natural)



Clustering
(Agrupamiento Natural)

Detección de
Anomalías Latentes



Detección de
Anomalías Latentes

Exploración Autónoma

Al ingresar al depósito, el robot se enfrenta a un **entorno no etiquetado**. Ya no existe un semáforo que le ordene qué hacer.

El controlador ahora debe procesar el caos de objetos esparcidos basándose únicamente en sus sensores.

"Buscando descubrir estructuras y grupos lógicos por su cuenta, sin intervención humana previa."

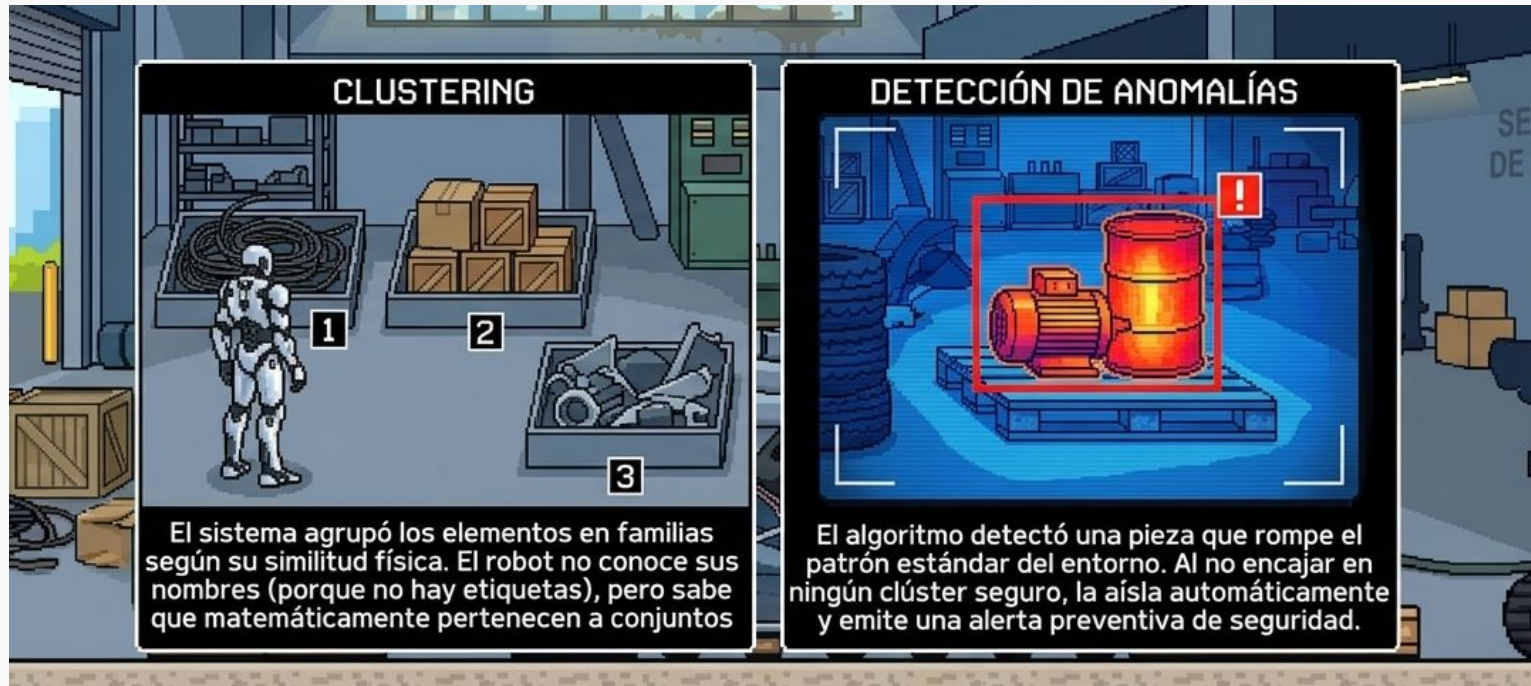


Procesamiento Autónomo



Resultados: Organización del Caos y Descubrimiento de Estructuras

Al procesar las características intrínsecas de los objetos, el algoritmo transforma el desorden en información útil. El resultado es el descubrimiento autónomo de la estructura del entorno.



Otro método: Reglas de Asociación

Mientras el **Clustering** agrupa objetos por similitud física, la **Asociación** descubre relaciones de dependencia (co-ocurrencia).

El objetivo principal es identificar qué elementos distintos tienden a **aparecer juntos** dentro de un mismo evento o transacción.

Esto permite optimizar la organización lógica basándose en el comportamiento real de los datos, no solo en su apariencia.



Otro método: Reglas de Asociación

Paso 1

El robot procesa de forma autónoma miles de registros de palets que salen del depósito.



Paso 2

El algoritmo detecta un patrón matemático latente: en el 85% de los casos donde se despacha una Soldadora, el pedido también incluye Máscaras de Protección.



Paso 3

Se establece una Regla de Asociación ($A \rightarrow B$). Esto permite reestructurar el layout del depósito para poner esos productos en la misma estantería, sin que un ingeniero haya programado esa regla previamente.



Aprendizaje No Supervisado



Tanto en el Clustering como en la Asociación, el sistema no cuenta con etiquetas previas (respuestas correctas) ni recibe recompensas o castigos.

El algoritmo se enfrenta a la información cruda y descubre la estructura matemática oculta de forma **100% autónoma**, valiéndose únicamente de la **estadística**.

Tipos de Modelos de Aprendizaje No Supervisado

AGRUPAMIENTO (CLUSTERING)

(Descubrimiento de Estructuras Latentes)

K-MEANS CLUSTERING



- 📷 Segmentación de Clientes por Comportamiento
- ⚠️ Organización de Datos Heterogéneos
- 🎯 Reducción de Varianza Intra-cluster

AGRUPAMIENTO JERÁRQUICO (HIERARCHICAL)



- 📷 Taxonomía de Especies
- ⚠️ Análisis de Redes Sociales
- 🎯 Estructura Jerárquica de Datos

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

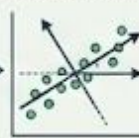
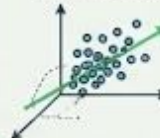


- 📷 Detección de Anomalías (Outliers)
- ⚠️ Clusterización de Densidad Irregular
- 🎯 Resistencia al Ruido

REDUCCIÓN DE DIMENSIONALIDAD

(DIMENSIONALITY REDUCTION) (Compresión de Datos)

ANÁLISIS DE COMPONENTES PRINCIPALES (PCA)



- 📷 Compresión de Imágenes
- 📈 Preprocesamiento de Datos
- 📊 Visualización de Datos de Alta Dimensión

T-DISTRIBUTED STOCHASTIC NEIGHBOR EMBEDDING (T-SNE)



High-D



Low-D

- 📷 Visualización de Datos Complejos (ej. Single-cell Seq)
- 📈 Exploración de Clusters
- 📊 Preservación de Estructura Local

AUTOENCODERS



- 📷 Compresión de Señales
- 📈 Generación de Datos
- 📊 Detección de Fraude (por Reconstrucción)

CONCEPTOS CLAVE



Datos de Entrenamiento
No Etiquetados



Medida de Similitud /
Distancia



Validación Externa/Interna
(Silueta, etc.)

Reducción de Dimensionalidad: Simplificando el Caos

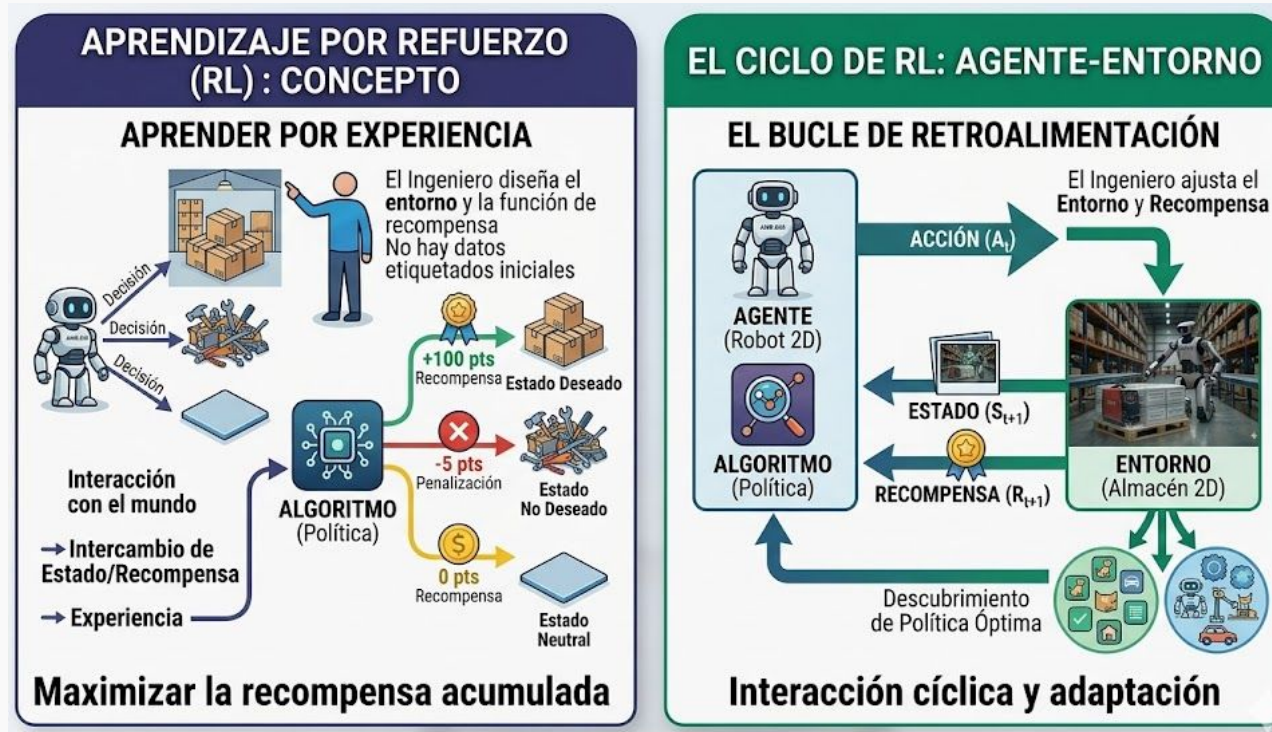
En una línea de producción compleja, el sistema de control recibe telemetría de decenas de sensores simultáneamente (vibración, temperatura, acústica, posición). Cada sensor representa una dimensión matemática. Con la Reducción de Dimensionalidad (PCA), comprimimos estas múltiples variables en solo 2 componentes principales, eliminando el ruido y preservando la información crítica para permitir una monitorización ágil y precisa del estado de la maquinaria.



Machine Learning

Existe un enfoque más para el Aprendizaje Automático:

- **Aprendizaje por refuerzo:** El arte de aprender a través de la experiencia, los premios y los castigos. (Reinforcement Learning)



SPider: Explorador 2.0

Para entender este nuevo mundo, dejamos atrás el depósito. Les presento a SPider, un robot explorador ágil y autónomo.

A diferencia de los modelos anteriores, SPider no espera que le demos datos etiquetados ni que le enseñemos patrones. SPider aprende viviendo en su entorno.



El laberinto de decisiones

Soltamos a SPider en un laberinto. No tiene un mapa, no tiene instrucciones y no conoce el camino de salida.

Su única guía es una señal de recompensa que se activa solo cuando logra sus objetivos.



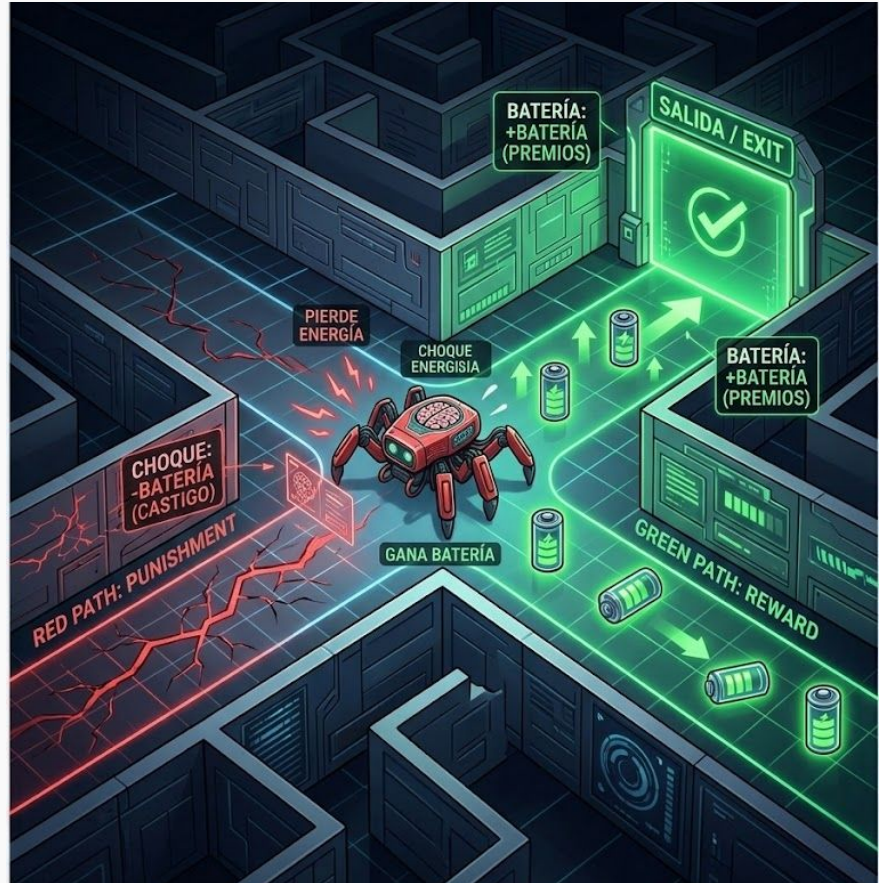
Aprendizaje por Ensayo y Error

Premios y Castigos

SPider explora el laberinto de forma aleatoria al principio. Cada decisión genera una consecuencia:

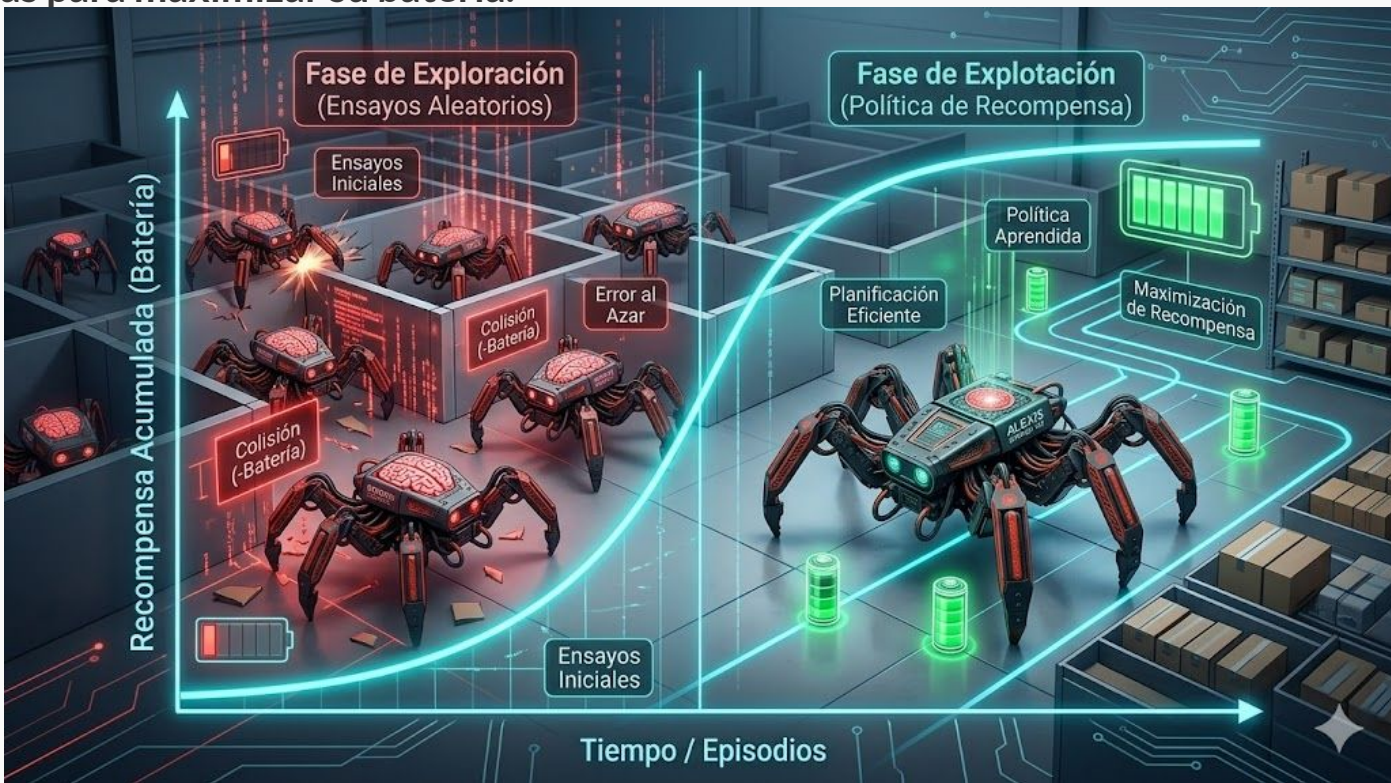
Recompensa (+): SPider avanza hacia la salida y gana batería.

Castigo (-): SPider choca contra un muro y pierde energía.



Curva de Aprendizaje

Con el tiempo, SPider deja de probar al azar y comienza a aplicar una **Política**: un conjunto de reglas aprendidas para maximizar su batería.



Inspiración Biológica: El Refuerzo Humano

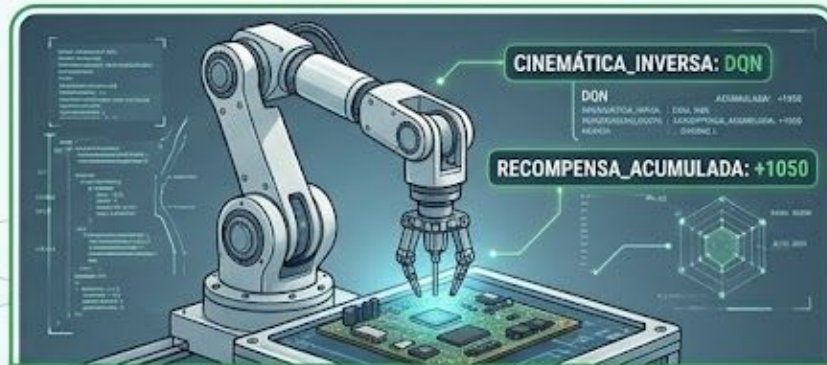
El Aprendizaje por Refuerzo modela matemáticamente el proceso cognitivo humano más básico. Cuando somos niños, no aprendemos con un **dataset etiquetado** previo. Aprendemos interactuando, tocar algo caliente genera un refuerzo negativo (dolor) que actualiza nuestra política interna, enseñándonos a evitar esa acción en el futuro.



RL: Aplicaciones y Ejemplos



1. Estrategia y Juegos (Entornos Reglas Fijas)



2. Robótica Avanzada (Control de Movimiento)



3. Navegación Autónoma (Tráfico Dinámico)



4. Finanzas Automatizadas (Maximización de Profit)

SUPERVISADO



REGRESIÓN

- Predicción de popularidad
- Pronóstico de mercado
- Clima y Vida útil (Soft Sensors)

CLASIFICACIÓN

- Detección de fraude
- Diagnóstico médico
- Retención de clientes (Churn)

DE REFUERZO



OPTIMIZACIÓN

- Navegación de robots (Spider)
- Decisiones en tiempo real
- Adquisición de habilidades
- Control de procesos industriales

ESTRATEGIA

- Juegos y Finanzas
- Sistemas Multi-agente

SIN SUPERVISIÓN



REDUCCIÓN DE DIMENSIÓN

- Visualización de Big Data (PCA)
- Compresión de señales
- Descubrimiento de estructuras

CLUSTERING Y ASOCIACIÓN

- Segmentación de clientes
- Reglas de Asociación (A -> B)
- Sistemas de recomendación

[]

02

Frameworks y Herramientas

{ }



Python



Python es un lenguaje de alto nivel de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código.

Python es un lenguaje de programación multiparadigma. Permite varios estilos: **programación orientada a objetos**, **programación imperativa** y **programación funcional**.

*Obs: En el fondo, Python es un lenguaje orientado a objetos, todo, absolutamente todo es un objeto.

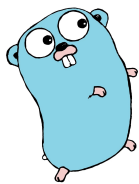
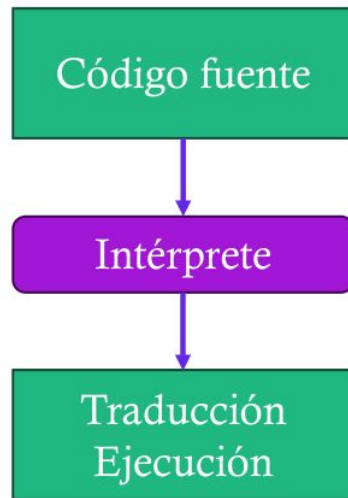
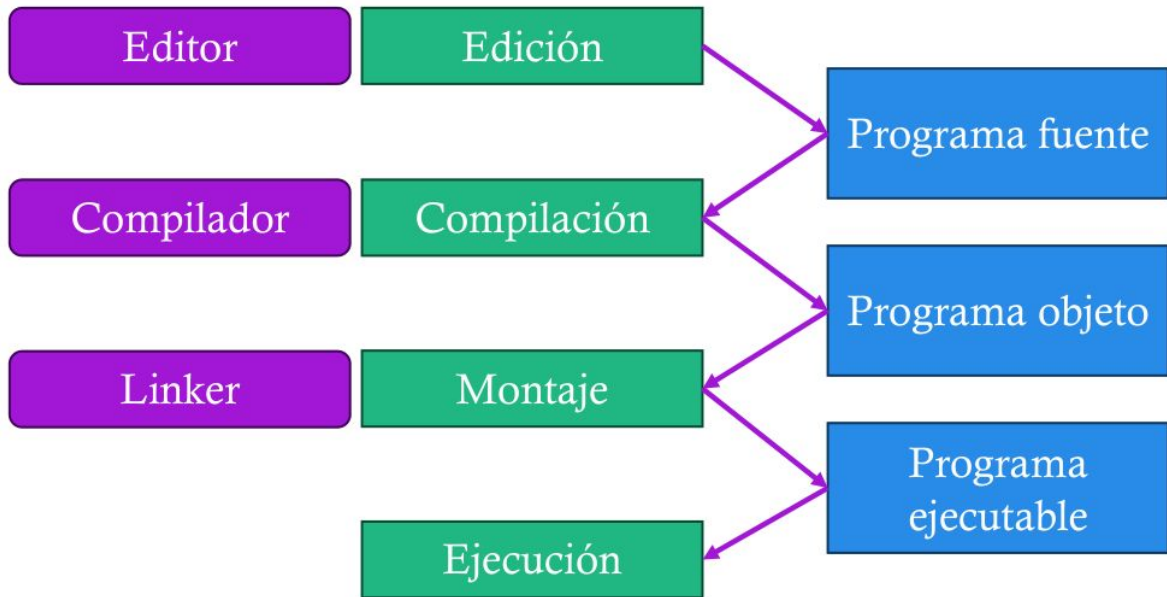
Python usa tipado dinámico y conteo de referencias para la gestión de memoria.

Python reemplazó en gran medida a **LISP** en IA, principalmente por ser multiparadigma.



```
9      ;;
10     ;; Triple the value of a number
11     ;;
12
13     (= (defun triple (X)
14         "Compute three times X." ; Inline c
15         (* 3 X))                ; be place
16
17     ;;
18     ;; Negate the sign of a number
19     ;;
20
21     (= (defun negate (X)
22         "Negate the value of X." ; This is
23         (/ X -1))
```

Python



Compilado

Interpretado

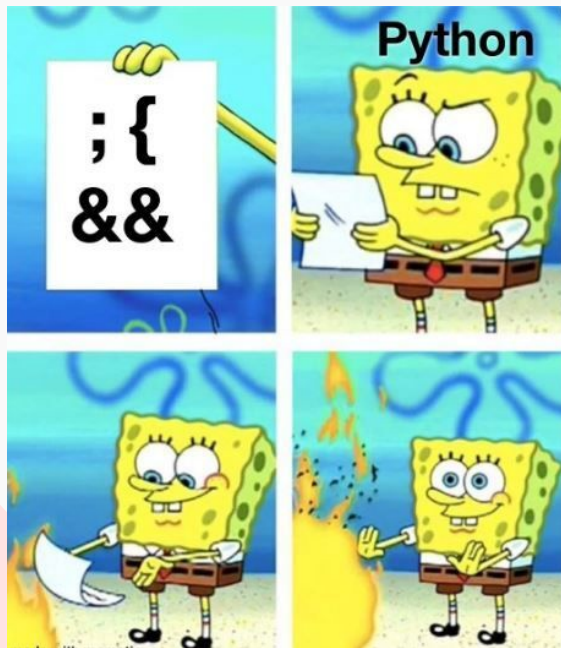


Python



Python es uno de los lenguajes más usados en **Ciencia de datos**. ¿Por qué?

Porque tiene una **sintaxis simple y es fácil** de adaptar para quienes no vienen de ambientes de ingeniería o ciencia de la informática.

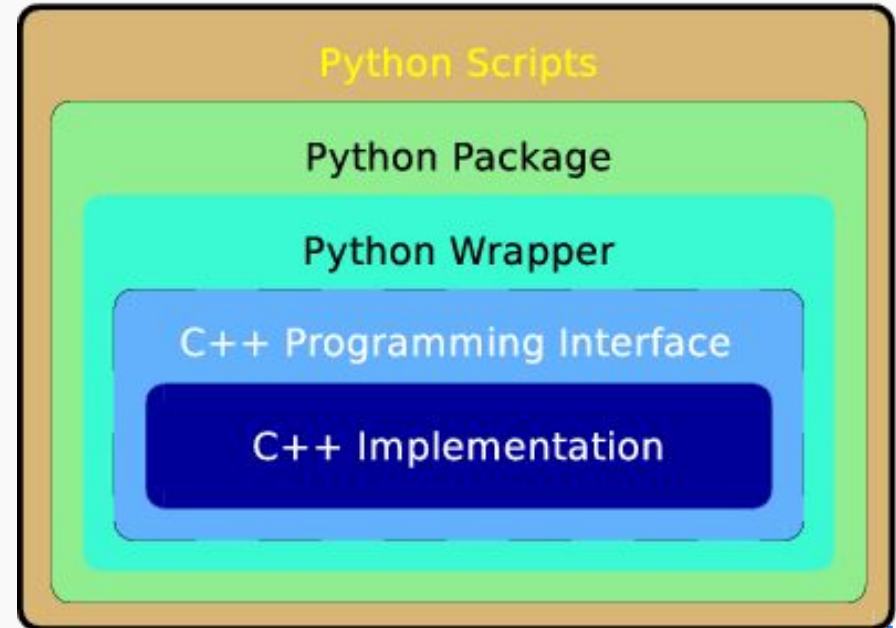
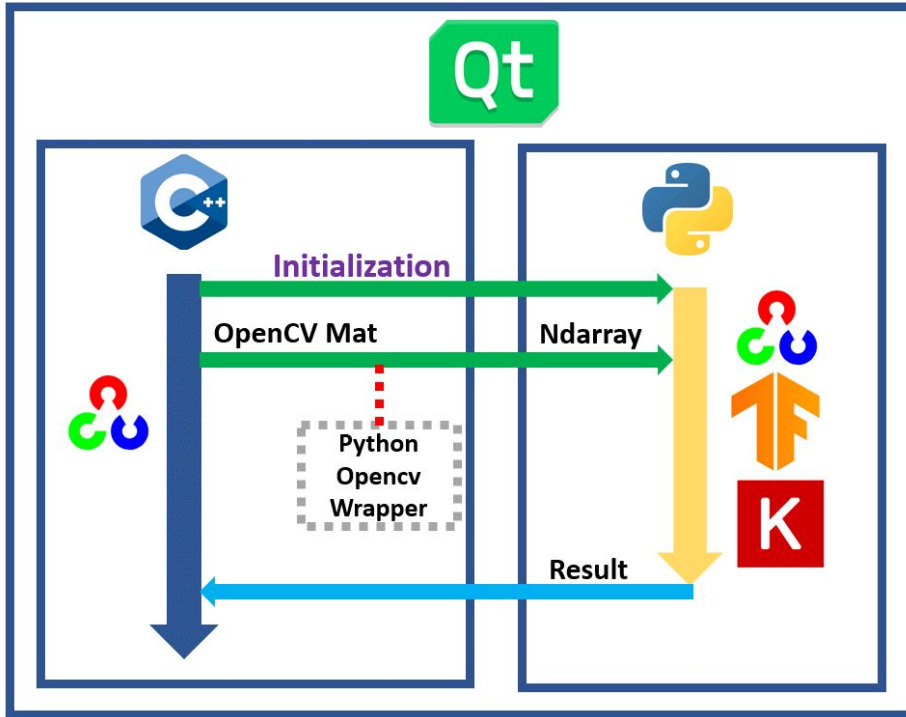


Slowest things on earth:



Python es famoso por ser lento comparado con lenguajes como C++, ¿por qué se usa en Machine Learning o IA?

{ }

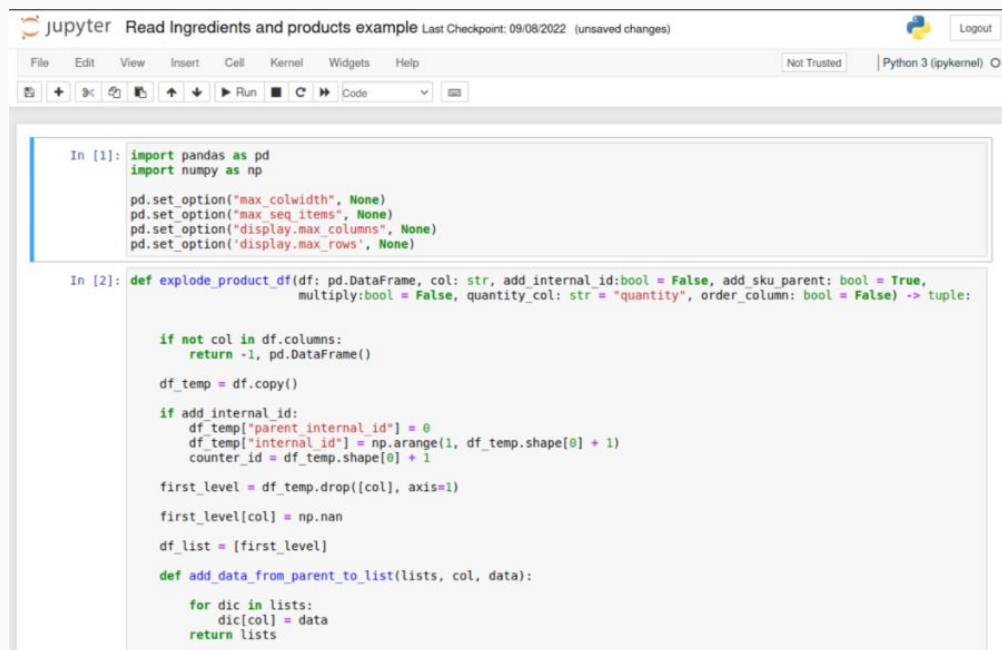


Jupyter Notebook



Es un **entorno computacional interactivo** basado en la web para crear documentos de notebook.

Jupyter Notebook es similar a la interfaz de notebook de otros programas como **Maple**, **Mathematica** y **SageMath**, un estilo de interfaz computacional que se originó con **Mathematica** en la década de 1980.



```
jupyter Read Ingredients and products example Last Checkpoint: 09/08/2022 (unsaved changes) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

In [1]: import pandas as pd
import numpy as np

pd.set_option("max_colwidth", None)
pd.set_option("max_seq_items", None)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)

In [2]: def explode_product_df(df: pd.DataFrame, col: str, add_internal_id: bool = False, add_sku_parent: bool = True,
multiply: bool = False, quantity_col: str = "quantity", order_column: bool = False) -> tuple:

    if not col in df.columns:
        return -1, pd.DataFrame()

    df_temp = df.copy()

    if add_internal_id:
        df_temp["parent_internal_id"] = 0
        df_temp["internal_id"] = np.arange(1, df_temp.shape[0] + 1)
        counter_id = df_temp.shape[0] + 1

    first_level = df_temp.drop([col], axis=1)

    first_level[col] = np.nan

    df_list = [first_level]

    def add_data_from_parent_to_list(lists, col, data):

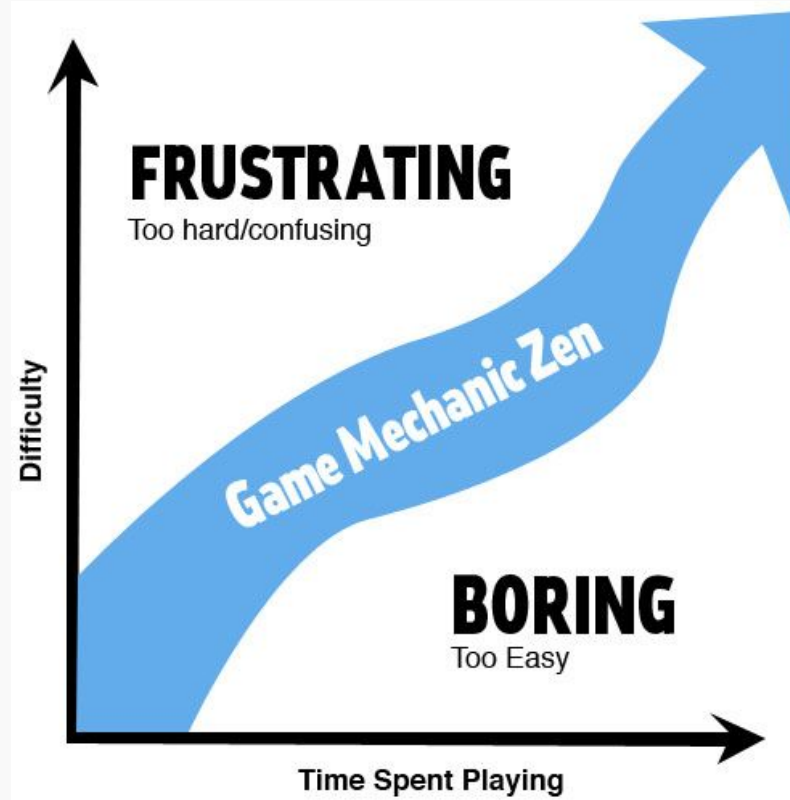
        for dic in lists:
            dic[col] = data
            return lists
```

Herramientas de Desarrollo



Vamos a ver tres formas de instalar nuestro entorno de desarrollo:

1. Modo Amateur
2. Modo Semi-Pro
3. Legendario



Modo Amateur

Google Colab permite escribir y ejecutar Python en el navegador:

- Sin configurar
- Fácil de compartir
- Acceso a GPUs sin cargo

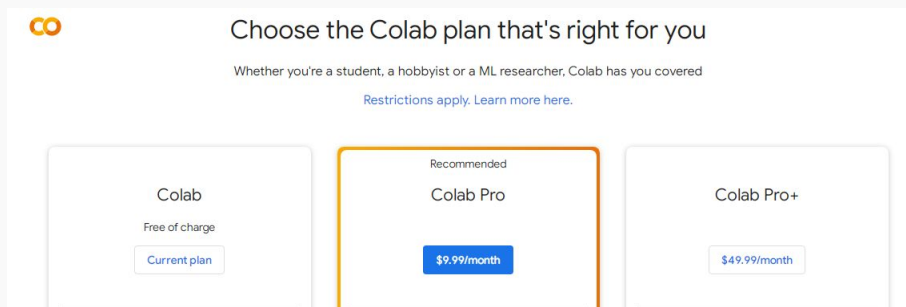


Es una **Jupyter Notebook** que corre en una máquina virtual de Google Cloud:

- Es gratuito
- Ofrece 12 GB de RAM y 100GB de disco.

Las notebooks quedan en Google Drive, fácil de compartir.

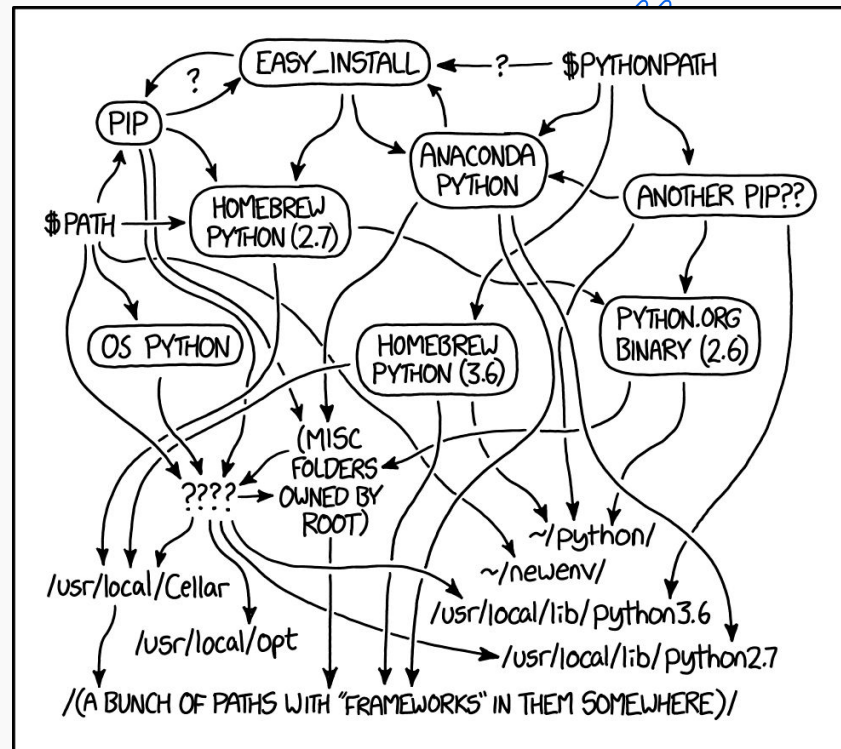
El **problema** es que no todo lo de esta materia funciona en Google Colab



()

()

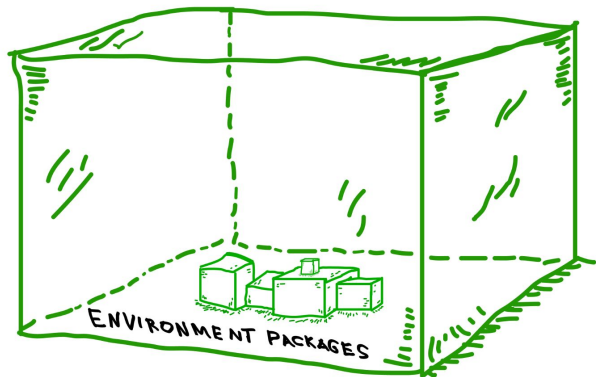
3. **Entornos virtuales.** Cuando trabajamos en distintos proyectos, no todos ellos requieren los mismos paquetes ni siquiera la misma versión de Python. Para ello podemos usar entornos virtuales que nos permiten instalar paquetes específicos al proyecto, también establecer qué versión de Python usaremos.



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.



Modo Semi-Pro



Virtual Environment 1



Python 3.8



Langchain 0.0.26



Pandas 1.0.1

Virtual Environment 2



Python 3.4



Django 4.1.1



Flask 2.2.1



Modo Semi-Pro



La forma más fácil de obtener todo esto es usar una herramienta que nos resuelva todo este problema y además que este pensado principalmente para ciencia de datos.



Anaconda es una distribución de Python y R, enfocado en computación científica (ciencia de datos, procesamiento en gran escala y aprendizaje automático). Está orientado a simplificar el despliegue y administración de los paquetes de software.

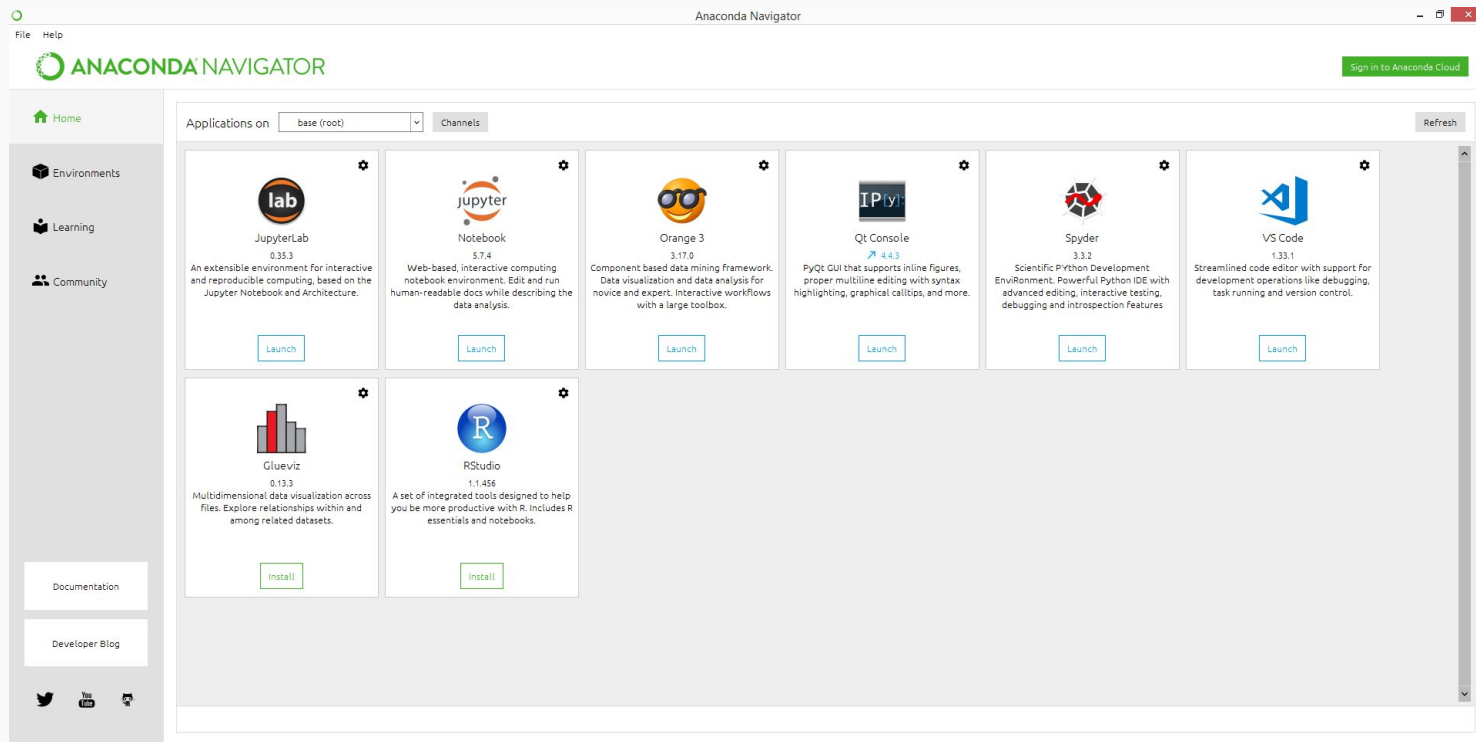
La versión de los paquetes en Anaconda es llevada a cabo por el sistema de manejo de paquete conda.



Modo Semi-Pro

Anaconda también maneja entornos virtuales usando conda. Además, nos provee una UI que permite configurar sin necesidad de pasar por consola.

Gran desventaja, pesa 5 Gb. Se puede evitar si se usa Miniconda (400 Mb), pero perdemos la UI.

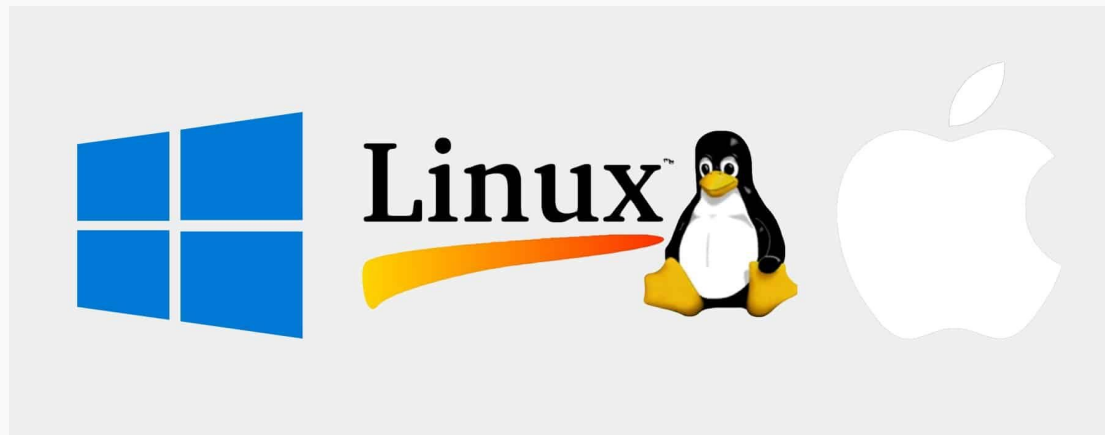
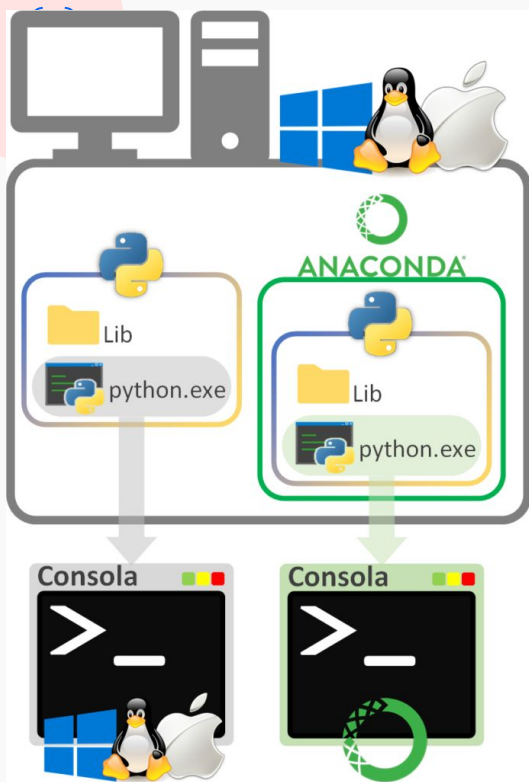


Modo Legendario

Similar a lo que vimos:

- Tener instalado al menos una versión Python
- Gestor de paquetes
- Entornos virtuales

Pero usando herramientas con mejor performance



Modo Legendario



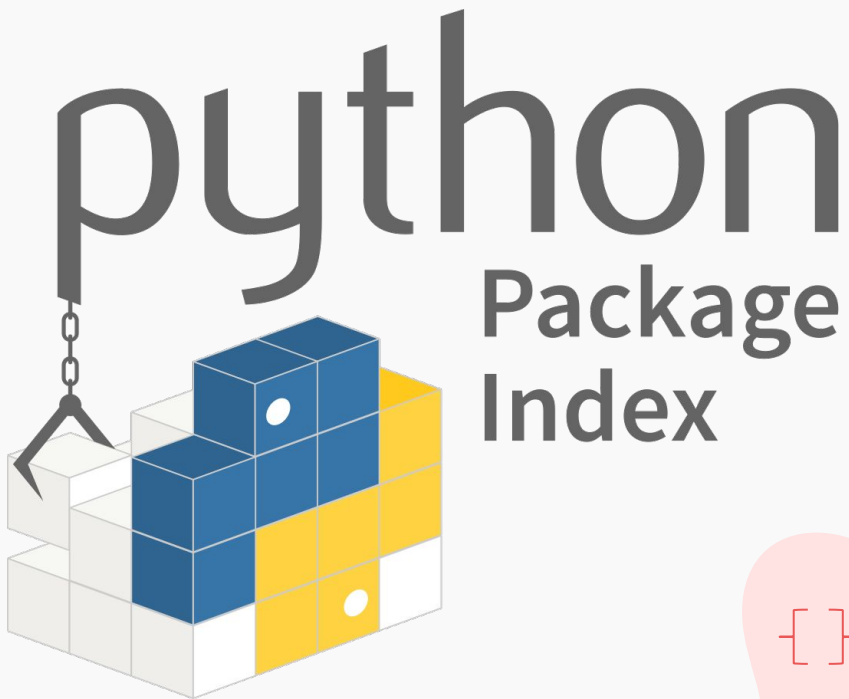
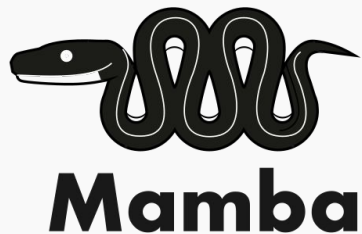
pip es un sistema de gestión de paquetes utilizado para instalar y administrar paquetes de software escritos en Python.

Muchos de estos paquetes pueden ser encontrados en **PyPI**. Python incluyen pip por defecto.



Existen otros gestores:

- uv
- Mamba
- PDM, etc



TM

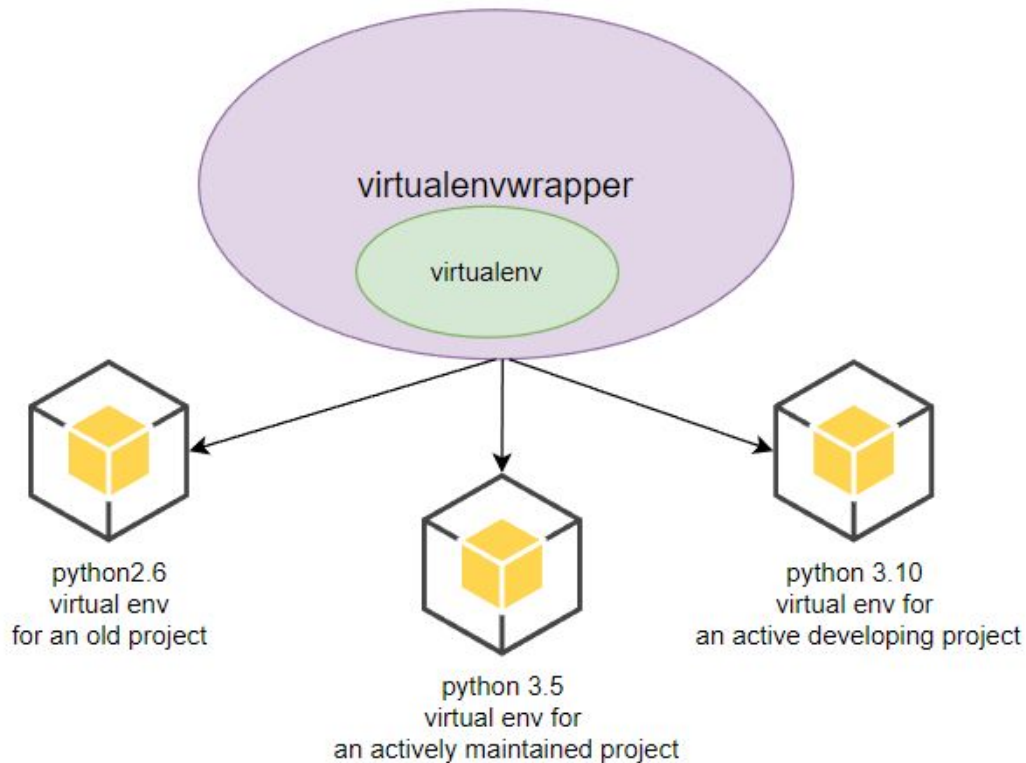
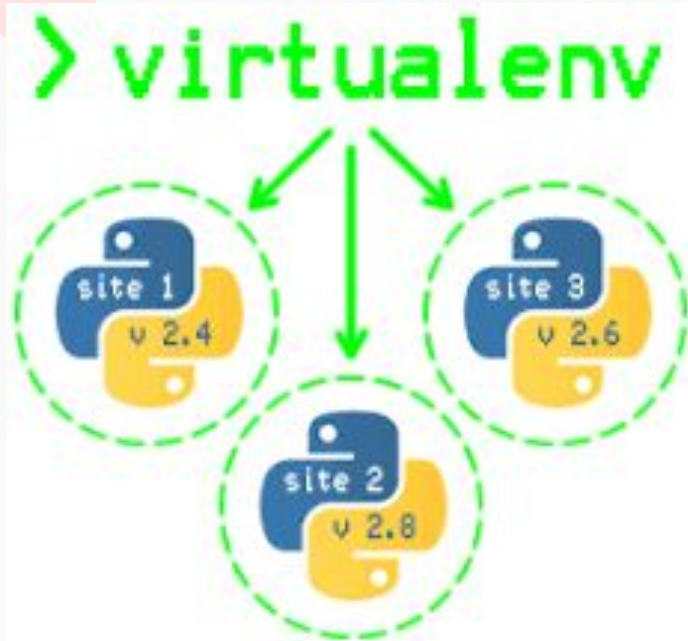


Modo Legendario

Entornos virtuales:

Hay múltiples formas de hacerlo, una de ellas es virtualenv.

- virtualenvwrapper
- pyenv-virtualenv



Modo Legendario



Hoy en día una herramienta que es muy popular en el desarrollo de Python es Poetry.



Poetry es una herramienta para gestión y empaquetado de dependencias en Python. Permite declarar las bibliotecas de las que depende el proyecto y se encarga de administrarla.

No instala automáticamente el intérprete de Python (lo tiene que hacer uno mismo).

Para arrancar un nuevo proyecto:

```
poetry new test-proyect
```

```
poetry env use python3.10
```

```
poetry add matplotlib
```

Lo bueno es que, al indicarle **una versión de Python**, se encarga de que los paquetes que se instalen sean compatibles con las versiones especificadas.

Eso se hace modificando el archivo `pyproject.toml`:

```
[tool.poetry.dependencies]
python = "^3.9.0"
```



Lenguajes de Programación para IA



Python

Lidera en lenguajes de desarrollo de Machine Learning debido a su simplicidad y facilidad de aprendizaje.



Ideal para iniciar en el mundo de IA especialmente Machine Learning (nivel principiante).

Posee librerías específicas como SciPy, NumPy, Pandas, etc. que permiten desarrollar álgebra lineal y métodos kernel de manera sencilla.



freeCodeCamp (🔥)



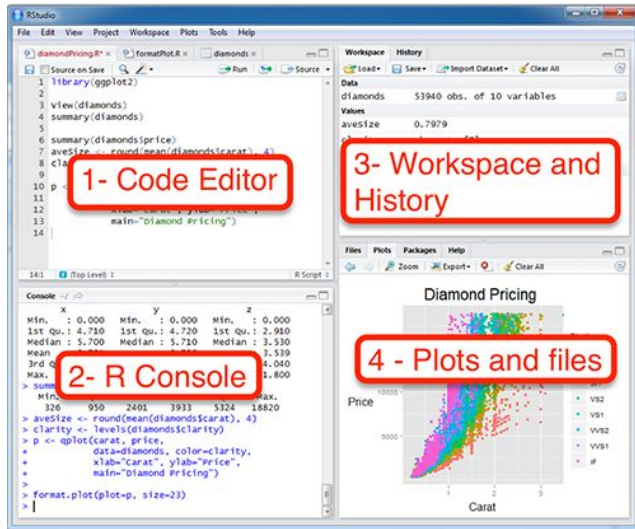
Lenguajes de Programación para IA

R (ar)



Está diseñado para análisis estadísticos y visualizaciones, se usa con frecuencia para desbloquear los patrones en grandes bloques de datos. »

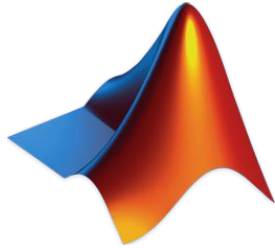
Es una alternativa gratuita a los costosos software estadísticos como **Matlab**.



```
6  
7 # gList Functions -----  
8  
9 gList <- function(...) {  
10   gl <- list(...)  
11   if (length(gl) == 0L ||  
12     all(sapply(gl, okGListelt, simplify=TRUE))) {  
13     # Ensure gList is "flat"  
14     # Don't want gList containing gList ...  
15     if (!all(sapply(gl, is.grob)))  
16       gl <- do.call("c", lapply(gl, as.gList))  
17     class(gl) <- c("gList")  
18     return(gl)  
19   } else {  
20     stop("Only 'grobs' allowed in 'gList'")  
21   }  
22 }
```

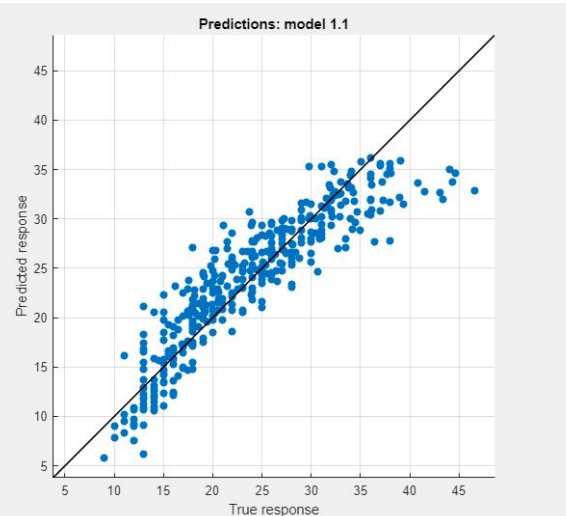
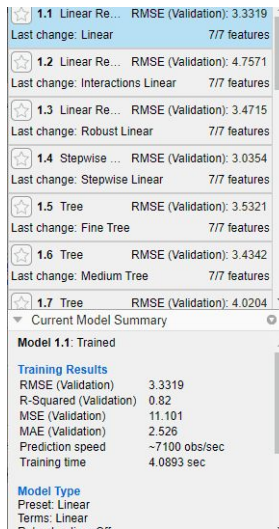
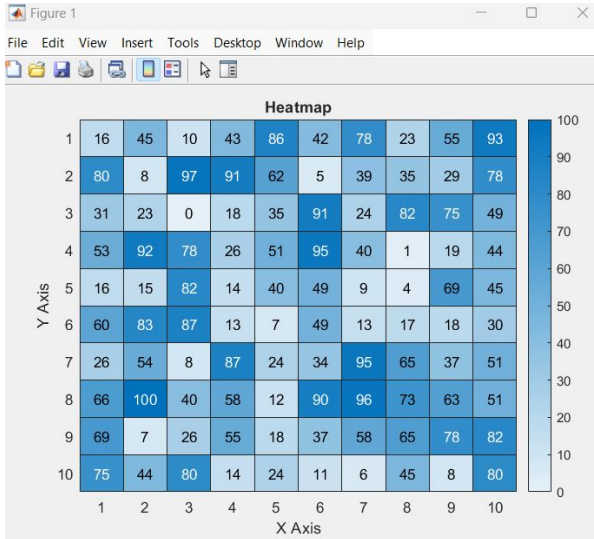
Lenguajes de Programación para IA

MATLAB



Es rápido, estable y seguro en el desarrollo de matemática compleja. >>>

Considerado como un lenguaje de núcleo duro para matemáticos y científicos. **Pero, tiene su costo económico.**



Lenguajes de Programación para IA



Julia



Diseñado para abordar las necesidades del análisis numérico de alto rendimiento y la ciencia computacional.



La librería base escrita en Julia se integró con las mejores librerías de código abierto C y Fortran (álgebra lineal, procesamiento de señales, etc).

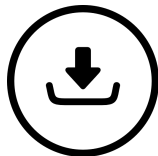
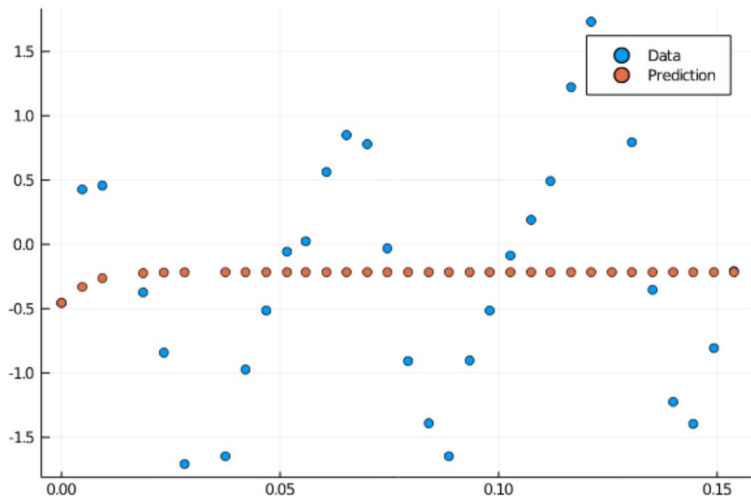
```
# Julia
```

```
m = [ 1 1; 1 -1 ] \ [ 2, 0 ]
```

```
# Python
```

```
import numpy as np
```

```
m = np.linalg.solve(  
    np.array([[1, 1], [1, -1]]),  
    np.array([2, 0]))
```



Lenguajes de Programación para IA



Velocidad

R fue desarrollado como lenguajes estadístico (suele ser el más lento).
Python depende de los paquetes instalados/consumidos.



Curva Aprendizaje

R por el lado de la perspectiva funcional.
Python el más complejo al usar el paradigma POO.



Costo Económico

El único lenguaje pago que necesita licencia para su uso es Matlab.



Comunidad

Todos son muy populares en el mercado y cuentan con un gran Respaldo de la comunidad. **Python es el más popular.**



Producción

Análisis estadístico →  Visión Computacional → 
Bioinformática/Biología →  Desarrollo General → 



IDE SPYDER

Scientific Python Development EnviRonment



Editor con finalización automática de código en tiempo real y definiciones de referencia. Visor de documentación, explorador de variables y una consola interactiva.

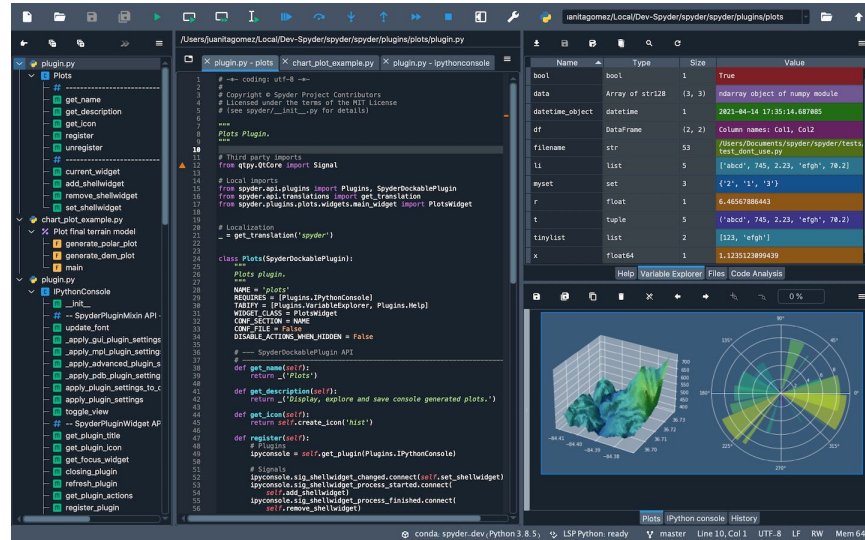
Está enfocado para el desarrollo de Data Science.
Se encuentra en el paquete de **Anaconda**.

Power



Logos shown above are trademarks of their respective owners

Ease of use





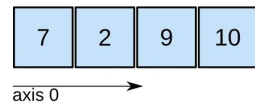
Librerías - NumPy

NumPy (NUMerical PYthon) es el paquete fundamental para la computación científica con Python. Contiene entre otras cosas:

- Estructura de datos principal: el arreglo N-dimensional
- Se puede integrar código en C/C++ y Fortran.
- Capacidades muy eficientes de álgebra lineal, transformación de Fourier y números aleatorios, etc.

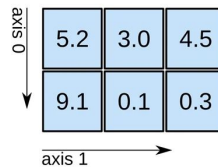
El principal beneficio de NumPy es que **permite una generación y manejo de datos extremadamente rápido.**

1D array



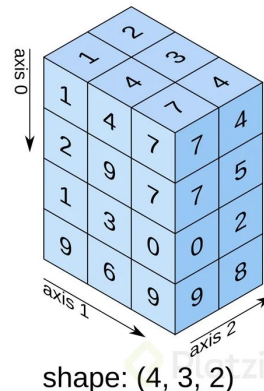
shape: (4,)

2D array



shape: (2, 3)

3D array

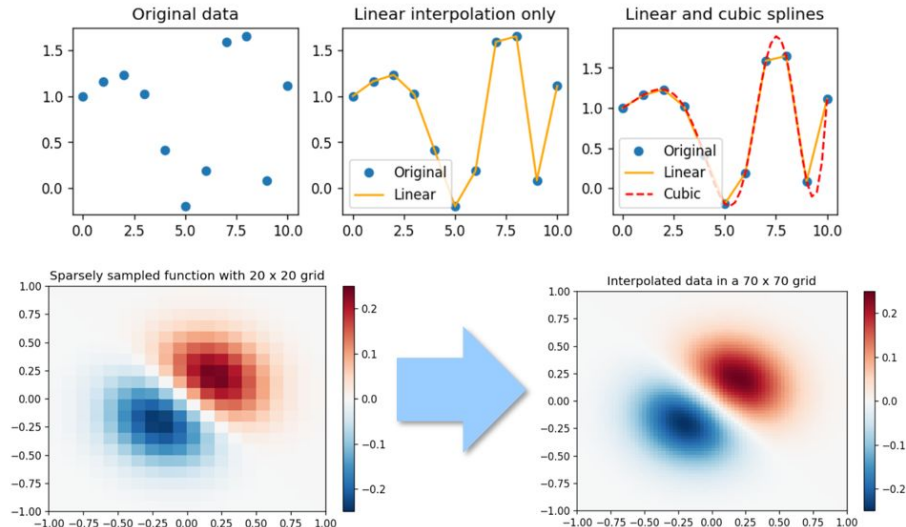




Librerías - SciPy

La biblioteca **SciPy** ("**Scientific Python**") está construida sobre NumPy y ofrece funciones científicas y estadísticas por encima de las funciones puramente matemáticas, por ejemplo:

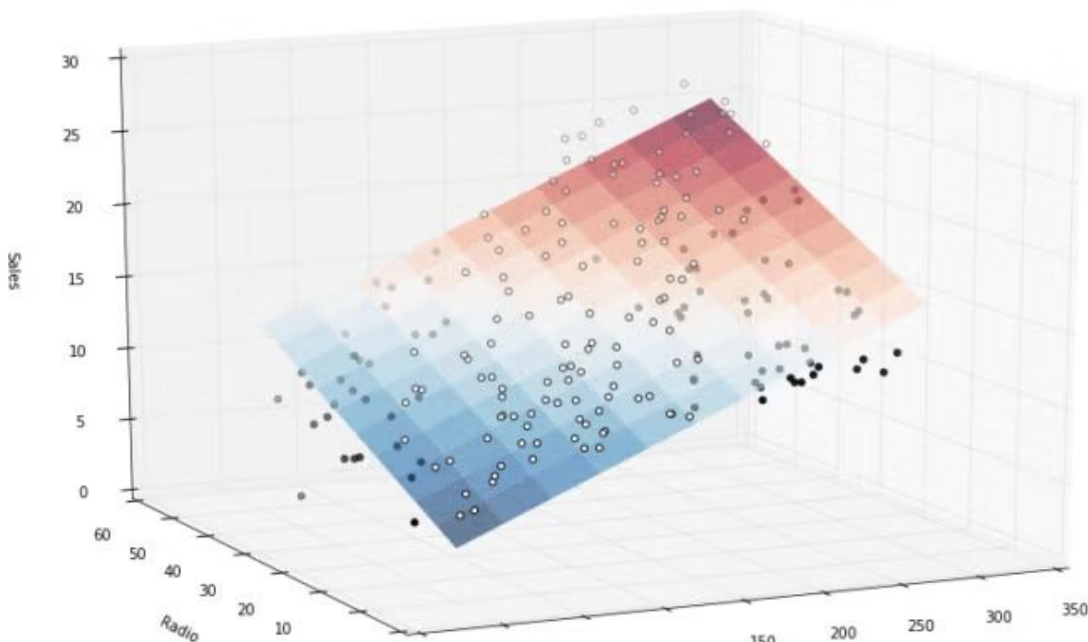
- Rutinas de álgebra lineal avanzada
- Optimización de funciones matemáticas
- Procesamiento de señales
- Distribuciones matemáticas



Librerías - Statsmodels

Una librería estadística que **está en auge hoy en día**, reemplazando a **Scipy** en lo que refiere a estadística es “Statsmodels”.

Es un módulo de Python que proporciona clases y funciones para la estimación de muchos modelos estadísticos diferentes, así como para realizar pruebas estadísticas y exploración de datos estadísticos.





Librerías - Pandas

Serie

Son estructuras 1D similares a los arrays de una dimensión.

Son homogéneas, es decir, sus elementos tienen que ser del **mismo tipo**, y su **tamaño es inmutable**, es decir, no se **puede cambiar**, aunque sí **su contenido**.

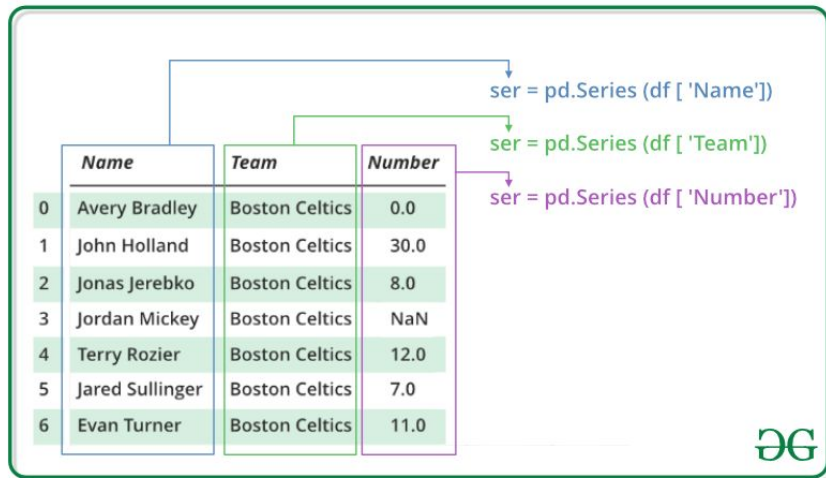
- Una Serie es similar a la **columna de una tabla** y es equivalente a una **columna de un DataFrame**.
- Dispone de un índice que asocia un nombre a cada elemento de la serie y a través de la cual se accede al elemento.

```
pd.Series([1, 2, 2, np.nan], index=['p', 'q', 'r', 's'])
```

	Data
p	1.0
q	2.0
r	2.0
s	NaN

dtype: float64

© w3resource.com





Librerías - Pandas

DataFrame

Un objeto del tipo DataFrame es una **colección bidimensional (2D)** ordenada en columnas con nombres y tipos, parecido a una tabla de Excel, en donde **cada columna es un objeto de tipo Series**, es decir, todos los datos de una misma columna son del mismo tipo, y las filas son registros que pueden contener datos de distintos tipos.

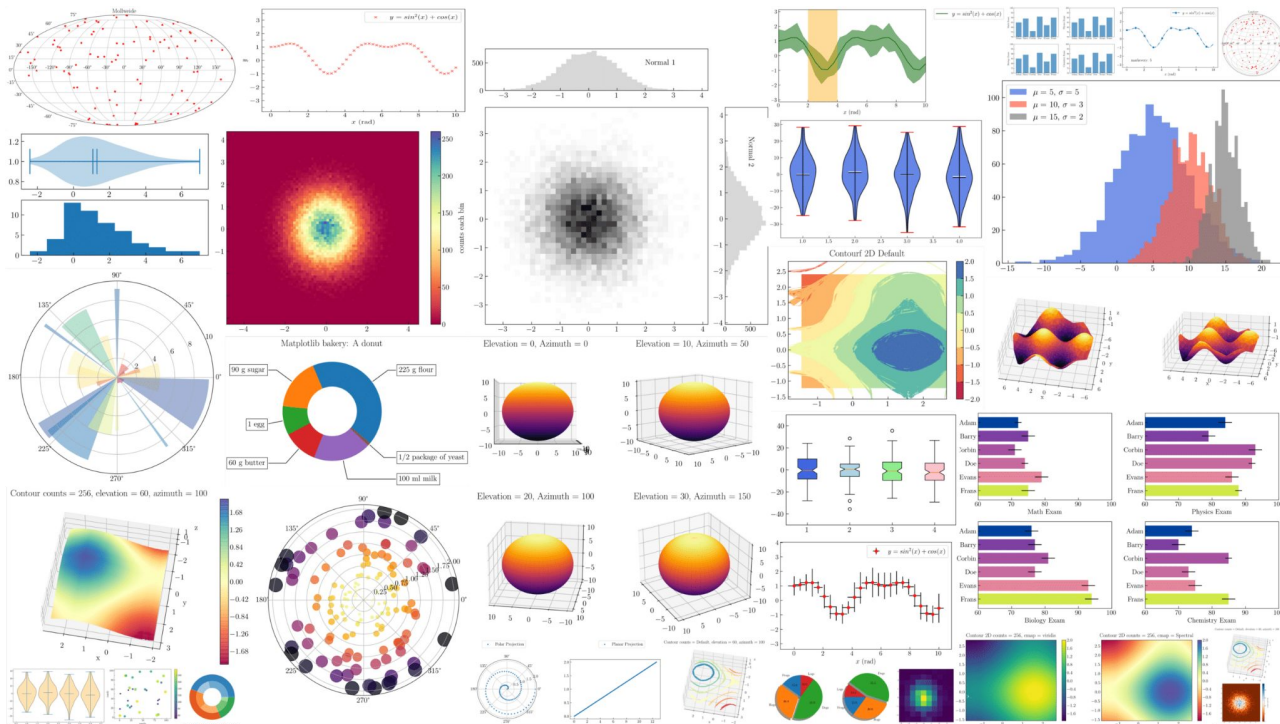
Un DataFrame **contiene dos índices**, uno para las filas y otro para las columnas, y se puede acceder a sus elementos mediante los nombres de las filas y las columnas.

Series 1			Series 2			Series 3			DataFrame			
Mango			Apple			Banana				Mango	Apple	Banana
0	4		0	5		0	2		0	4	5	2
1	5		1	4		1	3		1	5	4	3
2	6	+	2	3	+	2	5	=	2	6	3	5
3	3		3	0		3	2		3	3	0	2
4	1		4	2		4	7		4	1	2	7



Librerías - Matplotlib

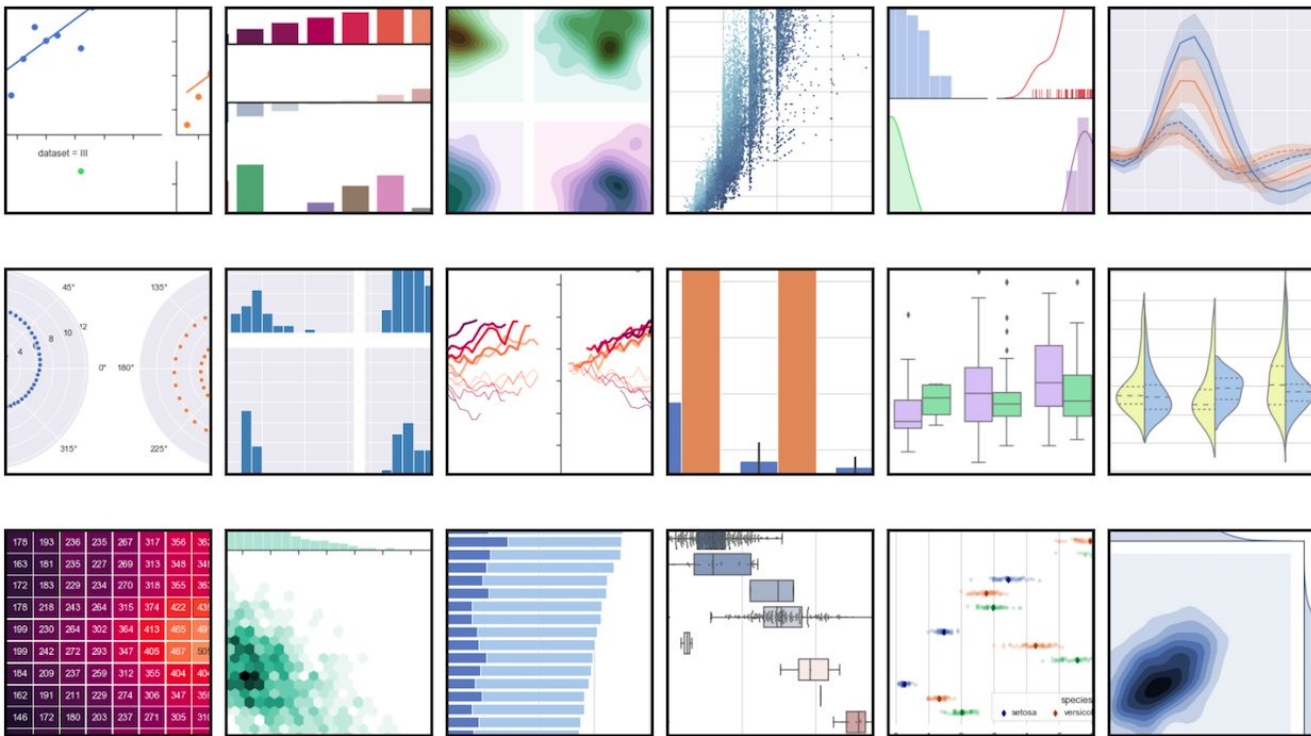
Matplotlib es una biblioteca de graficado para Python y NumPy. Proporciona una API orientada a objetos para incrustar gráficos en aplicaciones que utilizan kits de herramientas GUI de uso general como Tkinter, wxPython, Qt o GTK.





Librerías - Seaborn

Seaborn es una **biblioteca de visualización de datos** de Python basada en matplotlib. Proporciona una interfaz de alto nivel para dibujar gráficos estadísticos atractivos e informativos.

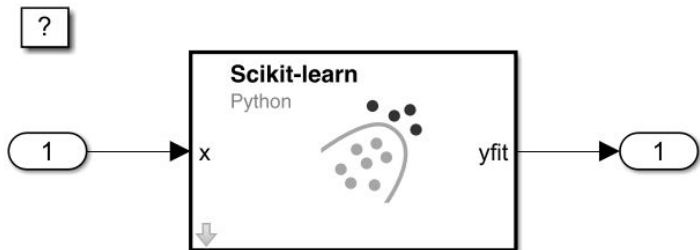




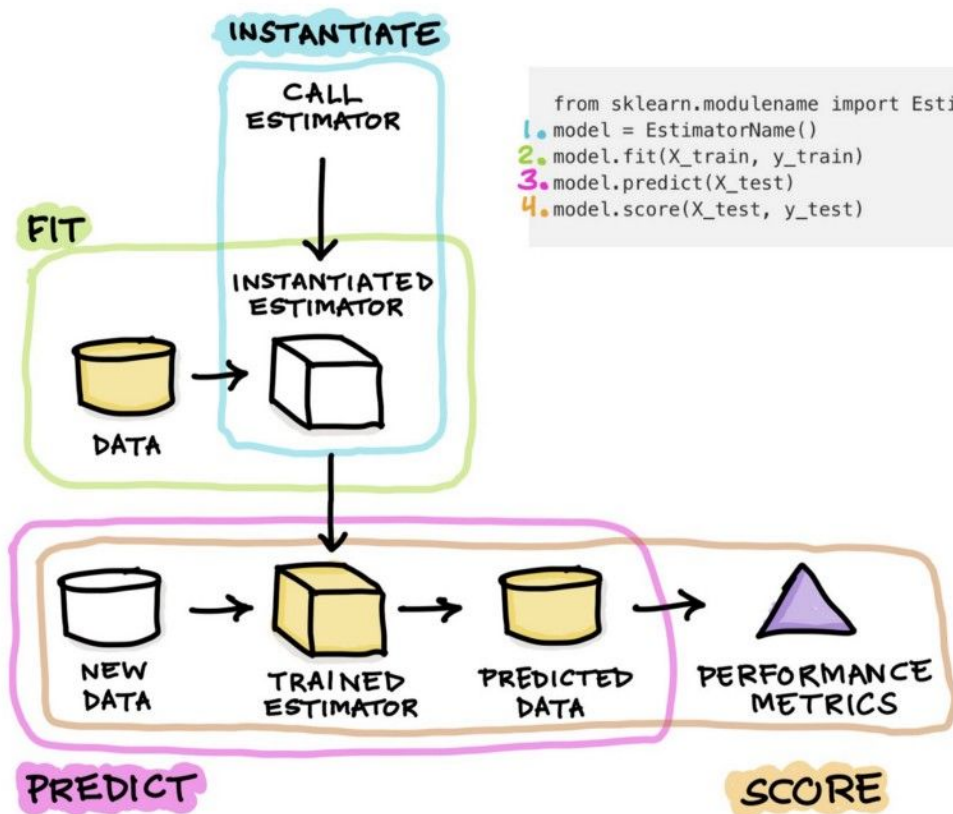
Librerías - Scikit-Learn

Es una biblioteca muy útil cuando se trabaja en **algoritmos de Machine Learning** en Python. Proporciona herramientas de procesamiento de datos y una gama de algoritmos de aprendizaje en Python.

Es la puerta de entrada al aprendizaje automático en Python.



Copyright 2024 The MathWorks, Inc.



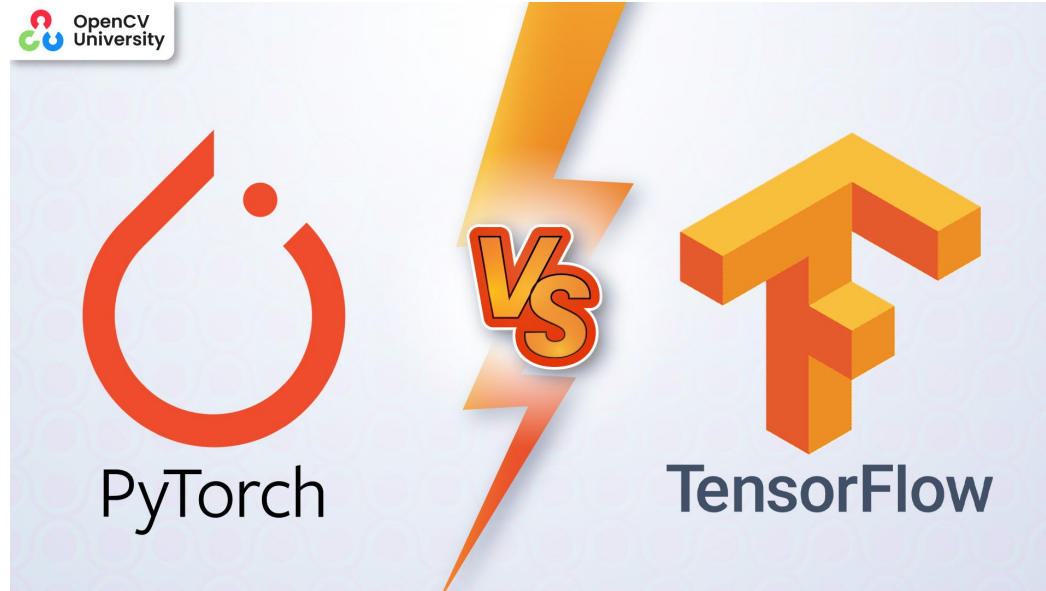
```
from sklearn.modulename import EstimatorName
1. model = EstimatorName()
2. model.fit(X_train, y_train)
3. model.predict(X_test)
4. model.score(X_test, y_test)
```

Librerías - PyTorch

PyTorch es una librería de **Deep Learning**. Es mantenida por Meta y es la principal competencia de otra famosa librería (tensorflow de Google).

Tiene un API para Python, como también para C++. PyTorch nos da toda una infraestructura de:

- Computación de tensores. Es similar a los arrays de Numpy pero con posibilidad de usarlos en GPU.
- Redes neuronales profundas.



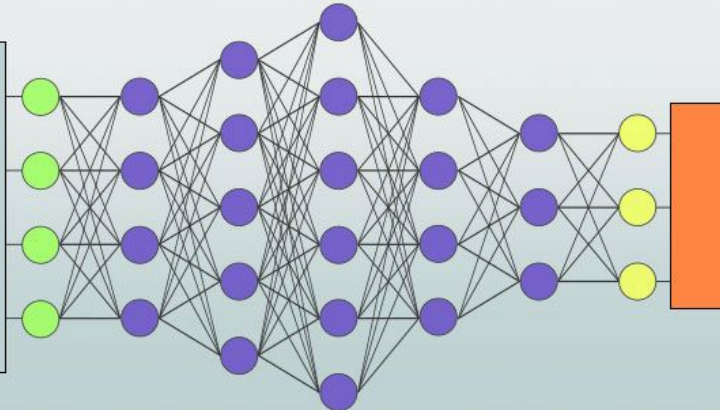
TensorFlow

Es un **framework** de código abierto desarrollado por **Google** que facilita la construcción y el entrenamiento de modelos de **Machine Learning** y **Deep Learning**.

Es una herramienta muy popular que se utiliza para implementar algoritmos de **aprendizaje automático**, ya que permite trabajar con estructuras de datos llamadas "**tensores**" de forma eficiente.



Tensorflow Image Classification



TensorFlow

Tensores:

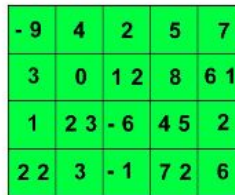
Estructuras de datos similares a **matrices** o **arreglos** multidimensionales.

Los tensores pueden representar **datos de diferentes dimensiones** (escalar, vector, matriz, etc.) y son la forma en que los datos se procesan en TensorFlow.

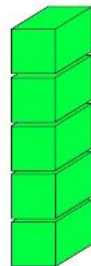
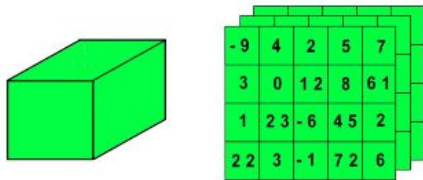
1D TENSOR /
VECTOR



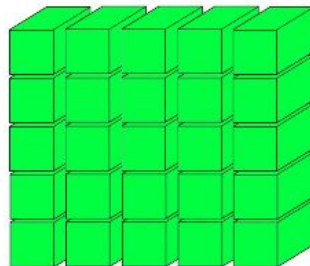
2D TENSOR /
MATRIX



3D TENSOR /
CUBE



4D TENSOR
VECTOR OF CUBES



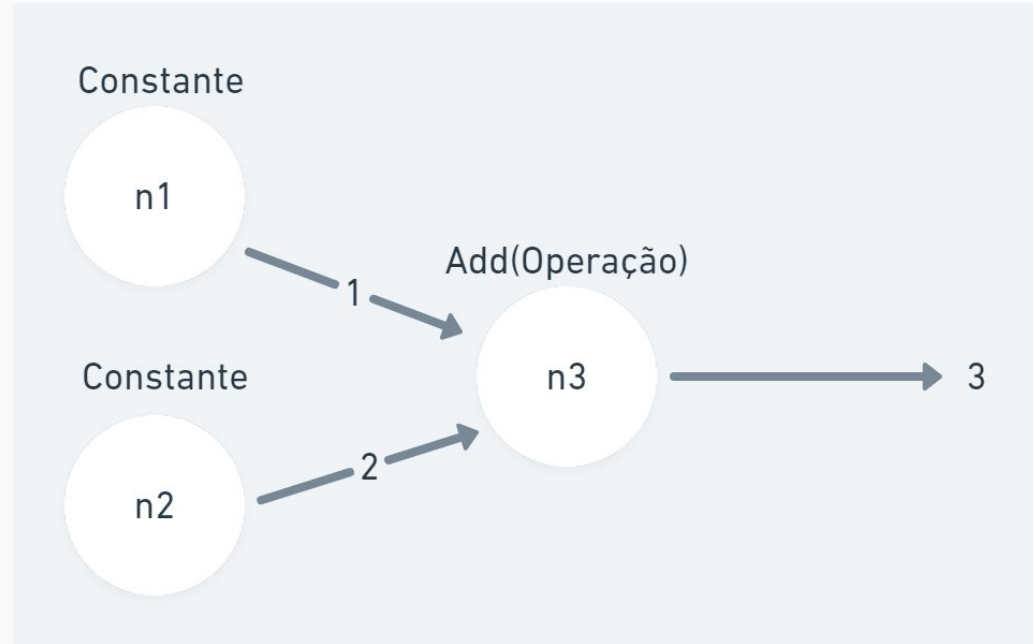
5D TENSOR
MATRIX OF CUBES

TensorFlow

Graficado de Computación:

TensorFlow utiliza un **grafo** de computación, donde los **nodos** representan **operaciones matemáticas** y las **aristas** son **los tensores que fluyen entre ellas**.

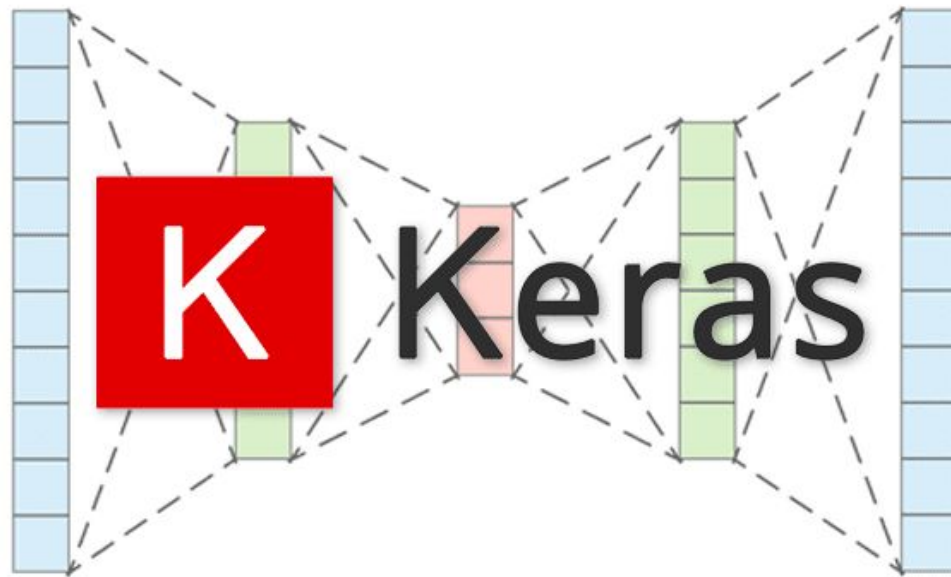
Esto permite ejecutar los cálculos de manera eficiente y paralela.

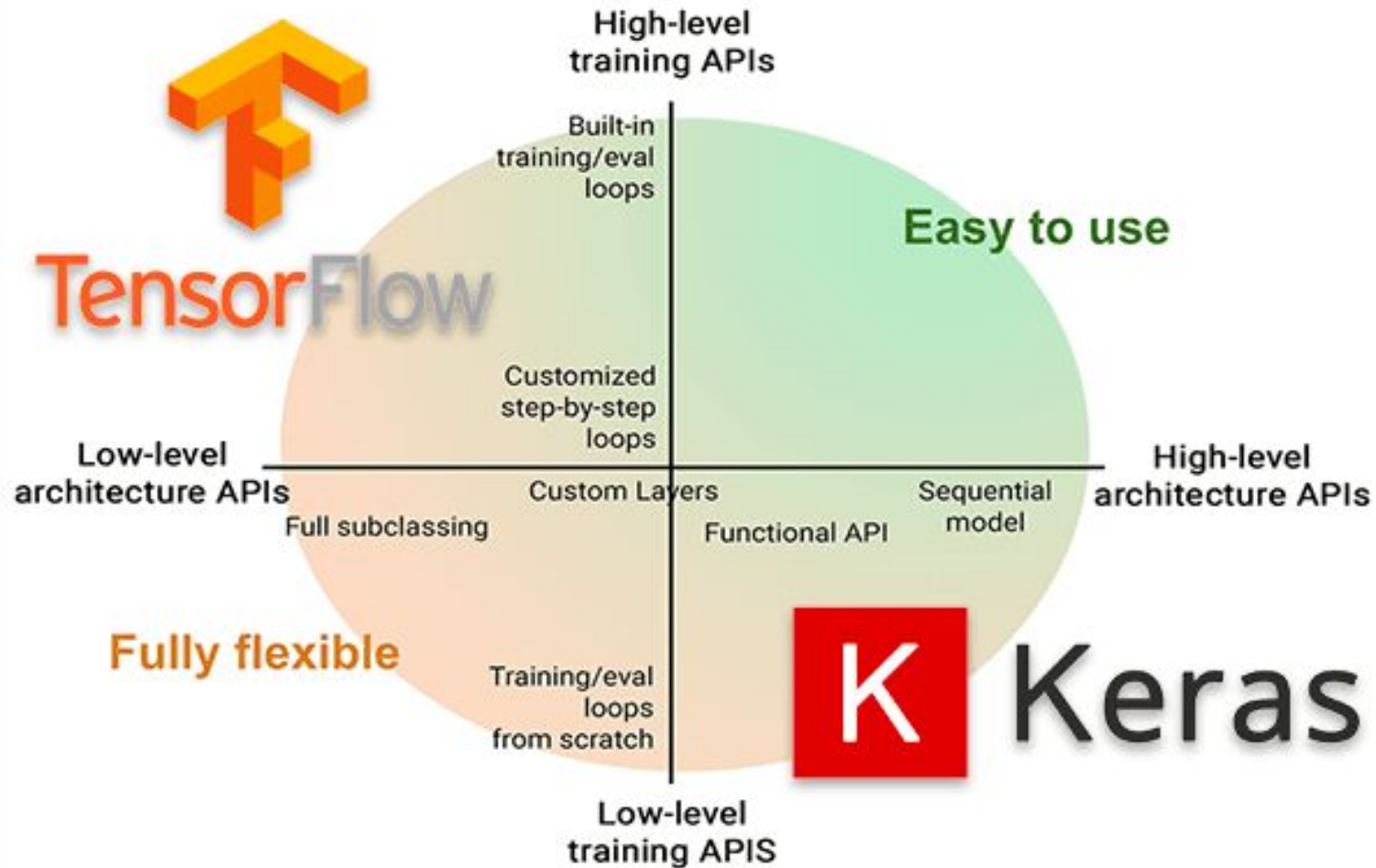


Keras

Es una **biblioteca** de alto nivel que funciona como una **interfaz** para **construir** y **entrenar modelos** de Machine Learning y Deep Learning de forma más sencilla e intuitiva.

Está integrada dentro de TensorFlow y permite **crear redes neuronales** con pocas líneas de código, haciendo que la implementación de modelos complejos sea mucho más accesible.





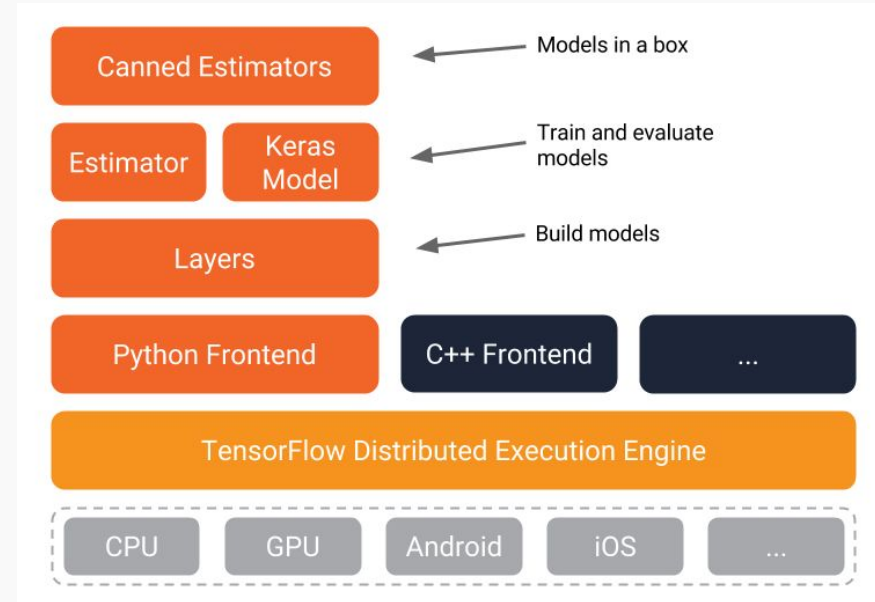
Actualmente:

TensorFlow es un **framework** de aprendizaje automático de bajo nivel, potente y flexible, diseñado para computación numérica de alto rendimiento, entrenamiento distribuido y escalabilidad masiva.

Keras es una **API** de alto nivel diseñada para ser extremadamente fácil de usar, modular y permitir una experimentación rápida. Su objetivo es simplificar la construcción de redes neuronales.

Cuando utilizamos **TensorFlow 2.x o posterior**, **Keras** está **integrado de forma nativa** como su interfaz oficial y recomendada. Se accede a él mediante **tf.keras**.

Ya no es necesario instalar Keras por separado al tener TensorFlow instalado.





[]

03

Live Coding (Pair Programming)

{ }

Python

Python es uno de los lenguajes más populares y utilizados para dar instrucciones a las computadoras.



Python

Python posee un sintaxis simple, lo que significa que es fácil de escribir, leer y aprender.



Python VS C++



Python Advantages

01

Easy to learn

02

Easy to access Libraries

03

Scientific community sharing (open source, many libraries)

Python Disadvantages

Slow

Interpreted(dependencies)

Not Typed

C++ Advantages

01

Execution speed

02

Pre-Compiled

03

Typed(well defined)

04

Modern Professional Libraries

C++ Disadvantages

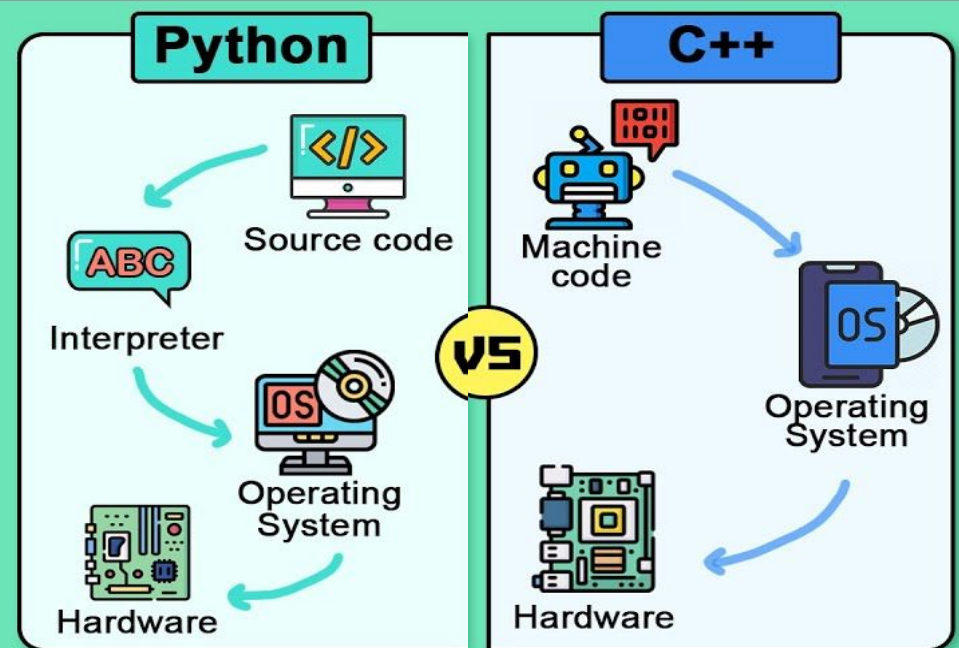
Learning curve

Harder to access Libraries



GLOBAL TECH™
COUNCIL

www.globaltechcouncil.org



Colab



Colab es un servicio alojado de Jupyter Notebook que no requiere configuración y que ofrece acceso gratuito a recursos de computación, como GPUs. Colab es una solución especialmente adecuada para el aprendizaje automático, la ciencia de datos y la educación.



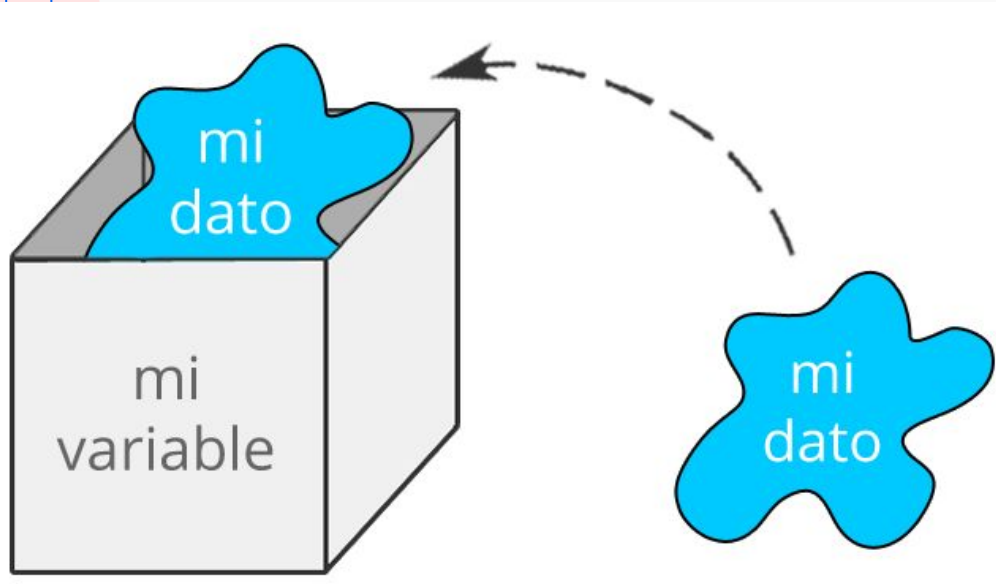
<https://colab.google/>



Memoria y Variables



Son espacios reservados en memoria (RAM) para poder almacenar datos. Poseen un nombre, un valor y un tipo.



```
ciudad = "Cordoba"
```

Nombre de la variable: **ciudad**
Valor de la variable: **Cordoba**



Memoria y Variables



Reglas para crear variables

1- No puede tener espacios.



[]

fecha de nacimiento

fechaDeNacimiento

2- No puede empezar con un número.

2daVariable

variable2da

3- No puede ser una palabra reservada (comando).

print, if, while, for, etc

snake_case

camelCase



[]

Tipos de Datos



Texto: Entre comillas dobles o simples



[]

“Soy un texto”

Numerico: No lleva comillas

Enteros: Sin punto decimal. Pueden ser positivos, negativos o cero.

123

-65

Flotante: Con punto decimal. Pueden ser positivos o negativos.

1.98

-9.7

Booleano: Verdadero o Falso

True

False

[]

Live Coding

El **live coding** es un método de enseñanza en el que los instructores escriben código durante las clases, mientras piensan en voz alta para que los alumnos puedan observar el proceso de programación.



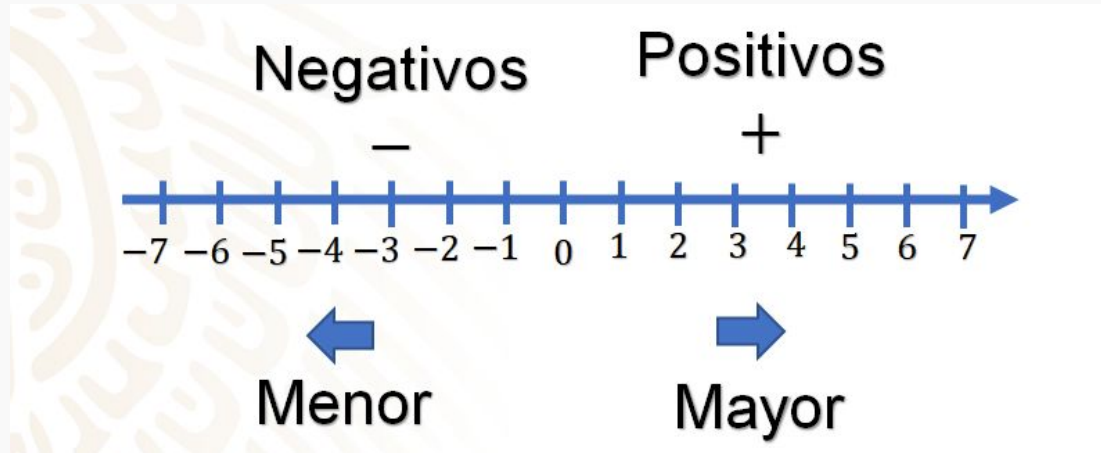


Trabajaremos en Google Colab

Introducción a Python

Ejercicio 1

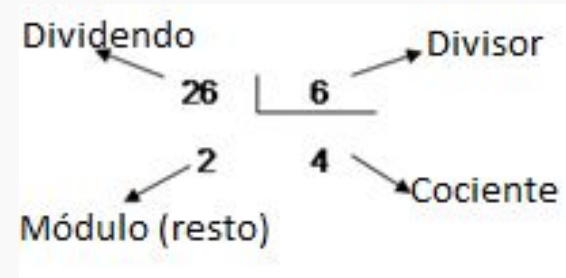
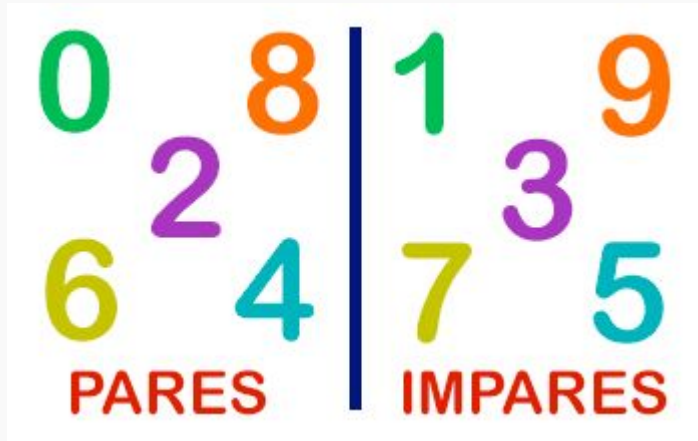
Escribir un programa que solicite al usuario un número e imprima si es positivo, negativo o cero.



{ }

Ejercicio 2

Escribir un programa que solicite al usuario un número e imprima si es par o impar.



Módulo	%	a % b 10 % 3 a % 3	{ }
--------	---	--------------------------	-----

Ejercicio 3

Se pide desarrollar un software para un dispositivo médico que informa a los pacientes sobre sus niveles de azúcar en la sangre.

`nivel_glucosa = 60` -> Nivel de glucosa bajo

`nivel_glucosa = 100` -> Rango normal (entre 60 y 150)

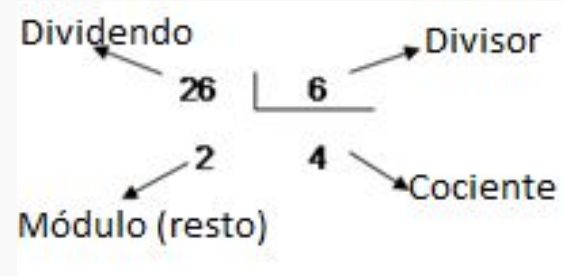
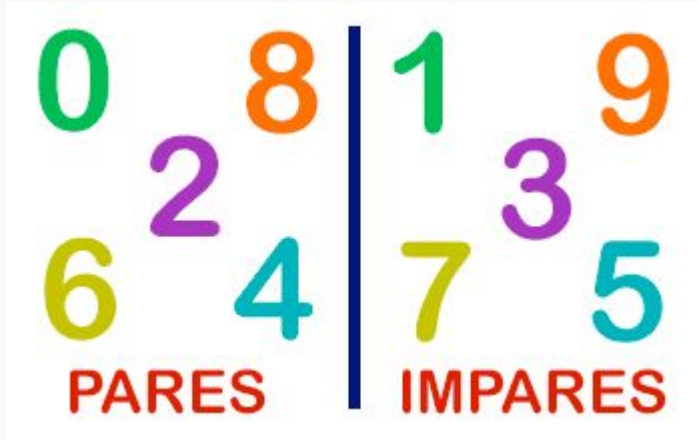
`nivel_glucosa = 150` -> Nivel de glucosa alto

{ }

Ejercicio 4

Realizar un programa que calcule la suma de números pares ingresados por el usuario. Siendo un total de 10 números ingresados.

Si contador % 2 == 0 Entonces "ES PAR"



Módulo	%	a % b 10 % 3 a % 3	{ }
--------	---	--------------------------	-----



Trabajaremos en Google Colab

Listas en Python



Ejercicio 5

Dada una lista con nombres, crear una nueva lista sin duplicados, manteniendo el orden original. Sin usar “Set()”

```
nombres = ['Ana', 'Luis', 'Ana', 'Pedro', 'Luis', 'María']
```

El resultado debe ser:

```
nombres = ['Ana', 'Luis', 'Pedro', 'María']
```

{ }

Ejercicio 6

Encontrar el segundo mayor número.

Dada una lista de números, encontrar el segundo valor más grande sin usar "sort()".

```
valores = [10, 5, 8, 20, 15]
```

Resultado esperado:

15

{ }



Trabajaremos en Google Colab

Funciones en Python

Ejercicio 7

Escribir una función llamada **contar_pares(lista)** que reciba una lista de números y devuelva cuántos números pares hay en esa lista.

Ejemplo:

contar_pares([1, 2, 3, 4, 6]) → 3

{ }

Ejercicio 8

Escribir una función **promedio_positivos(lista)** que devuelva el promedio solo de los números mayores a 0. Si no hay ninguno positivo, que devuelva 0.

Ejemplo:

promedio_positivos([-2, 5, 8, -1]) → 6.5

{ }

Ejercicio 9

Escribir una función **filtrar_palabras(lista, letra)** que reciba una lista de palabras y una letra, y devuelva una lista con las palabras que empiezan con esa letra (sin importar si es mayúscula o minúscula).

Ejemplo:

```
filtrar_palabras(["perro", "gato", "pato", "pez"], "p")  
→ ['perro', 'pato', 'pez']
```

NOTA: Pueden usar **.startswith()**

```
texto = "Hola mundo"
```

```
print(texto.startswith("Hola"))    # True
```

{ }

¿Ustedes que piensan?



Veán con atención la imagen:

- ¿Son los mismos objetos?
- ¿Qué características tienen en común?
- ¿En qué se diferencian?
- ¿Cómo los clasificarían/agruparían?



Sedan



Wagon/Minivan/MPV



SUV



Hatchback



Cabriolet/Coupe



Sport/Supercar



Pickup



4WD



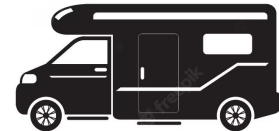
Mini Truck



Micro/Electric



Van



Campervan

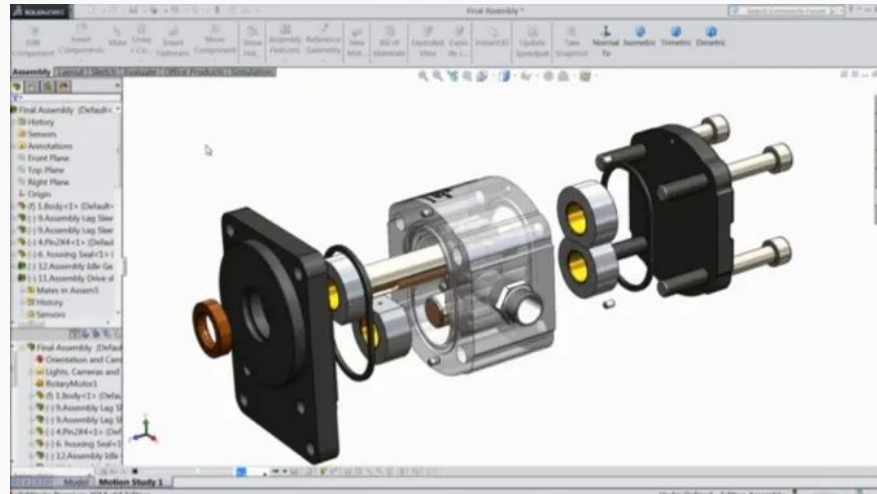


¿Qué es un Paradigma de programación?

Es una manera o estilo de programación. Existen diferentes formas de diseñar un programa y varios modos de trabajar para obtener los resultados que necesitan los programadores.

La Programación Orientada a Objetos (POO) es un paradigma que modela elementos del mundo real como objetos que tienen propiedades y comportamientos. Esto permite representar entidades y sus características para crear programas más intuitivos.

También permite organizar el código en módulos o clases reutilizables, y sus respectivas instancias y objetos, lo que facilita la creación y mantenimiento de programas complejos.



El Paradigma de Orientación a Objetos

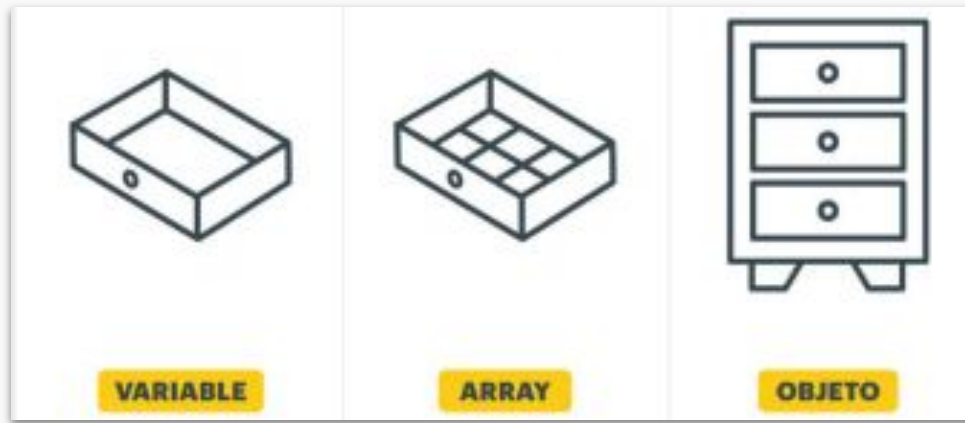


La Programación Orientada a objetos permite que el código sea reutilizable, organizado y fácil de mantener.



[]

Sigue el principio de desarrollo de software utilizado por muchos programadores **DRY (Don't Repeat Yourself)**, para evitar duplicar el código y crear de esta manera programas eficientes.



[]

Clases

Representa a una agrupación de “cosas”. **Es una plantilla para “crear” objetos**



nombre: *Jengi*



nombre: *Doris*



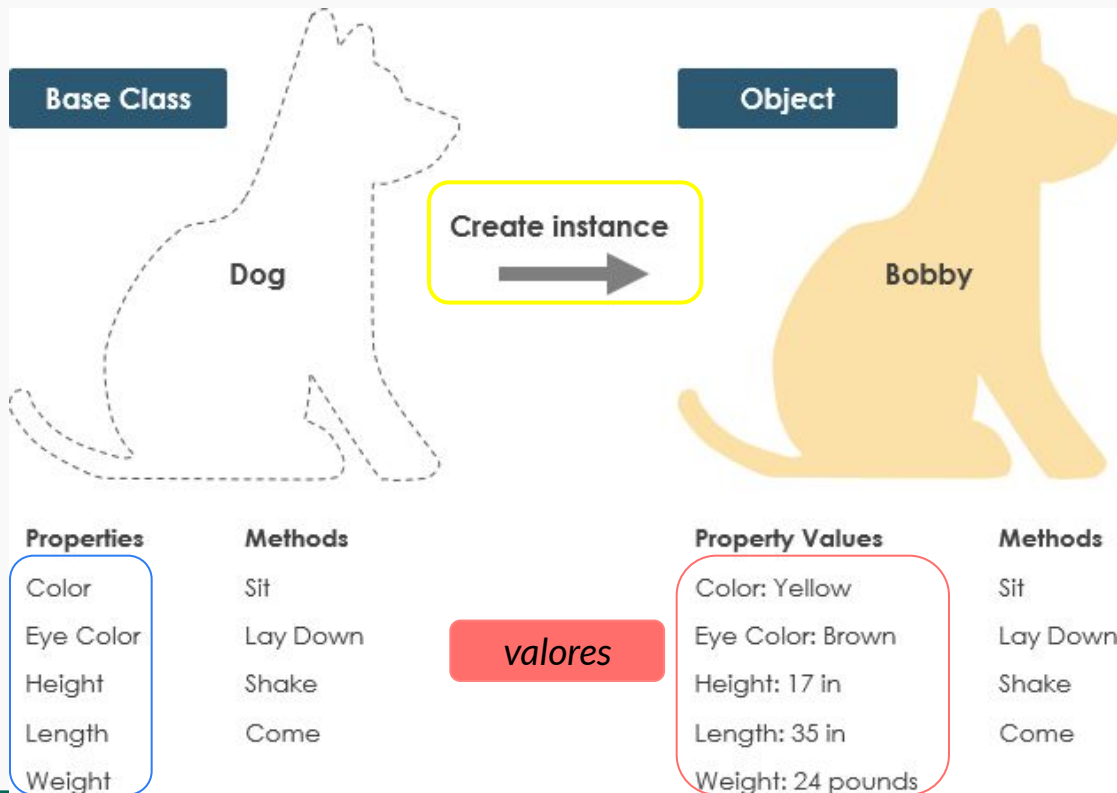
nombre: *Pepito*



Clases

Son las plantillas o moldes que utilizaremos para crear objetos.

Definen las **propiedades** y **comportamientos** que los objetos de esa clase tendrán.



Entonces...

¿Qué es un objeto?

Es un modelo que representa una entidad del mundo real



Atributos

CARACTERISTICAS

adjetivos

Métodos

RESPONSABILIDADES /
OPERACIONES

verbos



Atributos

CARACTERISTICAS

adjetivos

altura: 1.3m

peso: 30kg

color: negro

animal: ratón

nombre: Mickey



Métodos

OPERACIONES

verbos

altura: 1.3m

peso: 30kg

color: negro

animal: ratón

nombre: Mickey

hablar

caminar

cantar

saltar

bailar



Contexto
Disney



altura: 1.3m

hablar

peso: 30kg

caminar

color: negro

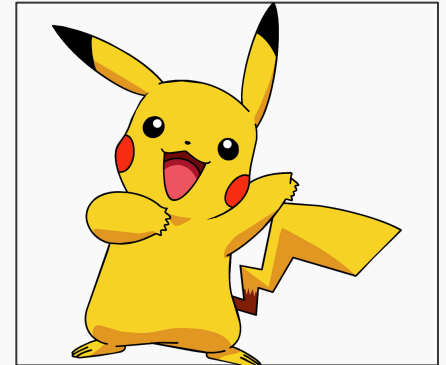
cantar

animal: ratón

saltar

nombre: Mickey

bailar



Objetos

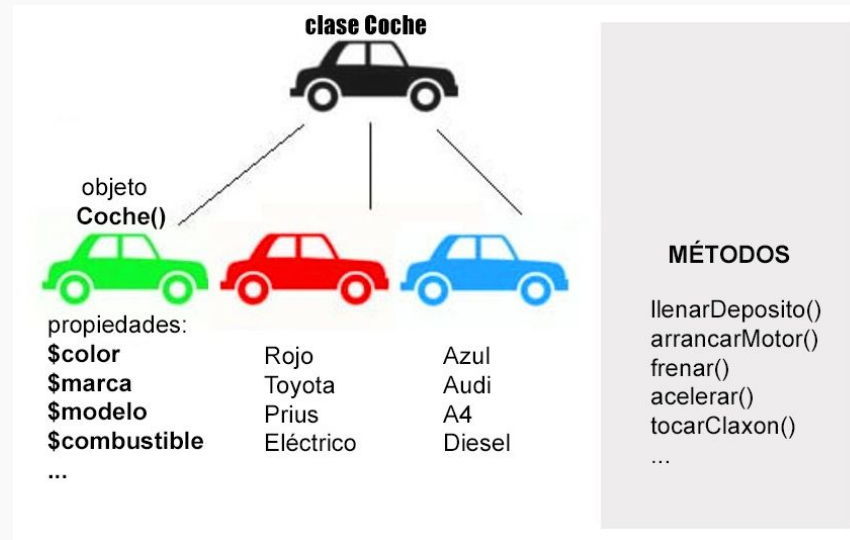


Son instancias específicas de una clase . Se crean a partir de la clase y tienen su propio estado y comportamiento.



Estado : viene dado por sus atributos, que almacenan los datos del objeto. Por ejemplo, el color y marca de un coche.

Comportamiento : viene dado por sus métodos que son las operaciones que puede realizar el objeto. Por ejemplo, el coche puede frenar y acelerar.



Métodos de una Clase

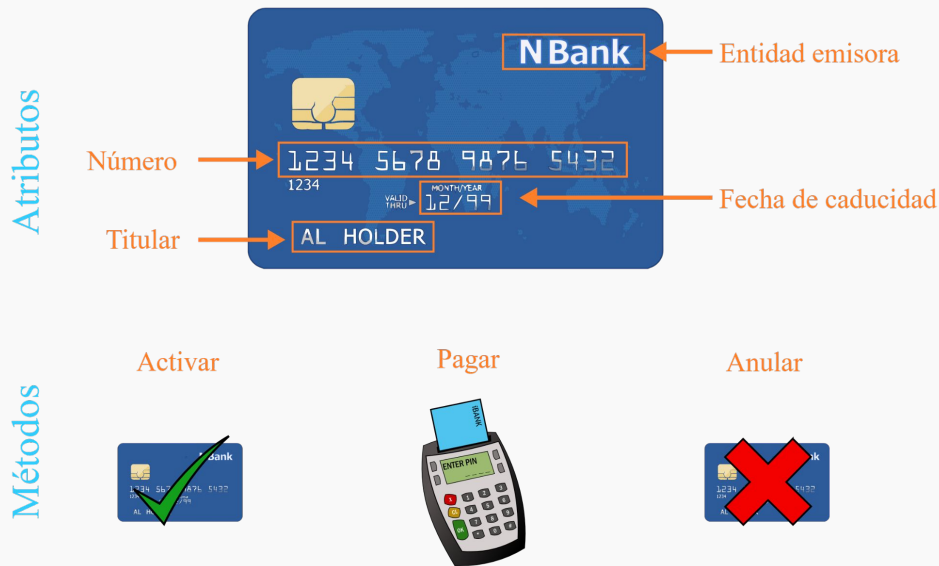


Son **funciones** o **procedimientos** que representan el **comportamiento** o las acciones que los objetos de esa clase pueden realizar.



[]

Los métodos pueden aceptar parámetros como entrada, realizar cálculos, **modificar el estado de los objetos** y devolver o no resultados como salida.



[]

Métodos Constructores



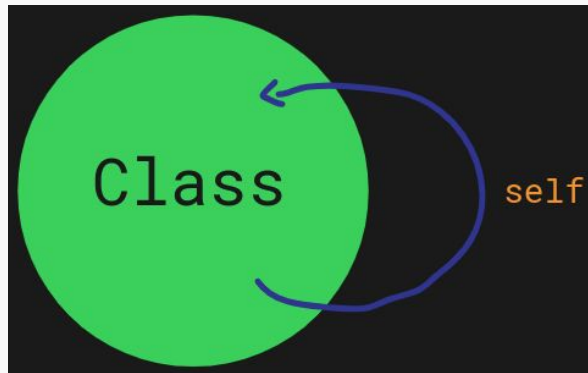
Los **constructores** son métodos **propios del objeto** que permiten la creación de dicho objeto. A la creación de un objeto se le denomina **instanciación**.



Los constructores se encargan de dar un **estado inicial** a nuestro objeto.

En Python, el constructor se define mediante un método llamado **`__init__`**

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```



Métodos Constructores



El método constructor se ejecuta cada vez que se instancia un objeto de la clase. Este método se utiliza para inicializar los atributos del objeto que se crea.



[]

La instancia de un objeto consiste en **asignar un espacio de memoria** al que se hace referencia con el nombre del objeto.

```
class Time:
    def __init__(self, x, y):
        if(x > 10):
            self.x = x
        else:
            self.x = 0

        if(y > 20):
            self.y = y
        else:
            self.y = 0
```

[]

Métodos Constructores



Una vez que se ha declarado una clase, se pueden crear objetos a partir de ella. A la creación de un objeto se le denomina instanciación. Por esta razón el término instancia y objeto se utilizan indistintamente.



()

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person1 = Person("David", 30)
```

{}]



Trabajaremos en Google Colab

POO en Python



Herencia



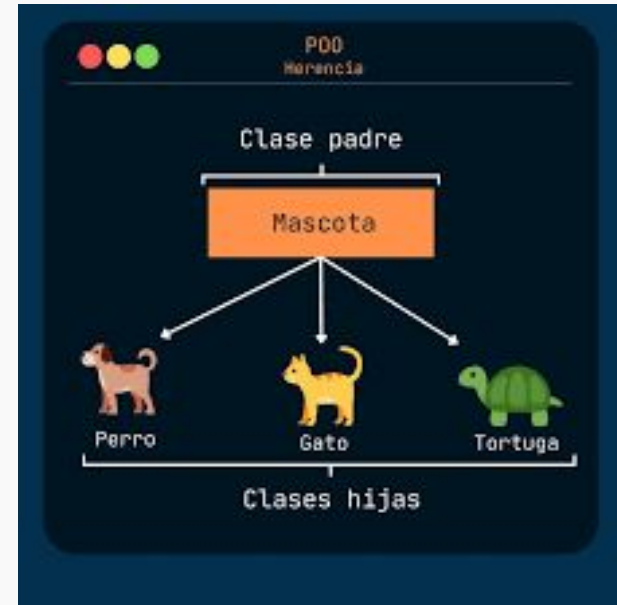
La herencia es un proceso mediante el cual se puede crear una clase hija que hereda de una clase padre, compartiendo sus métodos y atributos. Además de ello, una clase hija puede sobrescribir los métodos o atributos, o incluso definir unos nuevos.



[]

```
# Definimos una clase padre
class Animal:
    pass

# Creamos una clase hija que hereda de la padre
class Perro(Animal):
    pass
```



Encapsulamiento



Hace referencia al ocultamiento de los estado internos de una clase al exterior.
Encapsular consiste en hacer que los atributos o métodos internos a una clase no se puedan acceder ni modificar desde fuera, sino que tan solo el propio objeto pueda acceder a ellos.



[]

"Python por defecto no oculta los atributos y métodos de una clase al exterior"

```
class Clase:
    atributo_clase = "Hola"
    def __init__(self, atributo_instancia):
        self.atributo_instancia = atributo_instancia
```

```
mi_clase = Clase("Que tal")
mi_clase.atributo_clase
mi_clase.atributo_instancia
```

```
# 'Hola'
# 'Que tal'
```



Ambos atributos son
perfectamente accesibles
desde el exterior.

[]

Encapsulamiento



Las variables que comienzan por un doble guión bajo `__` la podemos considerar como atributos privados. Veamos un ejemplo:



```
>>> class Alumno():  
...     def __init__(self,nombre=""):  
...         self.nombre=nombre  
...         self.__secreto="asdasd"  
...
```

```
>>> a1=Alumno("jose")
```

```
>>> a1.__secreto
```

No podemos acceder
porque es privado

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

AttributeError: 'Alumno' object has no attribute '__secreto'

Tipo de Acceso \ Se accede desde	PRIVATE	PROTECTED	PUBLIC
La misma clase	Sí	Sí	Sí
Clases hijas	No	Sí	Sí
Clases externas	No	No	Sí



Encapsulamiento



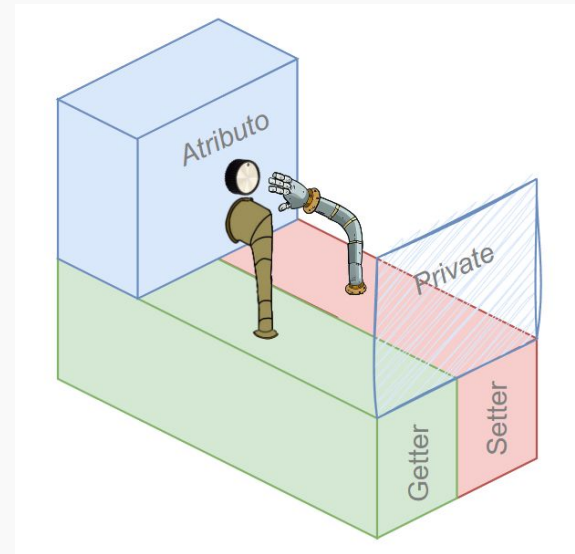
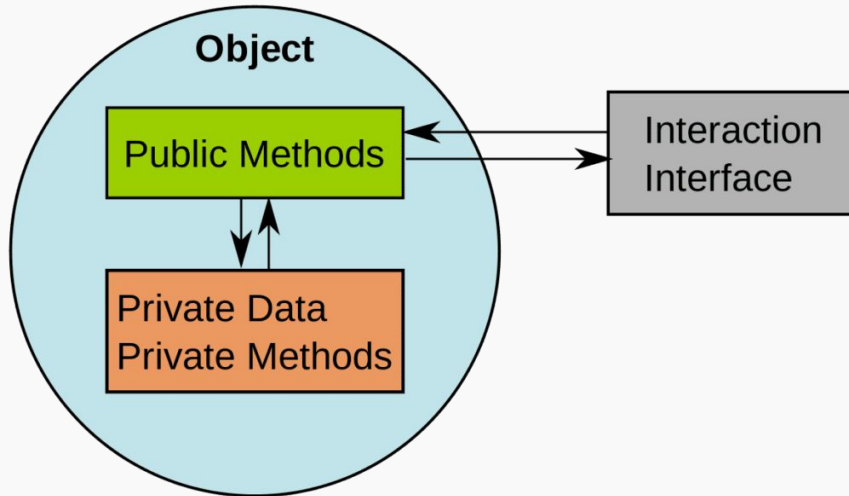
Para implementar la encapsulación y no permitir el acceso directo a los atributos, podemos poner los atributos ocultos y declarar métodos específicos para acceder y modificar los atributos.



[]

En Python, las propiedades (getters) nos permiten implementar la funcionalidad exponiendo estos métodos como atributos.

Los métodos setters son métodos que nos permiten modificar los atributos a través de un método.



[]

Propiedades: getters y setters



```
class circulo():
```

```
    def __init__(self,radio):  
        self.radio=radio
```

```
@property
```

```
def radio(self):  
    print("Estoy dando el radio")  
    return self.__radio
```

```
@radio.setter
```

```
def radio(self,radio):  
    if radio>=0:  
        self.__radio = radio  
    else:  
        print("Radio debe ser positivo")  
        self.__radio=0
```

Un **getter** (obtenido a través del decorador `@property`)  es un método que se ejecuta automáticamente cuando leemos un atributo.

- Por ejemplo, al hacer `obj.radio`, se llama internamente al método que definimos con `@property`.

Un **setter** (definido con `@atributo.setter`) es un método que se ejecuta automáticamente cuando asignamos un valor a ese atributo.

- Por ejemplo, al hacer `obj.radio = 3`, se llama internamente al método que definimos con `@radio.setter`.



Propiedades: getters y setters



```
class circulo():
    def __init__(self,radio):
        self.radio=radio

    @property
    def radio(self):
        print("Estoy dando el radio")
        return self.__radio

    @radio.setter
    def radio(self,radio):
        if radio>=0:
            self.__radio = radio
        else:
            print("Radio debe ser positivo")
            self.__radio=0
```

```
>>> c1=circulo(3)
>>> c1.radio
Estoy dando el radio
3
>>> c1.radio=4
>>> c1.radio=-1
Radio debe ser positivo
>>> c1.radio
0
```





Trabajaremos en Google Colab

Herencia en Python

Decoradores



Haciendo uso de los **decoradores**, es posible crear diferentes tipos de métodos.
Los decoradores son funciones que **modifican el comportamiento de otras funciones**.



```
[ ]  
def mi_decorador(funcion):  
    def nueva_funcion(a, b):  
        print("Se va a llamar")  
        c = funcion(a, b)  
        print("Se ha llamado")  
        return c  
    return nueva_funcion  
  
@mi_decorador  
def suma(a, b):  
    print("Entra en funcion suma")  
    return a + b  
  
suma(5,8)  
  
# Se va a llamar  
# Entra en funcion suma  
# Se ha llamado
```

Lo que realiza **mi_decorador()** es definir una nueva función que encapsula o envuelve la función que se pasa como entrada. Concretamente lo que hace es: hace uso de dos **print()**, uno antes y otro después de la llamada la función.

Por lo tanto, cualquier función que use **@mi_decorador** tendrá dos print, uno y al principio y otro al final, dando igual lo que realmente haga la función.



Tipos de Métodos



Haciendo uso de los decoradores, es posible crear diferentes tipos de métodos:

- Lo métodos de instancia “normales” que ya hemos visto como `metodo()`
- Métodos de clase usando el decorador `@classmethod`
- Y métodos estáticos usando el decorador `@staticmethod`



[]

```
class Clase:
    def metodo(self):
        return 'Método normal', self

    @classmethod
    def metododeclase(cls):
        return 'Método de clase', cls

    @staticmethod
    def metodoestatico():
        return "Método estático"
```

[]

Métodos de clase (classmethod)



A diferencia de los métodos de instancia, los métodos de clase reciben como argumento `cls`, que hace referencia a la clase. Por lo tanto, pueden acceder a la clase pero no a la instancia.



[]

```
class Clase:
    @classmethod
    def metododeclase(cls):
        return 'Método de clase', cls
```

No usamos self

Se pueden llamar sobre la clase.

```
Clase.metododeclase()
# ('Método de clase', __main__.Clase)
```

Pero también se pueden llamar sobre el objeto.

```
mi_clase.metododeclase()
# ('Método de clase', __main__.Clase)
```

Por lo tanto, los métodos de clase:

- **No** pueden acceder a los atributos de la instancia.
- Pero **si pueden** modificar los atributos de la clase.

[]

Métodos estáticos (staticmethod)



Los métodos estáticos se pueden definir con el decorador `@staticmethod` y no aceptan como parámetro ni la instancia ni la clase. Es por ello por lo que **no pueden modificar el estado ni de la clase ni de la instancia**. Pero por supuesto pueden aceptar parámetros de entrada.



```
class Clase:
    @staticmethod
    def metodoestatico():
        return "Método estático"
```

```
mi_clase = Clase()
Clase.metodoestatico()
mi_clase.metodoestatico()

# 'Método estático'
# 'Método estático'
```

Por lo tanto el uso de los métodos estáticos pueden resultar útil para indicar que un método **no modificará el estado de la instancia ni de la clase**. Es cierto que se podría hacer lo mismo con un método de instancia por ejemplo, pero a veces resulta importante indicar de alguna manera estas peculiaridades, evitando así futuros problemas y malentendidos. Ej. Para Interfaces.

En otras palabras, **los métodos estáticos se podrían ver como funciones normales, con la salvedad de que van ligadas a una clase concreta.**

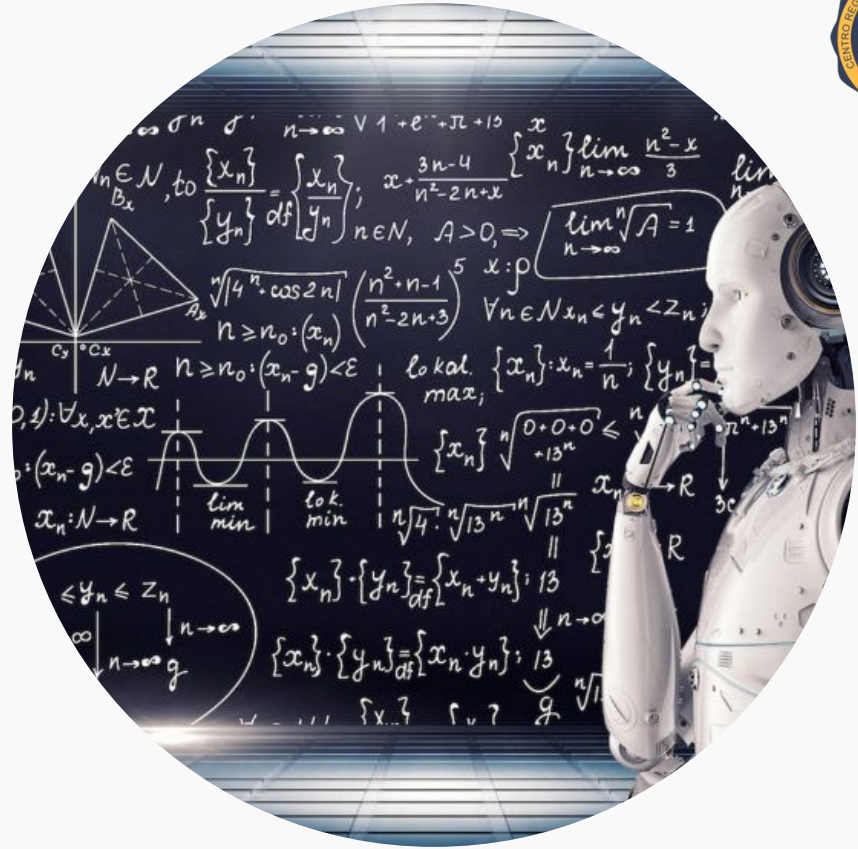


Trabajaremos en Google Colab

Métodos Especiales en Python

¡Gracias!

¿Consultas?



Ing. González H. Alexis